

End-to-End Training of Deep Visuomotor Policies

Sergey Levine[†]

Chelsea Finn[†]

Trevor Darrell

Pieter Abbeel

Division of Computer Science

University of California

Berkeley, CA 94720-1776, USA

[†]These authors contributed equally.

SVLEVINE@EECS.BERKELEY.EDU

CBFINN@EECS.BERKELEY.EDU

TREVOR@EECS.BERKELEY.EDU

PABBEEL@EECS.BERKELEY.EDU

Editor: Jan Peters

Abstract

Policy search methods can allow robots to learn control policies for a wide range of tasks, but practical applications of policy search often require hand-engineered components for perception, state estimation, and low-level control. In this paper, we aim to answer the following question: does training the perception and control systems jointly end-to-end provide better performance than training each component separately? To this end, we develop a method that can be used to learn policies that map raw image observations directly to torques at the robot's motors. The policies are represented by deep convolutional neural networks (CNNs) with 92,000 parameters, and are trained using a guided policy search method, which transforms policy search into supervised learning, with supervision provided by a simple trajectory-centric reinforcement learning method. We evaluate our method on a range of real-world manipulation tasks that require close coordination between vision and control, such as screwing a cap onto a bottle, and present simulated comparisons to a range of prior policy search methods.

Keywords: Reinforcement Learning, Optimal Control, Vision, Neural Networks

1. Introduction

Robots can perform impressive tasks under human control, including surgery (Lanfranco et al., 2004) and household chores (Wyrobek et al., 2008). However, designing the perception and control software for autonomous operation remains a major challenge, even for basic tasks. Policy search methods hold the promise of allowing robots to automatically learn new behaviors through experience (Kober et al., 2010b; Deisenroth et al., 2011; Kalakrishnan et al., 2011; Deisenroth et al., 2013). However, policies learned using such methods often rely on a number of hand-engineered components for perception and control, so as to present the policy with a more manageable and low-dimensional representation of observations and actions. The vision system in particular can be complex and prone to errors, and it is typically not improved during policy training, nor adapted to the goal of the task.

In this article, we aim to answer the following question: can we acquire more effective policies for sensorimotor control if the perception system is trained jointly with the control policy, rather than separately? In order to represent a policy that performs both

End-to-End Training of Deep Visuomotor Policies

Sergey Levine[†]

Chelsea Finn[†]

Trevor Darrell

Pieter Abbeel

Division of Computer Science

University of California

Berkeley, CA 94720-1776, USA

[†]这些作者的贡献是平等的。

SVLEVINE@EECS.BERKELEY.EDU

CBFINN@EECS.BERKELEY.EDU

TREVOR@EECS.BERKELEY.EDU

PABBEEL@EECS.BERKELEY.EDU

Editor: 简·彼得斯

Abstract

策略搜索方法可以让机器人学习各种任务的控制策略，但策略搜索的实际应用通常需要手工设计的组件来进行感知、状态估计和低级控制。在本文中，我们旨在回答以下问题：端到端联合训练感知和控制系统是否比单独训练每个组件提供更好的性能？为此，我们开发了一种方法，可用于学习将原始图像观察结果直接映射到机器人电机扭矩的策略。这些策略由具有 92,000 个参数的深度卷积神经网络 (CNN) 表示，并使用引导策略搜索方法进行训练，该方法将策略搜索转变为监督学习，并由简单的以轨迹为中心的强化学习方法提供监督。我们在一系列需要视觉和控制之间密切协调的现实世界操作任务（例如将盖子拧到瓶子上）上评估我们的方法，并与一系列先前的策略搜索方法进行模拟比较。

Keywords: 强化学习、最优控制、视觉、神经网络

1. Introduction

机器人可以在人类控制下执行令人印象深刻的任务，包括手术 (Lanfranco 等人, 2004) 和家务活 (Wyrobek 等人, 2008)。然而，设计用于自主操作的感知和控制软件仍然是一个重大挑战，即使对于基本任务也是如此。策略搜索方法有望让机器人通过经验自动学习新行为 (Kober et al., 2010b; Deisenroth et al., 2011; Kalakrishnan et al., 2011; Deisenroth et al., 2013)。然而，使用这种方法学习的策略通常依赖于许多手工设计的组件来进行感知和控制，以便为策略提供更易于管理和低维的观察和行动表示。视觉系统尤其复杂且容易出错，并且通常在政策训练期间不会得到改进，也不会适应任务的目标。

在本文中，我们旨在回答以下问题：如果感知系统与控制策略联合训练，而不是单独训练，我们能否获得更有效的感觉运动控制策略？为了表示执行两者的策略

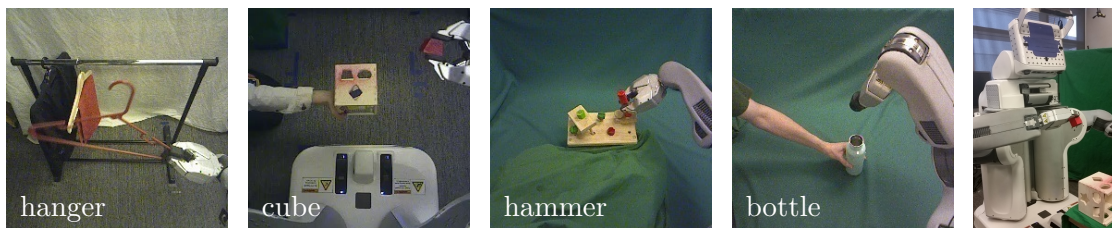


Figure 1: Our method learns visuomotor policies that directly use camera image observations (left) to set motor torques on a PR2 robot (right).

perception and control, we use deep neural networks. Deep neural network representations have recently seen widespread success in a variety of domains, such as computer vision and speech recognition, and even playing video games. However, using deep neural networks for real-world sensorimotor policies, such as robotic controllers that map image pixels and joint angles to motor torques, presents a number of unique challenges. Successful applications of deep neural networks typically rely on large amounts of data and direct supervision of the output, neither of which is available in robotic control. Real-world robot interaction data is scarce, and task completion is defined at a high level by means of a cost function, which means that the learning algorithm must determine on its own which action to take at each point. From the control perspective, a further complication is that observations from the robot’s sensors do not provide us with the full state of the system. Instead, important state information, such as the positions of task-relevant objects, must be inferred from inputs such as camera images.

We address these challenges by developing a guided policy search algorithm for sensorimotor deep learning, as well as a novel CNN architecture designed for robotic control. Guided policy search converts policy search into supervised learning, by iteratively constructing the training data using an efficient model-free trajectory optimization procedure. We show that this can be formalized as an instance of Bregman ADMM (BADMM) (Wang and Banerjee, 2014), which can be used to show that the algorithm converges to a locally optimal solution. In our method, the full state of the system is observable at training time, but not at test time. For most tasks, providing the full state simply requires positioning objects in one of several known positions for each trial during training. At test time, the learned CNN policy can handle novel, unknown configurations, and no longer requires full state information. Since the policy is optimized with supervised learning, we can use standard methods like stochastic gradient descent for training. Our CNNs have 92,000 parameters and 7 layers, including a novel spatial feature point transformation that provides accurate spatial reasoning and reduces overfitting. This allows us to train our policies with relatively modest amounts of data and only tens of minutes of real-world interaction time.

We evaluate our method by learning policies for inserting a block into a shape sorting cube, screwing a cap onto a bottle, fitting the claw of a toy hammer under a nail with various grasps, and placing a coat hanger on a rack with a PR2 robot (see Figure 1). These tasks require localization, visual tracking, and handling complex contact dynamics. Our results demonstrate improvements in consistency and generalization from training visuomotor policies end-to-end, when compared to training the vision and control components separately. We also present simulated comparisons that show that guided policy search outperforms a

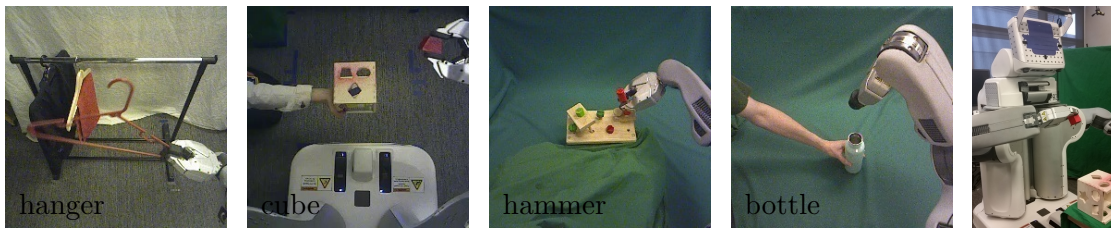


图 1: 我们的方法学习视觉运动策略, 直接使用相机图像观察 (左) 来设置 PR2 机器人 (右) 上的电机扭矩。

感知和控制, 我们使用深度神经网络。深度神经网络表示最近在多个领域取得了广泛的成功, 例如计算机视觉和语音识别, 甚至玩视频游戏。然而, 将深度神经网络用于现实世界的感觉运动策略, 例如将图像像素和关节角度映射到电机扭矩的机器人控制器, 提出了许多独特的挑战。深度神经网络的成功应用通常依赖于大量数据和对输出的直接监督, 而这两者在机器人控制中都不可用。现实世界的机器人交互数据稀缺, 任务完成是通过成本函数在高层定义的, 这意味着学习算法必须自行确定在每个点采取哪个动作。从控制的角度来看, 更加复杂的是机器人传感器的观察结果并不能为我们提供系统的完整状态。相反, 重要的状态信息 (例如与任务相关的对象的位置) 必须从相机图像等输入中推断出来。

我们通过开发用于感觉运动深度学习的引导策略搜索算法以及专为机器人控制设计的新 CNN 架构来应对这些挑战。引导策略搜索通过使用高效的无模型轨迹优化过程迭代构建训练数据, 将策略搜索转换为监督学习。我们证明这可以形式化为 Bregman ADMM (BADMM) 的一个实例 (Wang 和 Banerjee, 2014), 它可以用来表明算法收敛到局部最优解。在我们的方法中, 系统的完整状态在训练时可以观察到, 但在测试时则不能。对于大多数任务, 提供完整状态只需在训练期间将对象定位在每次试验的几个已知位置之一。在测试时, 学习到的 CNN 策略可以处理新颖的未知配置, 并且不再需要完整的状态信息。由于策略是通过监督学习来优化的, 因此我们可以使用随机梯度下降等标准方法进行训练。我们的 CNN 有 92,000 个参数和 7 层, 包括一种新颖的空间特征点变换, 可提供准确的空间推理并减少过度拟合。这使我们能够使用相对少量的数据和仅数十分钟的现实交互时间来训练我们的策略。

我们通过学习以下策略来评估我们的方法: 将块插入形状分类立方体、将瓶盖拧到瓶子上、使用各种抓握将玩具锤的爪子安装在钉子下, 以及使用 PR2 机器人将衣架放在架子上 (见图 1)。这些任务需要定位、视觉跟踪和处理复杂的接触动态。我们的结果表明, 与单独训练视觉和控制组件相比, 端到端训练视觉运动策略在一致性和泛化方面有所提高。我们还提出了模拟比较, 表明引导策略搜索优于

number of prior methods when training high-dimensional neural network policies. Some of the material in this article has previously appeared in two conference papers (Levine and Abbeel, 2014; Levine et al., 2015), which we extend to introduce visual input into the policy.

2. Related Work

Reinforcement learning and policy search methods (Gullapalli, 1990; Williams, 1992) have been applied in robotics for playing games such as table tennis (Kober et al., 2010b), object manipulation (Gullapalli, 1995; Peters and Schaal, 2008; Kober et al., 2010a; Deisenroth et al., 2011; Kalakrishnan et al., 2011), locomotion (Benbrahim and Franklin, 1997; Kohl and Stone, 2004; Tedrake et al., 2004; Geng et al., 2006; Endo et al., 2008), and flight (Ng et al., 2004). Several recent papers provide surveys of policy search in robotics (Deisenroth et al., 2013; Kober et al., 2013). Such methods are typically applied to one component of the robot control pipeline, which often sits on top of a hand-designed controller, such as a PD controller, and accepts processed input, for example from an existing vision pipeline (Kalakrishnan et al., 2011). Our method learns policies that map visual input and joint encoder readings directly to the torques at the robot’s joints. By learning the entire mapping from perception to control, the perception layers can be adapted to optimize task performance, and the motor control layers can be adapted to imperfect perception.

We represent our policies with convolutional neural networks (CNNs). CNNs have a long history in computer vision and deep learning (Fukushima, 1980; LeCun et al., 1989; Schmidhuber, 2015), and have recently gained prominence due to excellent results on a number of vision benchmarks (Ciresan et al., 2011; Krizhevsky et al., 2012; Ciresan et al., 2012; Girshick et al., 2014a; Tompson et al., 2014; LeCun et al., 2015; He et al., 2015). Most applications of CNNs focus on classification, where locational information is discarded by means of successive pooling layers to provide for invariance (Lee et al., 2009). Applications to localization typically either use a sliding window (Girshick et al., 2014a) or object proposals (Endres and Hoiem, 2010; Uijlings et al., 2013; Girshick et al., 2014b) to localize the object, reducing the task to classification, perform regression to a heatmap of manually labeled keypoints (Tompson et al., 2014), requiring precise knowledge of the object position in the image and camera calibration, or use 3D models to localize previously scanned objects (Pepik et al., 2012; Savarese and Fei-Fei, 2007). Many prior robotic applications of CNNs do not directly consider control, but employ CNNs for the perception component of a larger robotic system (Hadsell et al., 2009; Sung et al., 2015; Lenz et al., 2015b; Pinto and Gupta, 2015). We use a novel CNN architecture for our policies that automatically learn feature points that capture spatial information about the scene, without any supervision beyond the information from the robot’s encoders and camera.

Applications of deep learning in robotic control have been less prevalent in recent years than in visual recognition. Backpropagation through the dynamics and the image formation process is typically impractical, since they are often non-differentiable, and such long-range backpropagation can lead to extreme numerical instability, since the linearization of a suboptimal policy is likely to be unstable. This issue has also been observed in the related context of recurrent neural networks (Hochreiter et al., 2001; Pascanu and Bengio, 2012). The high dimensionality of the network also makes reinforcement learning difficult (Deisenroth et al., 2013). Pioneering early work on neural network control used

训练高维神经网络策略时现有方法的数量。本文中的一些材料之前曾出现在两篇会议论文中 (Levine 和 Abbeel, 2014 年; Levine 等人, 2015 年), 我们将其扩展以将视觉输入引入到政策中。

2. Related Work

强化学习和策略搜索方法 (Gullapalli, 1990; Williams, 1992) 已应用于玩乒乓球等游戏的机器人 (Kober 等, 2010b)、对象操作 (Gullapalli, 1995; Peters 和 Schaal, 2008; Kober 等, 2010a; Deisenroth 等, 2011; Kalakrishnan 等, 2011)、运动 (Benbrahim 和 Franklin, 1997; Kohl 和 Stone, 2004; Tedrake 等, 2004; Geng 等, 2006; Endo 等, 2008) 和飞行 (Ng 等, 2004)。最近的几篇论文提供了机器人技术中策略搜索的调查 (Deisenroth 等人, 2013 年; Kober 等人, 2013 年)。此类方法通常应用于机器人控制管道的一个组件, 该组件通常位于手动设计的控制器 (例如 PD 控制器) 之上, 并接受处理后的输入, 例如来自现有视觉管道的输入 (Kalakrishnan 等人, 2011)。我们的方法学习将视觉输入和关节编码器读数直接映射到机器人关节处的扭矩的策略。通过学习从感知到控制的整个映射, 感知层可以适应优化任务性能, 运动控制层可以适应不完美的感知。

我们用卷积神经网络 (CNN) 来表示我们的策略。CNN 在计算机视觉和深度学习方面有着悠久的历史 (Fukushima, 1980; LeCun 等人, 1989; Schmidhuber, 2015), 并且最近由于在许多视觉基准上的出色结果而受到关注 (Ciresan 等人, 2011; Krizhevsky 等人, 2012; Ciresan 等人, 2012; Girshick 等人, 2014a; 汤普森等人, 2014; LeCun 等人, 2015; CNN 的大多数应用都集中在分类上, 其中通过连续的池化层丢弃位置信息以提供不变性 (Lee 等, 2009)。本地化应用通常使用滑动窗口 (Girshick et al., 2014a) 或对象建议 (Eldredge and Hoiem, 2010; Uijlings et al., 2013; Girshick et al., 2014b) 来本地化对象, 减少分类任务, 对手动标记关键点的热图执行回归 (Tompson et al., 2014), 需要精确了解图像中的物体位置和相机校准, 或使用 3D 模型来定位先前扫描的物体 (Pepik 等人, 2012 年; Savarese 和 Fei-Fei, 2007 年)。许多先前的 CNN 机器人应用并不直接考虑控制, 而是将 CNN 用于大型机器人系统的感知组件 (Hadsell 等人, 2009; Sung 等人, 2015; Lenz 等人, 2015b; Pinto 和 Gupta, 2015)。我们的策略使用了一种新颖的 CNN 架构, 可以自动学习捕获场景空间信息的特征点, 除了来自机器人编码器和摄像头的信息之外, 无需任何监督。

近年来, 深度学习在机器人控制中的应用不如视觉识别中普遍。通过动力学和图像形成过程进行反向传播通常是不切实际的, 因为它们通常是不可微的, 并且这种长程反向传播可能导致极端的数值不稳定, 因为次优策略的线性化可能不稳定。在循环神经网络的相关背景中也观察到了这个问题 (Hochreiter 等人, 2001 年; Pascanu 和 Bengio, 2012 年)。网络的高维性也使得强化学习变得困难 (Deisenroth et al., 2013)。神经网络控制的开创性早期工作

small, simple networks (Pomerleau, 1989; Hunt et al., 1992; Bekey and Goldberg, 1992; Lewis et al., 1998; Bakker et al., 2003; Mayer et al., 2006), and has largely been supplanted by methods that use carefully designed policies that can be learned efficiently with reinforcement learning (Kober et al., 2013). More recent work on sensorimotor deep learning has tackled simple task-space motions (Lenz et al., 2015a; Lampe and Riedmiller, 2013) and used unsupervised learning to obtain low-dimensional state spaces from images (Lange et al., 2012). Such methods have been demonstrated on tasks with a low-dimensional underlying structure: Lenz et al. (2015a) controls the end-effector in 2D space, while Lange et al. (2012) controls a 2-dimensional slot car with 1-dimensional actions. Our experiments include full torque control of 7-DoF robotic arms interacting with objects, with 30-40 state dimensions. In simple synthetic environments, control from images has been addressed with image features (Jodogne and Piater, 2007), nonparametric methods (van Hoof et al., 2015), and unsupervised state-space learning (Böhmer et al., 2013; Jonschkowski and Brock, 2014). CNNs have also been trained to play video games with Q-learning, Monte Carlo tree search, and stochastic search (Mnih et al., 2013; Koutník et al., 2013; Guo et al., 2014), and have been applied to simple simulated control tasks (Watter et al., 2015; Lillicrap et al., 2015). However, such methods have only been demonstrated on synthetic domains that lack the visual complexity of the real world, and require an impractical number of samples for real-world robotic learning. Our method is sample efficient, requiring only minutes of interaction time. To the best of our knowledge, this is the first method that can train deep visuomotor policies for complex, high-dimensional manipulation skills with direct torque control.

Learning visuomotor policies on a real robot requires handling complex observations and high dimensional policy representations. We tackle these challenges using guided policy search. In guided policy search, the policy is optimized using supervised learning, which scales gracefully with the dimensionality of the policy. The training set for supervised learning can be constructed using trajectory optimization under known dynamics (Levine and Koltun, 2013a,b, 2014; Mordatch and Todorov, 2014) and trajectory-centric reinforcement learning methods that operate under unknown dynamics (Levine and Abbeel, 2014; Levine et al., 2015), which is the approach taken in this work. In both cases, the supervision is adapted to the policy, to ensure that the final policy can reproduce the training data. The use of supervised learning in the inner loop of iterative policy search has also been proposed in the context of imitation learning (Ross et al., 2011, 2013). However, such methods typically do not address the question of how the supervision should be adapted to the policy.

The goal of our approach is also similar to visual servoing, which performs feedback control on feature points in a camera image (Espiau et al., 1992; Mohta et al., 2014; Wilson et al., 1996). However, our visuomotor policies are entirely learned from real-world data, and do not require feature points or feedback controllers to be specified by hand. This allows our method much more flexibility in choosing how to use the visual signal. Our approach also does not require any sort of camera calibration, in contrast to many visual servoing methods (though not all – see e.g. Jägersand et al. (1997); Yoshimi and Allen (1994)).

3. Background and Overview

In this section, we define the visuomotor policy learning problem and present an overview of our approach. The core component of our approach is a guided policy search algorithm

小型、简单的网络 (Pomerleau, 1989; Hunt 等人, 1992; Bekey 和 Goldberg, 1992; Lewis 等人, 1998; Bakker 等人, 2003; Mayer 等人, 2006), 并且在很大程度上被使用精心设计的策略的方法所取代, 这些策略可以通过强化学习有效地学习 (Kober 等人, 2013)。最近关于感觉运动深度学习的工作已经解决了简单的任务空间运动 (Lenz 等人, 2015a; Lampe 和 Riedmiller, 2013), 并使用无监督学习从图像中获取低维状态空间 (Lange 等人, 2012)。这些方法已经在具有低维底层结构的任务上得到了证明: Lenz 等人. (2015a) 控制二维空间中的末端执行器, 而 Lange 等人. (2012) 用一维动作控制二维老虎车。我们的实验包括 7-DoF 机械臂与物体交互的全扭矩控制, 具有 30-40 个状态维度。在简单的合成环境中, 图像控制已通过图像特征 (Jodogne 和 Piater, 2007)、非参数方法 (van Hoof 等人, 2015) 和无监督状态空间学习 (Böhmer 等人, 2013; Jonschkowski 和 Brock, 2014) 来解决。CNN 还被训练用于使用 Q 学习、蒙特卡洛树搜索和随机搜索来玩视频游戏 (Mnih 等人, 2013 年; Koutník 等人, 2013 年; Guo 等人, 2014 年), 并已应用于简单的模拟控制任务 (Watter 等人, 2015 年; Lillicrap 等人, 2015 年)。然而, 此类方法仅在缺乏现实世界视觉复杂性的合成领域得到验证, 并且需要不切实际的样本数量来进行现实世界的机器人学习。我们的方法样本效率高, 只需要几分钟的交互时间。据我们所知, 这是第一种可以通过直接扭矩控制来训练深度视觉运动策略以实现复杂、高维操作技能的方法。

在真实机器人上学习视觉运动策略需要处理复杂的观察结果和高维策略表示。我们使用引导策略搜索来应对这些挑战。在引导策略搜索中, 使用监督学习来优化策略, 监督学习可以随着策略的维度进行优雅地扩展。监督学习的训练集可以使用已知动态下的轨迹优化 (Levine and Koltun, 2013a,b, 2014; Mordatch and Todorov, 2014) 和未知动态下运行的以轨迹为中心的强化学习方法 (Levine and Abbeel, 2014; Levine et al., 2015) 来构建, 这是本工作中采用的方法。在这两种情况下, 监督都会适应策略, 以确保最终策略可以重现训练数据。在模仿学习的背景下也提出了在迭代策略搜索的内循环中使用监督学习 (Ross et al., 2011, 2013)。然而, 这些方法通常没有解决监管应如何适应政策的问题。

我们方法的目标也类似于视觉伺服, 它对相机图像中的特征点执行反馈控制 (Espiau et al., 1992; Mohta et al., 2014; Wilson et al., 1996)。然而, 我们的视觉运动策略完全是从现实世界的学习中学习的, 不需要手动指定特征点或反馈控制器。这使得我们的方法在选择如何使用视觉信号方面更加灵活。与许多视觉伺服方法相比, 我们的方法也不需要任何类型的相机校准 (尽管不是全部, 参见 Jägersand 等人 (1997 年); Yoshimi 和 Allen (1994 年))。

3. Background and Overview

在本节中, 我们定义视觉运动政策学习问题并概述我们的方法。我们方法的核心组成部分是引导策略搜索算法

that separates the problem of learning visuomotor policies into separate supervised learning and trajectory learning phases, each of which is easier than optimizing the policy directly. We also discuss a policy architecture suitable for end-to-end learning of vision and control, and a training setup that allows our method to be applied to real robotic platforms.

3.1 Definitions and Problem Formulation

In policy search, the goal is to learn a policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ that allows an agent to choose actions $\mathbf{u}_t$ in response to observations $\mathbf{o}_t$ to control a dynamical system, such as a robot. The policy comes from some parametric class parameterized by θ , which could be, for example, the weights of a neural network. The system is defined by states $\mathbf{x}_t$, actions $\mathbf{u}_t$, and observations $\mathbf{o}_t$. For example, $\mathbf{x}_t$ might include the joint angles of the robot, the positions of objects in the world, and their time derivatives, $\mathbf{u}_t$ might consist of motor torque commands, and $\mathbf{o}_t$ might include an image from the robot’s onboard camera. In this paper, we address finite horizon episodic tasks with $t \in [1, \dots, T]$. The states evolve in time according to the system dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$, and the observations are, in general, a stochastic consequence of the states, according to $p(\mathbf{o}_t|\mathbf{x}_t)$. Neither the dynamics nor the observation distribution are assumed to be known in general. For notational convenience, we will use $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ to denote the distribution over actions under the policy conditioned on the state. However, since the policy is conditioned on the observation $\mathbf{o}_t$, this distribution is in fact given by $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) = \int \pi_\theta(\mathbf{u}_t|\mathbf{o}_t)p(\mathbf{o}_t|\mathbf{x}_t)d\mathbf{o}_t$. The dynamics and $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ together induce a distribution over trajectories $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \mathbf{x}_2, \mathbf{u}_2, \dots, \mathbf{x}_T, \mathbf{u}_T\}$:

$$\pi_\theta(\tau) = p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t).$$

The goal of a task is given by a cost function $\ell(\mathbf{x}_t, \mathbf{u}_t)$, and the objective in policy search is to minimize the expectation $E_{\pi_\theta(\tau)}[\sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t)]$, which we will abbreviate as $E_{\pi_\theta(\tau)}[\ell(\tau)]$. A summary of the notation used in the paper is provided in Table 1.

3.2 Approach Summary

Our methods consists of two main components, which are illustrated in Figure 3. The first is a supervised learning algorithm that trains policies of the form $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t) = \mathcal{N}(\mu^\pi(\mathbf{o}_t), \Sigma^\pi(\mathbf{o}_t))$, where both $\mu^\pi(\mathbf{o}_t)$ and $\Sigma^\pi(\mathbf{o}_t)$ are general nonlinear functions. In our implementation, $\mu^\pi(\mathbf{o}_t)$ is a deep convolutional neural network, while $\Sigma^\pi(\mathbf{o}_t)$ is an observation-independent learned covariance, though other representations are possible. The second component is a trajectory-centric reinforcement learning (RL) algorithm that generates guiding distributions $p_i(\mathbf{u}_t|\mathbf{x}_t)$ that provide the supervision used to train the policy. These two components form a policy search algorithm that can be used to learn complex robotic tasks using only a high-level cost function $\ell(\mathbf{x}_t, \mathbf{u}_t)$. During training, only samples from the guiding distributions $p_i(\mathbf{u}_t|\mathbf{x}_t)$ are generated by running rollouts on the physical system, which avoids the need to execute partially trained neural network policies on physical hardware.

Supervised learning will not, in general, produce a policy with good long-horizon performance, since a small mistake on the part of the policy will place the system into states that are outside the distribution in the training data, causing compounding errors. To

它将学习视觉运动策略的问题分为单独的监督学习和轨迹学习阶段，每个阶段都比直接优化策略更容易。我们还讨论了适合视觉和控制端到端学习的策略架构，以及允许我们的方法应用于真实机器人平台的训练设置。

3.1 Definitions and Problem Formulation

在策略搜索中，目标是学习策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ ，该策略允许代理根据观察值 $\mathbf{o}_t$ 选择动作 $\mathbf{u}_t$ ，以控制动态系统（例如机器人）。该策略来自某个由 θ 参数化的参数类，例如，它可以是神经网络的权重。该系统由状态 $\mathbf{x}_t$ 、动作 $\mathbf{u}_t$ 和观察 $\mathbf{o}_t$ 定义。例如， $\mathbf{x}_t$ 可能包括机器人的关节角度、世界上物体的位置及其时间导数， $\mathbf{u}_t$ 可能包括电机扭矩命令，而 $\mathbf{o}_t$ 可能包括来自机器人机载摄像头的图像。在本文中，我们使用 $t \in [1, \dots, T]$ 解决有限视野情景任务。状态根据系统动力学 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 随时间演化，并且根据 $p(\mathbf{o}_t|\mathbf{x}_t)$ ，观测结果通常是状态的随机结果。动力学和观测分布均不被假定为一般已知。为了符号方便，我们将使用 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 来表示以状态为条件的策略下的动作分布。然而，由于该策略以观察 $\mathbf{o}_t$ 为条件，因此该分布实际上由 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) = \int \pi_\theta(\mathbf{u}_t|\mathbf{o}_t)p(\mathbf{o}_t|\mathbf{x}_t)d\mathbf{o}_t$ 给出。动力学和 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 共同导致轨迹 $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \mathbf{x}_2, \mathbf{u}_2, \dots, \mathbf{x}_T, \mathbf{u}_T\}$ 上的分布：

$$\pi_\theta(\tau) = p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t).$$

任务的目标由成本函数 $\ell(\mathbf{x}_t, \mathbf{u}_t)$ 给出，策略搜索的目标是最小化期望 $E_{\pi_\theta(\tau)}[\sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t)]$ ，我们将其缩写为 $E_{\pi_\theta(\tau)}[\ell(\tau)]$ 。表 1 总结了本文中使用的符号。

3.2 Approach Summary

我们的方法由两个主要部分组成，如图 3 所示。第一个是监督学习算法，用于训练 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t) = \mathcal{N}(\mu^\pi(\mathbf{o}_t), \Sigma^\pi(\mathbf{o}_t))$ 形式的策略，其中 $\mu^\pi(\mathbf{o}_t)$ 和 $\Sigma^\pi(\mathbf{o}_t)$ 都是一般非线性函数。在我们的实现中， $\mu^\pi(\mathbf{o}_t)$ 是一个深度卷积神经网络，而 $\Sigma^\pi(\mathbf{o}_t)$ 是一个与观察无关的学习协方差，尽管其他表示也是可能的。第二个组成部分是一个以轨迹为中心的强化学习（RL）算法，它生成指导分布 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ ，提供用于训练策略的监督。这两个组件形成了一个策略搜索算法，可用于仅使用高级成本函数 $\ell(\mathbf{x}_t, \mathbf{u}_t)$ 来学习复杂的机器人任务。在训练期间，通过在物理系统上运行 rollout 来仅生成来自引导分布 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 的样本，这避免了在物理硬件上执行部分训练的神经网络策略的需要。

一般来说，监督学习不会产生具有良好长期性能的策略，因为策略的一个小错误就会使系统进入训练数据分布之外的状态，从而导致复合错误。到

symbol	definition	example/details
$\mathbf{x}_t$	Markovian system state at time step $t \in [1, T]$	joint angles, end-effector pose, object positions, and their velocities; dimensionality: 14 to 32
$\mathbf{u}_t$	control or action at time step $t \in [1, T]$	joint motor torque commands; dimensionality: 7 (for the PR2 robot)
$\mathbf{o}_t$	observation at time step $t \in [1, T]$	RGB camera image, joint encoder readings & velocities, end-effector pose; dimensionality: around 200,000
τ	trajectory: $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \mathbf{x}_2, \mathbf{u}_2, \dots, \mathbf{x}_T, \mathbf{u}_T\}$	notational shorthand for a sequence of states and actions
$\ell(\mathbf{x}_t, \mathbf{x}_t)$	cost function that defines the goal of the task	distance between an object in the gripper and the target
$p(\mathbf{x}_{t+1} \mathbf{x}_t, \mathbf{u}_t)$	unknown system dynamics	physics that govern the robot and any objects it interacts with
$p(\mathbf{o}_t \mathbf{x}_t)$	unknown observation distribution	stochastic process that produces camera images from system state
$\pi_\theta(\mathbf{u}_t \mathbf{o}_t)$	learned nonlinear global policy parameterized by weights θ	convolutional neural network, such as the one in Figure 2
$\pi_\theta(\mathbf{u}_t \mathbf{x}_t)$	$\int \pi_\theta(\mathbf{u}_t \mathbf{o}_t)p(\mathbf{o}_t \mathbf{x}_t)d\mathbf{o}_t$	notational shorthand for observation-based policy conditioned on state
$p_i(\mathbf{u}_t \mathbf{x}_t)$	learned local time-varying linear-Gaussian controller for initial state $\mathbf{x}_1^i$	time-varying linear-Gaussian controller has form $\mathcal{N}(\mathbf{K}_{ti}\mathbf{x}_t + \mathbf{k}_{ti}, \mathbf{C}_{ti})$
$\pi_\theta(\tau)$	trajectory distribution for $\pi_\theta(\mathbf{u}_t \mathbf{x}_t)$: $p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t \mathbf{x}_t)p(\mathbf{x}_{t+1} \mathbf{x}_t, \mathbf{u}_t)$	notational shorthand for trajectory distribution induced by a policy

Table 1: Summary of the notation frequently used in this article.

avoid this issue, the training data must come from the policy’s own state distribution (Ross et al., 2011). We achieve this by alternating between trajectory-centric RL and supervised learning. The RL stage adapts to the current policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$, providing supervision at states that are iteratively brought closer to the states visited by the policy. This is formalized as a variant of the BADMM algorithm (Wang and Banerjee, 2014) for constrained optimization, which can be used to show that, at convergence, the policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ and the guiding distributions $p_i(\mathbf{u}_t|\mathbf{x}_t)$ will exhibit the same behavior. This algorithm is derived in Section 4. The guiding distributions are substantially easier to optimize than learning the policy parameters directly (e.g., using model-free reinforcement learning), because they use the full state of the system $\mathbf{x}_t$, while the policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ only uses the observations. This means that the method requires the full state to be known during training, but not at test time. This makes it possible to efficiently learn complex visuomotor policies, but imposes additional assumptions on the observability of $\mathbf{x}_t$ during training that we discuss in Section 4.

When learning visuomotor tasks, the policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ is represented by a novel convolutional neural network (CNN) architecture, which we describe in Section 5.2. CNNs have enjoyed considerable success in computer vision (LeCun et al., 2015), but the most popular

symbol	definition	example/details
$\mathbf{x}_t$	Markovian system state at time step $t \in [1, T]$	joint angles, end-effector pose, object positions, and their velocities; dimensionality: 14 to 32
$\mathbf{u}_t$	control or action at time step $t \in [1, T]$	joint motor torque commands; dimensionality: 7 (for the PR2 robot)
$\mathbf{o}_t$	observation at time step $t \in [1, T]$	RGB camera image, joint encoder readings & velocities, end-effector pose; dimensionality: around 200,000
τ	trajectory: $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \mathbf{x}_2, \mathbf{u}_2, \dots, \mathbf{x}_T, \mathbf{u}_T\}$	notational shorthand for a sequence of states and actions
$\ell(\mathbf{x}_t, \mathbf{x}_t)$	cost function that defines the goal of the task	distance between an object in the gripper and the target
$p(\mathbf{x}_{t+1} \mathbf{x}_t, \mathbf{u}_t)$	unknown system dynamics	physics that govern the robot and any objects it interacts with
$p(\mathbf{o}_t \mathbf{x}_t)$	unknown observation distribution	stochastic process that produces camera images from system state
$\pi_\theta(\mathbf{u}_t \mathbf{o}_t)$	learned nonlinear global policy parameterized by weights θ	convolutional neural network, such as the one in Figure 2
$\pi_\theta(\mathbf{u}_t \mathbf{x}_t)$	$\int \pi_\theta(\mathbf{u}_t \mathbf{o}_t)p(\mathbf{o}_t \mathbf{x}_t)d\mathbf{o}_t$	notational shorthand for observation-based policy conditioned on state
$p_i(\mathbf{u}_t \mathbf{x}_t)$	learned local time-varying linear-Gaussian controller for initial state $\mathbf{x}_1^i$	time-varying linear-Gaussian controller has form $\mathcal{N}(\mathbf{K}_{ti}\mathbf{x}_t + \mathbf{k}_{ti}, \mathbf{C}_{ti})$
$\pi_\theta(\tau)$	trajectory distribution for $\pi_\theta(\mathbf{u}_t \mathbf{x}_t)$: $p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t \mathbf{x}_t)p(\mathbf{x}_{t+1} \mathbf{x}_t, \mathbf{u}_t)$	notational shorthand for trajectory distribution induced by a policy

表 1: 本文中常用符号的摘要。

为了避免这个问题, 训练数据必须来自策略自己的状态分布 (Ross et al., 2011)。我们通过交替以轨迹为中心的强化学习和监督学习来实现这一目标。RL 阶段适应当前的策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$, 对迭代地接近策略访问的状态的状态提供监督。这被形式化为用于约束优化的 BADMM 算法 (Wang 和 Banerjee, 2014) 的变体, 它可以用来表明, 在收敛时, 策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 和引导分布 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 将表现出相同的行为。该算法在第 4 节中推导。引导分布比直接学习策略参数 (例如, 使用无模型强化学习) 更容易优化, 因为它们使用系统 $\mathbf{x}_t$ 的完整状态, 而策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 仅使用观察结果。这意味着该方法需要在训练期间而不是在测试时了解完整状态。这使得有效学习复杂的视觉运动策略成为可能, 但我们在第 4 节中讨论的训练期间 $\mathbf{x}_t$ 的可观察性施加了额外的假设。

当学习视觉运动任务时, 策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 由一种新颖的卷积神经网络 (CNN) 架构表示, 我们在第 5.2 节中对此进行了描述。CNN 在计算机视觉领域取得了相当大的成功 (LeCun 等人, 2015), 但最流行的

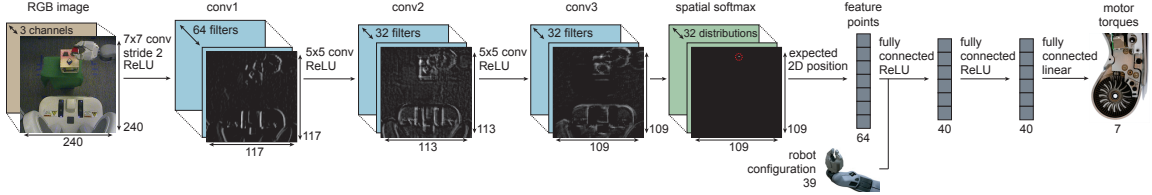


Figure 2: Visuomotor policy architecture. The network contains three convolutional layers, followed by a spatial softmax and an expected position layer that converts pixel-wise features to feature points, which are better suited for spatial computations. The points are concatenated with the robot configuration, then passed through three fully connected layers to produce the torques.

architectures rely on large datasets and focus on semantic tasks such as classification, often intentionally discarding spatial information. Our architecture, illustrated in Figure 2, uses a fixed transformation from the last convolutional layer to a set of spatial feature points, which form a concise representation of the visual scene suitable for feedback control. Our network has 7 layers and around 92,000 parameters, which presents a major challenge for standard policy search methods (Deisenroth et al., 2013).

To reduce the amount of experience needed to train visuomotor policies, we also introduce a pretraining scheme that allows us to train effective policies with a relatively small number of iterations. The pretraining steps are illustrated in Figure 3. The intuition behind our pretraining is that, although we ultimately seek to obtain sensorimotor policies that combine both vision and control, low-level aspects of vision can be initialized independently. To that end, we pretrain the convolutional layers of our network by predicting elements of $\mathbf{x}_t$ that are not provided in the observation $\mathbf{o}_t$, such as the positions of objects in the scene. We also initially train the guiding trajectory distributions $p_i(\mathbf{u}_t|\mathbf{x}_t)$ independently of the convolutional network until the trajectories achieve a basic level of competence at the task, and then switch to full guided policy search with end-to-end training of $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$. In our implementation, we also initialize the first layer filters from the model of Szegedy et al. (2014), which is trained on ImageNet (Deng et al., 2009) classification. The initialization and pretraining scheme is described in Section 5.2.

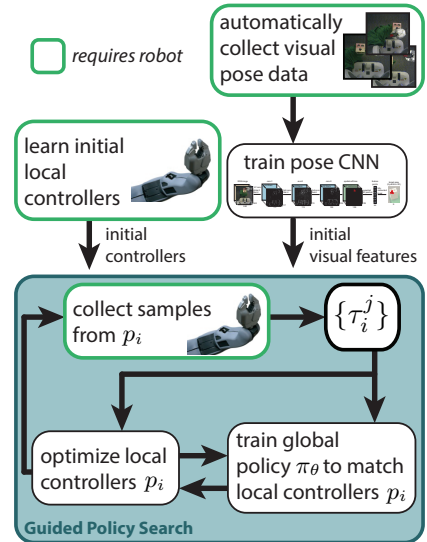


Figure 3: Diagram of our approach, including the main guided policy search phase and initialization phases.

4. Guided Policy Search with BADMM

Guided policy search transforms policy search into a supervised learning problem, where the training set is generated by a simple trajectory-centric RL algorithm. This algorithm

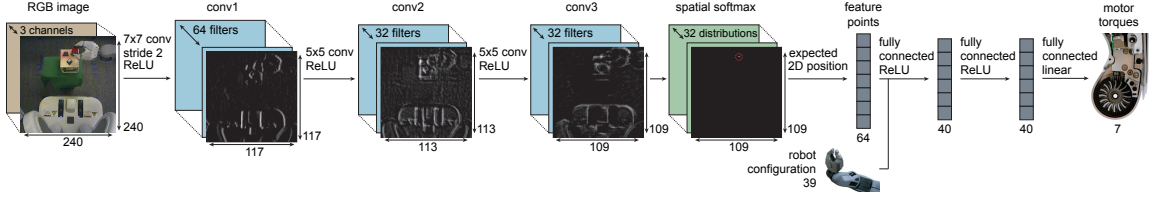


图 2: 视觉运动政策架构。该网络包含三个卷积层, 后面是一个空间softmax和一个将像素级特征转换为特征点的预期位置层, 这更适合空间计算。这些点与机器人配置连接起来, 然后通过三个完全连接的层来产生扭矩。

架构依赖于大型数据集, 并专注于分类等语义任务, 通常故意丢弃空间信息。我们的架构如图 2 所示, 使用从最后一个卷积层到一组空间特征点的固定变换, 形成适合反馈控制的视觉场景的简明表示。我们的网络有 7 层和大约 92,000 个参数, 这对标准策略搜索方法提出了重大挑战 (Deisenroth 等, 2013)。

为了减少训练视觉运动策略所需的经验量, 我们还引入了一种预训练方案, 使我们能够以相对较少的迭代次数训练有效的策略。预训练步骤如图 3 所示。预训练背后的直觉是, 尽管我们最终寻求获得结合视觉和控制的视觉运动策略, 但视觉的低级方面可以独立初始化。为此, 我们通过预测观察 $\mathbf{o}_t$ 中未提供的 $\mathbf{x}_t$ 元素 (例如场景中对象的位置) 来预训练网络的卷积层。我们还最初独立于卷积网络训练引导轨迹分布 $p_i(\mathbf{u}_t|\mathbf{x}_t)$, 直到轨迹达到任务的基本能力水平, 然后切换到通过端到端训练 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 的完全引导策略搜索。在我们的实现中, 我们还根据 Szegedy 等人的模型初始化第一层过滤器。(2014), 它是在 ImageNet (Deng et al., 2009) 分类上进行训练的。初始化和预训练方案在 5.2 节中描述。

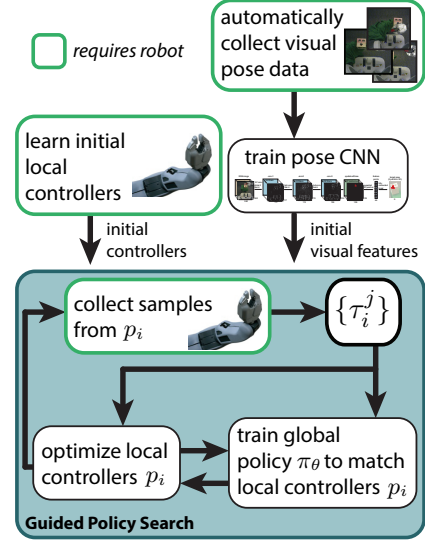


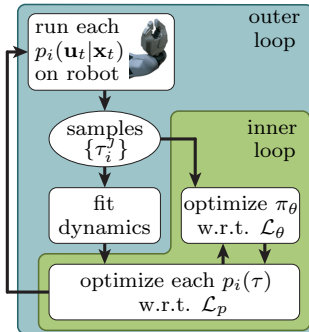
图 3: 我们的方法图, 包括主要的引导策略搜索阶段和初始化阶段

4. Guided Policy Search with BADMM

引导策略搜索将策略搜索转变为监督学习问题, 其中训练集由简单的以轨迹为中心的强化学习算法生成。这个算法

optimizes linear-Gaussian controllers $p_i(\mathbf{u}_t|\mathbf{x}_t)$, and is described in Section 4.2. We refer to the trajectory distribution induced by $p_i(\mathbf{u}_t|\mathbf{x}_t)$ as $p_i(\tau)$. Each $p_i(\mathbf{u}_t|\mathbf{x}_t)$ succeeds from different initial states. For example, in the task of placing a cap on a bottle, these initial states correspond to different positions of the bottle. By training on trajectories for multiple bottle positions, the final CNN policy can succeed from all initial states, and can generalize to other states from the same distribution.

The final policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ learned with guided policy search is only provided with observations $\mathbf{o}_t$ of the full state $\mathbf{x}_t$, and the dynamics are assumed to be unknown. A diagram of this method, which corresponds to an expanded version of the guided policy search box in Figure 3, is shown on the right. In the outer loop, we draw sample trajectories $\{\tau_i^j\}$ for each initial state on the physical system by running the corresponding controller $p_i(\mathbf{u}_t|\mathbf{x}_t)$. The samples are used to fit the dynamics $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ that are used to improve $p_i(\mathbf{u}_t|\mathbf{x}_t)$, and serve as training data for the policy. The inner loop alternates between optimizing each $p_i(\tau)$ and optimizing the policy to match these trajectory distributions. The policy is trained to predict the actions along each trajectory from the observations $\mathbf{o}_t$, rather than the full state $\mathbf{x}_t$. This allows the policy to directly use raw observations at test time. This alternating optimization can be framed as an instance of the BADMM algorithm (Wang and Banerjee, 2014), which converges to a solution where the trajectory distributions and the policy have the same state distribution. This allows greedy supervised training of the policy to produce a policy with good long-horizon performance.



4.1 Algorithm Derivation

Policy search methods minimize the expected cost $E_{\pi_\theta}[\ell(\tau)]$, where $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_T, \mathbf{u}_T\}$ is a trajectory, and $\ell(\tau) = \sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t)$ is the cost of an episode. In the fully observed case, the expectation is taken under $\pi_\theta(\tau) = p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$. The final policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ is conditioned on the observations $\mathbf{o}_t$, but $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ can be recovered as $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) = \int \pi_\theta(\mathbf{u}_t|\mathbf{o}_t)p(\mathbf{o}_t|\mathbf{x}_t)d\mathbf{o}_t$. We will present the derivation in this section for $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$, but we do not require knowledge of $p(\mathbf{o}_t|\mathbf{x}_t)$ in the final algorithm. As discussed in Section 4.3, the integral will be evaluated with samples from the real system, which include both $\mathbf{x}_t$ and $\mathbf{o}_t$. We begin by rewriting the expected cost minimization as a constrained problem:

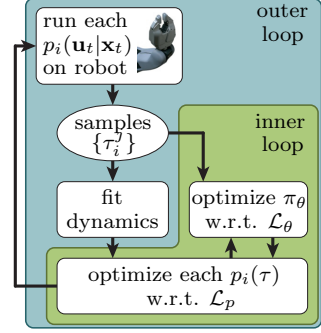
$$\min_{p, \pi_\theta} E_p[\ell(\tau)] \text{ s.t. } p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \quad \forall \mathbf{x}_t, \mathbf{u}_t, t, \quad (1)$$

where we will refer to $p(\tau)$ as a guiding distribution. This formulation is equivalent to the original problem, since the constraint forces the two distributions to be identical. However, if we approximate the initial state distribution $p(\mathbf{x}_1)$ with samples $\mathbf{x}_1^i$, we can choose $p(\tau)$ to be a class of distributions that is much easier to optimize than π_θ , as we will show later. This will allow us to use simple local learning methods for $p(\tau)$, without needing to train the complex neural network policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ directly with reinforcement learning, which would require a prohibitive amount of experience on real physical systems.

The constrained problem can be solved by a dual descent method, which alternates between minimizing the Lagrangian with respect to the primal variables, and incrementing

优化线性高斯控制器 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ ，并在 4.2 节中进行了描述。我们将 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 引起的轨迹分布称为 $p_i(\tau)$ 。每个 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 都从不同的初始状态成功。例如，在给瓶子盖上盖子的任务中，这些初始状态对应于瓶子的不同位置。通过对多个瓶子位置的轨迹进行训练，最终的 CN N 策略可以从所有初始状态取得成功，并且可以推广到相同分布的其他状态。

通过引导策略搜索学习到的最终策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 仅提供完整状态 $\mathbf{x}_t$ 的观察值 $\mathbf{o}_t$ ，并且假设动态未知。右侧显示了该方法的图表，对应于图 3 中引导策略搜索框的扩展版本。在外循环中，我们通过运行相应的控制器 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 为物理系统上的每个初始状态绘制样本轨迹 $\{\tau_i^j\}$ 。这些样本用于拟合用于改进 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 的动态 $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ ，并用作策略的训练数据。内循环交替进行



优化每个 $p_i(\tau)$ 并优化策略以匹配这些轨迹分布。该策略经过训练，可以根据观察值 $\mathbf{o}_t$ 而不是完整状态 $\mathbf{x}_t$ 来预测沿每个轨迹的操作。这允许策略在测试时直接使用原始观察结果。这种交替优化可以被构建为 BADMM 算法的一个实例 (Wang 和 Banerjee, 2014)，该算法收敛到轨迹分布和策略具有相同状态分布的解决方案。这允许对策略进行贪婪的监督训练，以产生具有良好长期性能的策略。

4.1 Algorithm Derivation

策略搜索方法最小化预期成本 $E_{\pi_\theta}[\ell(\tau)]$ ，其中 $\tau = \{\mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_T, \mathbf{u}_T\}$ 是轨迹， $\ell(\tau) = \sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t)$ 是一个情节的成本。在完全观察到的情况下，期望在 $\pi_\theta(\tau) = p(\mathbf{x}_1) \prod_{t=1}^T \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 下进行。最终策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 以观测值 $\mathbf{o}_t$ 为条件，但 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 可以恢复为 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) = \int \pi_\theta(\mathbf{u}_t|\mathbf{o}_t)p(\mathbf{o}_t|\mathbf{x}_t)d\mathbf{o}_t$ 。我们将在本节中介绍 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 的推导，但在最终算法中我们不需要了解 $p(\mathbf{o}_t|\mathbf{x}_t)$ 。正如第 4.3 节中所讨论的，积分将使用来自真实系统的样本进行评估，其中包括 $\mathbf{x}_t$ 和 $\mathbf{o}_t$ 。我们首先将预期成本最小化重写为约束问题：

$$\min_{p, \pi_\theta} E_p[\ell(\tau)] \text{ s.t. } p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \quad \forall \mathbf{x}_t, \mathbf{u}_t, t, \quad (1)$$

我们将 $p(\tau)$ 称为指导分布。该公式与原始问题等效，因为约束强制两个分布相同。然而，如果我们用样本 $\mathbf{x}_1^i$ 近似初始状态分布 $p(\mathbf{x}_1)$ ，我们可以选择 $p(\tau)$ 作为一类比 π_θ 更容易优化的分布，正如我们稍后将展示的。这将使我们能够对 $p(\tau)$ 使用简单的本地学习方法，而无需直接通过强化学习来训练复杂的神经网络策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ ，这需要在真实物理系统上拥有大量的经验。

约束问题可以通过双下降法来解决，该方法交替最小化相对于原始变量的拉格朗日量和递增

the Lagrange multipliers by their subgradient. Minimization of the Lagrangian with respect to $p(\tau)$ and θ is done in alternating fashion: minimizing with respect to θ corresponds to supervised learning (making π_θ match $p(\tau)$), and minimizing with respect to $p(\tau)$ consists of one or more trajectory optimization problems. The dual descent method we use is based on BADMM (Wang and Banerjee, 2014), a variant of ADMM (Boyd et al., 2011) that augments the Lagrangian with a Bregman divergence between the constrained variables. We use the KL-divergence as the Bregman constraint, which is particularly convenient for working with probability distributions. We will also modify the constraint $p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ by multiplying both sides by $p(\mathbf{x}_t)$, to get $p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)$. This constraint is equivalent, but has the convenient property that we can express the Lagrangian in terms of expectations. The BADMM augmented Lagrangians for θ and p are therefore given by

$$\begin{aligned}\mathcal{L}_\theta(\theta, p) &= \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p) \\ \mathcal{L}_p(p, \theta) &= \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(\theta, p),\end{aligned}$$

where $\lambda_{\mathbf{x}_t, \mathbf{u}_t}$ is the Lagrange multiplier for state $\mathbf{x}_t$ and action $\mathbf{u}_t$ at time t , and $\phi_t^\theta(\theta, p)$ are $\phi_t^p(\theta, p)$ are expectations of the KL-divergences:

$$\begin{aligned}\phi_t^p(p, \theta) &= E_{p(\mathbf{x}_t)}[D_{\text{KL}}(p(\mathbf{u}_t|\mathbf{x}_t) \parallel \pi_\theta(\mathbf{u}_t|\mathbf{x}_t))] \\ \phi_t^\theta(\theta, p) &= E_{p(\mathbf{x}_t)}[D_{\text{KL}}(\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \parallel p(\mathbf{u}_t|\mathbf{x}_t))].\end{aligned}$$

Dual descent with alternating primal minimization is then described by the following steps:

$$\begin{aligned}\theta &\leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p) \\ p &\leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(p, \theta) \\ \lambda_{\mathbf{x}_t, \mathbf{u}_t} &\leftarrow \lambda_{\mathbf{x}_t, \mathbf{u}_t} + \alpha \nu_t (\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) - p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)).\end{aligned}$$

This procedure is an instance of BADMM, and therefore inherits its convergence guarantees. Note that we drop terms that are independent of the optimization variables on each line. The parameter α is a step size. As with most augmented Lagrangian methods, the weight ν_t is set heuristically, as described in Appendix A.1.

The dynamics only affect the optimization with respect to $p(\tau)$. In order to make this optimization efficient, we choose $p(\tau)$ to be a mixture of N Gaussians $p_i(\tau)$, one for each initial state sample $\mathbf{x}_1^i$. This makes the action conditionals $p_i(\mathbf{u}_t|\mathbf{x}_t)$ and the dynamics $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ linear-Gaussian, as discussed in Section 4.2. This is a reasonable choice when the system is deterministic, or the noise is Gaussian or small, and we found that this approach is sufficiently tolerant to noise for use on real physical systems. Our choice of p also assumes that the policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ is conditionally Gaussian. This is also reasonable, since the mean and covariance of $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ can be any nonlinear function of the observations

拉格朗日乘子除以其次梯度。关于 $p(\tau)$ 和 θ 的拉格朗日最小化是以交替方式完成的：关于 θ 的最小化对应于监督学习（使 π_θ 匹配 $p(\tau)$ ），关于 $p(\tau)$ 的最小化由一个或多个轨迹优化问题组成。我们使用的双下降法基于 BADMM (Wang 和 Banerjee, 2014)，它是 ADMM (Boyd 等人, 2011) 的一种变体，它通过约束变量之间的 Bregman 散度来增强拉格朗日。我们使用 KL 散度作为 Bregman 约束，这对于处理概率分布特别方便。我们还将修改约束 $p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ ，将两边乘以 $p(\mathbf{x}_t)$ ，得到 $p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)$ 。该约束是等价的，但具有方便的属性，我们可以用期望来表达拉格朗日量。因此， θ 和 p 的 BADMM 增广拉格朗日量由下式给出

$$\begin{aligned}\mathcal{L}_\theta(\theta, p) &= \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p) \\ \mathcal{L}_p(p, \theta) &= \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(\theta, p),\end{aligned}$$

其中 $\lambda_{\mathbf{x}_t, \mathbf{u}_t}$ 是状态 $\mathbf{x}_t$ 和动作 $\mathbf{u}_t$ 在时间 t 的拉格朗日乘子， $\phi_t^\theta(\theta, p)$ 是 $\phi_t^p(\theta, p)$ 是 KL 散度的期望：

$$\begin{aligned}\phi_t^p(p, \theta) &= E_{p(\mathbf{x}_t)}[D_{\text{KL}}(p(\mathbf{u}_t|\mathbf{x}_t) \parallel \pi_\theta(\mathbf{u}_t|\mathbf{x}_t))] \\ \phi_t^\theta(\theta, p) &= E_{p(\mathbf{x}_t)}[D_{\text{KL}}(\pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \parallel p(\mathbf{u}_t|\mathbf{x}_t))].\end{aligned}$$

然后通过以下步骤描述具有交替原始最小化的双下降：

$$\begin{aligned}\theta &\leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p) \\ p &\leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(p, \theta) \\ \lambda_{\mathbf{x}_t, \mathbf{u}_t} &\leftarrow \lambda_{\mathbf{x}_t, \mathbf{u}_t} + \alpha \nu_t (\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) - p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)).\end{aligned}$$

该过程是 BADMM 的一个实例，因此继承了它的收敛保证。请注意，我们删除了与每行优化变量无关的项。参数 α 是步长。与大多数增强拉格朗日方法一样，权重 ν_t 是启发式设置的，如附录 A.1 中所述。

动态仅影响针对 $p(\tau)$ 的优化。为了使这种优化有效，我们选择 $p(\tau)$ 作为 N 高斯函数 $p_i(\tau)$ 的混合，每个初始状态样本 $\mathbf{x}_1^i$ 一个。这使得动作条件 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 和动态 $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 呈线性高斯分布，如第 4.2 节中所述。当系统是确定性的，或者噪声是高斯分布的或很小的时候，这是一个合理的选择，并且我们发现这种方法对于在实际物理系统上使用具有足够的容忍噪声的能力。我们对 p 的选择还假设策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 是条件高斯的。这也是合理的，因为 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 的均值和协方差可以是观测值的任何非线性函数

$\mathbf{o}_t$, which themselves are a function of the unobserved state $\mathbf{x}_t$. In Section 4.2, we show how these assumptions enable each $p_i(\tau)$ to be optimized very efficiently. We will refer to $p_i(\tau)$ as guiding distributions, since they serve to focus the policy on good, low-cost behaviors.

Aside from learning $p_i(\tau)$, we must choose a tractable way to represent the infinite set of constraints $p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)$. One approximate approach proposed in prior work is to replace the exact constraints with expectations of features (Peters et al., 2010). When the features consist of linear, quadratic, or higher order monomial functions of the random variable, this can be viewed as a constraint on the moments of the distributions. If we only use the first moment, we get a constraint on the expected action: $E_{p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t] = E_{\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t]$. If the stochasticity in the dynamics is low, as we assumed previously, the optimal solution for each $p_i(\tau)$ will have low entropy, making this first moment constraint a reasonable approximation. The KL-divergence terms in the augmented Lagrangians will still serve to softly enforce agreement between the higher moments. While this simplification is quite drastic, we found that it was more stable in practice than including higher moments, likely because these higher moments are harder to estimate accurately with a limited number of samples. The alternating optimization is now given by

$$\theta \leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^\theta(\theta, p) \quad (2)$$

$$p \leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^p(p, \theta) \quad (3)$$

$$\lambda_{\mu t} \leftarrow \lambda_{\mu t} + \alpha \nu_t (E_{\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t] - E_{p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t]),$$

where $\lambda_{\mu t}$ is the Lagrange multiplier on the expected action at time t . In the rest of the paper, we will use $\mathcal{L}_\theta(\theta, p)$ and $\mathcal{L}_p(p, \theta)$ to denote the two augmented Lagrangians in Equations (2) and (3), respectively. In the next two sections, we will describe how $\mathcal{L}_p(p, \theta)$ can be optimized with respect to p under unknown dynamics, and how $\mathcal{L}_\theta(\theta, p)$ can be optimized for complex, high-dimensional policies. Implementation details of the BADMM optimization are presented in Appendix A.1.

4.2 Trajectory Optimization under Unknown Dynamics

Since the Lagrangian $\mathcal{L}_p(p, \theta)$ in the previous section factorizes over the mixture elements in $p(\tau) = \sum_i p_i(\tau)$, we describe the trajectory optimization method for a single Gaussian $p(\tau)$. When there are multiple mixture elements, this procedure is applied in parallel to each $p_i(\tau)$. Since $p(\tau)$ is Gaussian, the conditionals $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ and $p(\mathbf{u}_t|\mathbf{x}_t)$, which correspond to the dynamics and the controller, are time-varying linear-Gaussian, and given by

$$p(\mathbf{u}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t, \mathbf{C}_t) \quad p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t} \mathbf{x}_t + f_{\mathbf{u}t} \mathbf{u}_t + f_{ct}, \mathbf{F}_t).$$

This type of controller can be learned efficiently with a small number of real-world samples, making it a good choice for optimizing the guiding distributions. Since a different set of time-varying linear-Gaussian dynamics is fitted for each initial state, this dynamics representation can model any continuous deterministic system that can be locally linearized. Stochastic dynamics can violate the local linearity assumption in principle, but we found that in practice this representation was well suited for a wide variety of noisy real-world tasks.

$\mathbf{o}_t$, 它们本身是未观察到的状态 $\mathbf{x}_t$ 的函数。在第 4.2 节中, 我们展示了这些假设如何使每个 $p_i(\tau)$ 得到非常有效的优化。我们将 $p_i(\tau)$ 称为指导分布, 因为它们用于将策略集中在良好的、低成本的行为上。

除了学习 $p_i(\tau)$ 之外, 我们还必须选择一种易于处理的方式来表示无限的约束集 $p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)$ 。之前的工作中提出的一种近似方法是用特征期望代替精确约束 (Peters et al., 2010)。当特征由随机变量的线性、二次或高阶单项式函数组成时, 这可以被视为对分布矩的约束。如果我们只使用第一时刻, 我们就会得到对预期动作的约束: $E_{p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t] = E_{\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t]$ 。如果动力学中的随机性较低, 正如我们之前假设的那样, 每个 $p_i(\tau)$ 的最优解将具有较低的熵, 从而使一阶矩约束成为合理的近似值。增广拉格朗日量中的 KL 散度项仍将有助于温和地强制执行更高矩之间的一致性。虽然这种简化相当彻底, 但我们发现它在实践中比包含更高矩更稳定, 可能是因为使用有限数量的样本很难准确估计这些更高矩。交替优化现在由下式给出

$$\theta \leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^\theta(\theta, p) \quad (2)$$

$$p \leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^p(p, \theta) \quad (3)$$

$$\lambda_{\mu t} \leftarrow \lambda_{\mu t} + \alpha \nu_t (E_{\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t] - E_{p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t]),$$

其中 $\lambda_{\mu t}$ 是时间 t 时预期动作的拉格朗日乘子。在本文的其余部分, 我们将使用 $\mathcal{L}_\theta(\theta, p)$ 和 $\mathcal{L}_p(p, \theta)$ 分别表示方程 (2) 和 (3) 中的两个增广拉格朗日量。在接下来的两节中, 我们将描述如何在未知动态下针对 p 优化 $\mathcal{L}_p(p, \theta)$, 以及如何针对复杂的高维策略优化 $\mathcal{L}_\theta(\theta, p)$ 。BADMM 优化的实施细节在附录 A.1 中给出。

4.2 Trajectory Optimization under Unknown Dynamics

由于上一节中的拉格朗日 $\mathcal{L}_p(p, \theta)$ 分解了 $p(\tau) = \sum_i p_i(\tau)$ 中的混合元素, 因此我们描述单个高斯 $p(\tau)$ 的轨迹优化方法。当存在多个混合元素时, 此过程并行应用于每个 $p_i(\tau)$ 。由于 $p(\tau)$ 是高斯分布, 因此对应于动力学和控制器的条件 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 和 $p(\mathbf{u}_t|\mathbf{x}_t)$ 是时变线性高斯分布, 由下式给出

$$p(\mathbf{u}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t, \mathbf{C}_t) \quad p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t} \mathbf{x}_t + f_{\mathbf{u}t} \mathbf{u}_t + f_{ct}, \mathbf{F}_t).$$

这种类型的控制器可以通过少量的现实世界样本进行有效学习, 使其成为优化引导分布的良好选择。由于针对每个初始状态都拟合了一组不同的时变线性高斯动力学, 因此这种动力学表示可以对任何可以局部线性化的连续确定性系统进行建模。随机动力学原则上可能违反局部线性假设, 但我们发现实际上这种表示非常适合各种嘈杂的现实世界任务。

The dynamics are determined by the environment. If they are known, $p(\mathbf{u}_t|\mathbf{x}_t)$ can be optimized with a variant of the iterative linear-quadratic-Gaussian regulator (iLQG) (Li and Todorov, 2004; Levine and Koltun, 2013a), which is a variant of DDP (Jacobson and Mayne, 1970). In the case of unknown dynamics, we can fit $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ to sample trajectories sampled from the trajectory distribution at the previous iteration, denoted $\hat{p}(\tau)$. If $\hat{p}(\tau)$ is too different from $p(\tau)$, these samples will not give a good estimate of $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$, and the optimization will diverge. To avoid this, we can bound the change from $\hat{p}(\tau)$ to $p(\tau)$ in terms of their KL-divergence by a step size ϵ , producing the following constrained problem:

$$\min_{p(\tau) \in \mathcal{N}(\tau)} \mathcal{L}_p(p, \theta) \text{ s.t. } D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon.$$

This type of policy update has previously been proposed by several authors in the context of policy search (Bagnell and Schneider, 2003; Peters and Schaal, 2008; Peters et al., 2010; Levine and Abbeel, 2014). In the case when $p(\tau)$ is Gaussian, this problem can be solved efficiently using dual gradient descent, while the dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ are fitted to samples gathered by running the previous controller $\hat{p}(\mathbf{u}_t|\mathbf{x}_t)$ on the robot. Fitting a global Gaussian mixture model to tuples $(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1})$ and using it as a prior for fitting the dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ serves to greatly reduce the sample complexity. We describe the dynamics fitting procedure in detail in Appendix A.3.

Note that the trajectory optimization cost function $\mathcal{L}_p(p, \theta)$ also depends on the policy $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$, while we only have access to $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$. In order to compute a local quadratic expansion of the KL-divergence term $D_{\text{KL}}(p(\mathbf{u}_t|\mathbf{x}_t) \parallel \pi_\theta(\mathbf{u}_t|\mathbf{x}_t))$ inside $\mathcal{L}_p(p, \theta)$ for iLQG, we also estimate a linearization of the mean of the conditionally Gaussian policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ with respect to the state $\mathbf{x}_t$, using the same procedure that we use to linearize the dynamics. The data for this estimation consists of tuples $\{\mathbf{x}_t^i, E_{\pi_\theta(\mathbf{u}_t|\mathbf{o}_t^i)}[\mathbf{u}_t]\}$, which we can obtain because both the states $\mathbf{x}_t^i$ and the observations $\mathbf{o}_t^i$ are available for all of the samples evaluated on the real physical system.

This constrained optimization is performed in the “inner loop” of the optimization described in the previous section, and the KL-divergence constraint $D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon$ imposes a step size on the trajectory update. The overall algorithm then becomes an instance of generalized BADMM (Wang and Banerjee, 2014). Note that the augmented Lagrangian $\mathcal{L}_p(p, \theta)$ consists of an expectation under $p(\tau)$ of a quantity that is independent of p . We can locally approximate this quantity with a quadratic by using a quadratic expansion of $\ell(\mathbf{x}_t, \mathbf{u}_t)$, and fitting a linear-Gaussian to $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ with the same method we used for the dynamics. We can then solve the primal optimization in the dual gradient descent procedure with a standard LQR backward pass. This is significantly simpler and much faster than the forward-backward dynamic programming procedure employed in previous work (Levine and Abbeel, 2014; Levine and Koltun, 2014). This improvement is enabled by the use of BADMM, which allows us to always formulate the KL-divergence term in the Lagrangian with the distribution being optimized as the first argument. Since the KL-divergence is convex in its first argument, this makes the corresponding optimization significantly easier. The details of this LQR-based dual gradient descent algorithm are derived in Appendix A.4.

We can further improve the efficiency of the method by allowing samples from multiple trajectories $p_i(\tau)$ to be used to fit a shared dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$, while the controllers $p_i(\mathbf{u}_t|\mathbf{x}_t)$ are allowed to vary. This makes sense when the initial states of these trajectories

动态是由环境决定的。如果已知，则可以使用迭代线性二次高斯调节器 (iLQG) 的变体 (Li 和 Todorov, 2004; Levine 和 Koltun, 2013a) 来优化 $p(\mathbf{u}_t|\mathbf{x}_t)$ ，该变体是 DDP 的变体 (Jacobson 和 Mayne, 1970)。在未知动力学的情况下，我们可以将 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 拟合到从上一次迭代的轨迹分布中采样的样本轨迹，表示为 $\hat{p}(\tau)$ 。如果 $\hat{p}(\tau)$ 与 $p(\tau)$ 相差太大，这些样本将无法很好地估计 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ ，并且优化将会发散。为了避免这种情况，我们可以通过步长 ϵ 来限制从 $\hat{p}(\tau)$ 到 $p(\tau)$ 的 KL 散度变化，从而产生以下约束问题：

$$\min_{p(\tau) \in \mathcal{N}(\tau)} \mathcal{L}_p(p, \theta) \text{ s.t. } D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon.$$

这种类型的政策更新先前已由多位作者在政策搜索的背景下提出 (Bagnell 和 Schneider, 2003; Peters 和 Schaal, 2008; Peters 等人, 2010; Levine 和 Abbeel, 2014)。在 $p(\tau)$ 为高斯分布的情况下，可以使用双梯度下降有效地解决这个问题，而动态 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 则适合通过在机器人上运行先前的控制器 $\hat{p}(\mathbf{u}_t|\mathbf{x}_t)$ 收集的样本。将全局高斯混合模型拟合到元组 $(\mathbf{x}_t, \mathbf{u}_t, \mathbf{x}_{t+1})$ 并将其用作拟合动态 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 的先验，可以大大降低样本复杂性。我们在附录 A.3 中详细描述了动力学拟合过程。

请注意，轨迹优化成本函数 $\mathcal{L}_p(p, \theta)$ 也取决于策略 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ ，而我们只能访问 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 。为了计算 iLQG 的 $\mathcal{L}_p(p, \theta)$ 内 KL 散度项 $D_{\text{KL}}(p(\mathbf{u}_t|\mathbf{x}_t) \parallel \pi_\theta(\mathbf{u}_t|\mathbf{x}_t))$ 的局部二次展开，我们还使用与动态线性化相同的过程来估计条件高斯策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 相对于状态 $\mathbf{x}_t$ 的均值的线性化。此估计的数据由元组 $\{\mathbf{x}_t^i, E_{\pi_\theta(\mathbf{u}_t|\mathbf{o}_t^i)}[\mathbf{u}_t]\}$ 组成，我们可以获得这些数据，因为状态 $\mathbf{x}_t^i$ 和观测值 $\mathbf{o}_t^i$ 对于在真实物理系统上评估的所有样本都可用。

这种约束优化是在上一节中描述的优化的“内循环”中执行的，并且 KL 散度约束 $D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon$ 对轨迹更新施加了步长。然后整个算法就成为广义 BADMM 的一个实例 (Wang 和 Banerjee, 2014)。请注意，增广拉格朗日 $\mathcal{L}_p(p, \theta)$ 由独立于 p 的量的 $p(\tau)$ 下的期望组成。我们可以通过使用 $\ell(\mathbf{x}_t, \mathbf{u}_t)$ 的二次展开来局部近似这个量，并使用我们用于动力学的相同方法将线性高斯拟合到 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 。然后，我们可以使用标准 LQR 向后传递来解决双梯度下降过程中的原始优化问题。这比之前的工作中使用的前向-后向动态规划过程要简单得多，速度也快得多 (Levine 和 Abbeel, 2014 年; Levine 和 Koltun, 2014 年)。这种改进是通过使用 BADMM 实现的，它允许我们始终以拉格朗日函数形式表示 KL 散度项，并将优化的分布作为第一个参数。由于 KL 散度的第一个参数是凸的，这使得相应的优化变得更加容易。这种基于 LQR 的双梯度下降算法的详细信息参见附录 A.4。

我们可以通过允许使用多个轨迹 $p_i(\tau)$ 的样本来拟合共享动态 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ ，同时允许控制器 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 变化，从而进一步提高该方法的效率。当这些轨迹的初始状态

are similar, and they therefore visit similar regions. This allows us to draw just a single sample from each $p_i(\tau)$ at each iteration, allowing us to handle many more initial states.

4.3 Supervised Policy Optimization

Since the policy parameters θ participate only in the constraints of the optimization problem in Equation (1), optimizing the policy corresponds to minimizing the KL-divergence between the policy and trajectory distribution, as well as the expectation of $\lambda_{\mu t}^T \mathbf{u}_t$. For a conditional Gaussian policy of the form $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t) = \mathcal{N}(\mu^\pi(\mathbf{o}_t), \Sigma^\pi(\mathbf{o}_t))$, the objective is

$$\begin{aligned} \mathcal{L}_\theta(\theta, p) = & \frac{1}{2N} \sum_{i=1}^N \sum_{t=1}^T E_{p_i(\mathbf{x}_t, \mathbf{o}_t)} [\text{tr}[\mathbf{C}_{ti}^{-1} \Sigma^\pi(\mathbf{o}_t)] - \log |\Sigma^\pi(\mathbf{o}_t)| \\ & + (\mu^\pi(\mathbf{o}_t) - \mu_{ti}^p(\mathbf{x}_t)) \mathbf{C}_{ti}^{-1} (\mu^\pi(\mathbf{o}_t) - \mu_{ti}^p(\mathbf{x}_t)) + 2\lambda_{\mu t}^T \mu^\pi(\mathbf{o}_t)], \end{aligned}$$

where $\mu_{ti}^p(\mathbf{x}_t)$ is the mean of $p_i(\mathbf{u}_t|\mathbf{x}_t)$ and $\mathbf{C}_{ti}$ is the covariance, and the expectation is evaluated using samples from each $p_i(\tau)$ with corresponding observations $\mathbf{o}_t$. The observations are sampled from $p(\mathbf{o}_t|\mathbf{x}_t)$ by recording camera images on the real system. Since the input to $\mu^\pi(\mathbf{o}_t)$ and $\Sigma^\pi(\mathbf{o}_t)$ is not the state $\mathbf{x}_t$, but only an observation $\mathbf{o}_t$, we can train the policy to directly use raw observations. Note that $\mathcal{L}_\theta(\theta, p)$ is simply a weighted quadratic loss on the difference between the policy mean and the mean action of the trajectory distribution, offset by the Lagrange multiplier. The weighting is the precision matrix of the conditional in the trajectory distribution, which is equal to the curvature of its cost-to-go function (Levine and Koltun, 2013a). This has an intuitive interpretation: $\mathcal{L}_\theta(\theta, p)$ penalizes deviation from the trajectory distribution, with a penalty that is locally proportional to its cost-to-go. At convergence, when the policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ takes the same actions as $p_i(\mathbf{u}_t|\mathbf{x}_t)$, their Q-functions are equal, and the supervised policy objective becomes equivalent to the policy iteration objective (Levine and Koltun, 2014).

In this work, we optimize $\mathcal{L}_\theta(\theta, p)$ with respect to θ using stochastic gradient descent (SGD), a standard method for neural network training. The covariance of the Gaussian policy does not depend on the observation in our implementation, though adding this dependence would be straightforward. Since training complex neural networks requires a substantial number of samples, we found it beneficial to include sampled observations from previous iterations into the policy optimization, evaluating the action $\mu_{ti}^p(\mathbf{x}_t)$ at their corresponding states using the current trajectory distributions. Since these samples come from the wrong state distribution, we use importance sampling and weight them according to the ratio of their probability under the current distribution $p(\mathbf{x}_t)$ and the one they were sampled from, which is straightforward to evaluate under the estimated linear-Gaussian dynamics (Levine and Koltun, 2013b).

4.4 Comparison with Prior Guided Policy Search Methods

We presented a guided policy search method where the policy is trained on observations, while the trajectories are trained on the full state. The BADMM formulation of guided policy search is new to this work, though several prior guided policy search methods based on constrained optimization have been proposed. Levine and Koltun (2014) proposed a formulation similar to Equation (1), but with a constraint on the KL-divergence between

相似，因此他们访问相似的地区。这使我们能够在每次迭代时从每个 $p_i(\tau)$ 中仅抽取一个样本，从而使我们能够处理更多的初始状态。

4.3 Supervised Policy Optimization

由于策略参数 θ 仅参与等式 (1) 中优化问题的约束，因此优化策略对应于最小化策略和轨迹分布之间的KL散度以及 $\lambda_{\mu t}^T \mathbf{u}_t$ 的期望。对于 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t) = \mathcal{N}(\mu^\pi(\mathbf{o}_t), \Sigma^\pi(\mathbf{o}_t))$ 形式的条件高斯策略，目标是

$$\begin{aligned} \mathcal{L}_\theta(\theta, p) = & \frac{1}{2N} \sum_{i=1}^N \sum_{t=1}^T E_{p_i(\mathbf{x}_t, \mathbf{o}_t)} [\text{tr}[\mathbf{C}_{ti}^{-1} \Sigma^\pi(\mathbf{o}_t)] - \log |\Sigma^\pi(\mathbf{o}_t)| \\ & + (\mu^\pi(\mathbf{o}_t) - \mu_{ti}^p(\mathbf{x}_t)) \mathbf{C}_{ti}^{-1} (\mu^\pi(\mathbf{o}_t) - \mu_{ti}^p(\mathbf{x}_t)) + 2\lambda_{\mu t}^T \mu^\pi(\mathbf{o}_t)], \end{aligned}$$

其中 $\mu_{ti}^p(\mathbf{x}_t)$ 是 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 的平均值， $\mathbf{C}_{ti}$ 是协方差，并且使用来自每个 $p_i(\tau)$ 的样本以及相应的观测值 $\mathbf{o}_t$ 来评估期望。通过在真实系统上记录摄像机图像，从 $p(\mathbf{o}_t|\mathbf{x}_t)$ 中采样观测值。由于 $\mu^\pi(\mathbf{o}_t)$ 和 $\Sigma^\pi(\mathbf{o}_t)$ 的输入不是状态 $\mathbf{x}_t$ ，而只是观察值 $\mathbf{o}_t$ ，因此我们可以训练策略直接使用原始观察值。请注意， $\mathcal{L}_\theta(\theta, p)$ 只是策略均值和轨迹分布的平均动作之间差异的加权二次损失，由拉格朗日乘数抵消。权重是轨迹分布中条件的精度矩阵，等于其成本函数的曲率 (Levine and Koltun, 2013a)。这有一个直观的解释： $\mathcal{L}_\theta(\theta, p)$ 惩罚偏离轨迹分布的行为，其惩罚与其成本成正比。在收敛时，当策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 采取与 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 相同的操作时，它们的 Q 函数相等，并且监督策略目标变得等于策略迭代目标 (Levine 和 Koltun, 2014)

在这项工作中，我们使用随机梯度下降 (SGD) (神经网络训练的标准方法) 相对于 θ 优化 $\mathcal{L}_\theta(\theta, p)$ 。高斯策略的协方差并不依赖于我们实现中的观察，尽管添加这种依赖关系是很简单的。由于训练复杂的神经网络需要大量样本，我们发现将先前迭代的采样观察结果纳入策略优化中是有益的，使用当前轨迹分布评估相应状态下的动作 $\mu_{ti}^p(\mathbf{x}_t)$ 。由于这些样本来自错误的状态分布，我们使用重要性采样，并根据它们在当前分布 $p(\mathbf{x}_t)$ 下的概率与它们采样的概率之比对它们进行加权，这在估计的线性高斯动力学下很容易评估 (Levine 和 Koltun, 2013b)。

4.4 Comparison with Prior Guided Policy Search Methods

我们提出了一种引导策略搜索方法，其中策略根据观察进行训练，而轨迹根据完整状态进行训练。尽管已经提出了几种基于约束优化的先前引导策略搜索方法，但引导策略搜索的 BADMM 公式对于这项工作来说是新的。Levine 和 Koltun (2014) 提出了类似于方程 (1) 的公式，但对之间的 KL 散度有限制

$p(\tau)$ and π_θ . This results in a more complex, non-convex forward-backward trajectory optimization phase. Since the BADMM formulation solves a convex problem during the trajectory optimization phase, it is substantially faster and easier to implement and use, especially when the number of trajectories $p_i(\tau)$ is large.

The use of ADMM for guided policy search was also proposed by Mordatch and Todorov (2014) for deterministic policies under known dynamics. This approach requires known, deterministic dynamics and trains deterministic policies. Furthermore, because this approach uses a simple quadratic augmented Lagrangian term, it further requires penalty terms on the gradient of the policy to account for local feedback. Our approach enforces this feedback behavior due to the higher moments included in the KL-divergence term, but does not require computing the second derivative of the policy.

5. End-to-End Visuomotor Policies

Guided policy search allows us to optimize complex, high-dimensional policies with raw observations, such as when the input to the policy consists of images from a robot’s onboard camera. However, leveraging this capability to directly learn policies for visuomotor control requires designing a policy representation that is both data-efficient and capable of learning complex control strategies directly from raw visual inputs. In this section, we describe a deep convolutional neural network (CNN) model that is uniquely suited to this task. Our approach combines a novel spatial soft-argmax layer with a pretraining procedure that provides for flexibility and data-efficiency.

5.1 Visuomotor Policy Architecture

Our visuomotor policy runs at 20 Hz on the robot, mapping monocular RGB images and the robot configurations to joint torques on a 7 DoF arm. The configuration includes the angles of the joints and the pose of the end-effector (defined by 3 points in the space of the end-effector), as well as their velocities, but does not include the position of the target object or goal, which must be determined from the image. CNNs often use pooling to discard the locational information that is necessary to determine positions, since it is an irrelevant distractor for tasks such as object classification (Lee et al., 2009). Because locational information is important for control, our policy does not use pooling. Additionally, CNNs built for spatial tasks such as human pose estimation often also rely on the availability of location labels in image-space, such as hand-labeled keypoints (Tompson et al., 2014). We propose a novel CNN architecture capable of estimating spatial information from an image without direct supervision in image space. Our pose estimation experiments, discussed in Section 5.2, show that this network can learn useful visual features using only 3D position information provided by the robot, and no camera calibration. Further training the network with guided policy search to directly output motor torques causes it to acquire *task-specific* visual features. Our experiments in Section 6.4 show that this improves performance beyond the level achieved with features trained only for pose estimation.

Our network architecture is shown in Figure 2. The visual processing layers of the network consist of three convolutional layers, each of which learns a bank of filters that are applied to patches centered on every pixel of its input. These filters form a hierarchy of local image features. Each convolutional layer is followed by a rectifying nonlinearity of

$p(\tau)$ 和 π_{θ} 。这导致更复杂的、非凸的前向-后向轨迹优化阶段。由于 BADMM 公式在轨迹优化阶段解决了凸问题，因此它的实现和使用速度更快且更容易，特别是当轨迹 $p_i(\tau)$ 的数量很大时。

Mordatch 和 Todorov (2014) 还提出使用 ADMM 进行引导策略搜索，以实现已知动态下的确定性策略。这种方法需要已知的确定性动态并训练确定性策略。此外，由于该方法使用简单的二次增广拉格朗日项，因此还需要策略梯度的惩罚项以考虑局部反馈。由于 KL 散度项中包含更高的矩，我们的方法强制执行这种反馈行为，但不需要计算策略的二阶导数。

5. End-to-End Visuomotor Policies

引导策略搜索使我们能够通过原始观察来优化复杂的高维策略，例如当策略的输入包含来自机器人机载摄像头的图像时。然而，利用这种能力直接学习视觉运动控制策略需要设计一种策略表示，该策略表示既具有数据效率，又能够直接从原始视觉输入中学习复杂的控制策略。在本节中，我们将描述一种特别适合此任务的深度卷积神经网络 (CNN) 模型。我们的方法将新颖的空间 soft-argmax 层与预训练程序相结合，提供灵活性和数据效率。

5.1 Visuomotor Policy Architecture

我们的视觉运动策略在机器人上以 20 Hz 的频率运行，将单眼 RGB 图像和机器人配置映射到 7 DoF 手臂上的关节扭矩。配置包括关节的角度和末端执行器的姿态（由末端执行器空间中的3个点定义）以及它们的速度，但不包括目标物体或目标的位置，这必须从图像中确定。CNN 经常使用池化来丢弃确定位置所需的位置信息，因为它对于对象分类等任务来说是不相关的干扰因素（Lee 等，2009）。由于位置信息对于控制很重要，因此我们的策略不使用池化。此外，为空间任务（例如人体姿势估计）构建的 CNN 通常也依赖于图像空间中位置标签的可用性，例如手动标记的关键点（Tompson 等人，2014）。我们提出了一种新颖的 CNN 架构，能够估计图像的空间信息，而无需在图像空间中进行直接监督。我们的姿态估计实验（第 5.2 节中讨论）表明，该网络可以仅使用机器人提供的 3D 位置信息来学习有用的视觉特征，而无需相机校准。通过引导策略搜索进一步训练网络以直接输出电机扭矩，使其获得 *task-specific* 视觉特征。我们在第 6.4 节中的实验表明，这提高了性能，超出了仅针对姿态估计训练的特征所达到的水平。

我们的网络架构如图 2 所示。网络的视觉处理层由三个卷积层组成，每个卷积层学习一组滤波器，这些滤波器应用于以其输入的每个像素为中心的补丁。这些过滤器形成了局部图像特征的层次结构。每个卷积层后面都有一个校正非线性

the form $a_{cij} = \max(0, z_{cij})$ for each channel c and each pixel coordinate (i, j) . The third convolutional layer contains 32 response maps with resolution 109×109 . These response maps are passed through a spatial softmax function of the form $s_{cij} = e^{a_{cij}} / \sum_{i'j'} e^{a_{ci'j'}}$. Each output channel of the softmax is a probability distribution over the location of a feature in the image. To convert from this distribution to a coordinate representation (f_{cx}, f_{cy}) , the network calculates the expected image position of each feature, yielding a 2D coordinate for each channel: $f_{cx} = \sum_{ij} s_{cij} x_{ij}$ and $f_{cy} = \sum_{ij} s_{cij} y_{ij}$, where (x_{ij}, y_{ij}) is the image-space position of the point (i, j) in the response map. Since this is a linear operation, it corresponds to a fixed, sparse fully connected layer with weights $W_{cix} = x_{ij}$ and $W_{c jy} = y_{ij}$. The combination of the spatial softmax and expectation operator implement a kind of soft-argmax. The spatial feature points (f_{cx}, f_{cy}) are concatenated with the robot’s configuration and fed into two fully connected layers, each with 40 rectified units, followed by linear connections to the torques. The full network contains about 92,000 parameters, of which 86,000 are in the convolutional layers.

The spatial softmax and the expected position computation serve to convert pixel-wise representations in the convolutional layers to spatial coordinate representations, which can be manipulated by the fully connected layers into 3D positions or motor torques. The softmax also provides lateral inhibition, which suppresses low, erroneous activations, only keeping strong activations that are more likely to be accurate. This makes our policy more robust to distractors, providing generalization to novel visual variation. We compare our architecture with more standard alternatives in Section 6.3 and evaluate robustness to visual distractors in Section 6.4. However, the proposed architecture is also in some sense more specialized for visuomotor control, in contrast to more general standard convolutional networks. For example, not all perception tasks require information that can be coherently summarized by a set of spatial locations.

5.2 Visuomotor Policy Training

The guided policy search trajectory optimization phase uses the full state of the system, though the final policy only uses the observations. This type of instrumented training is a natural choice for many robotics tasks, where the robot is trained under controlled conditions, but must then act intelligently in uncontrolled, real-world situations. In our tasks, the unobserved variables are the pose of a target object (e.g. the bottle on which a cap must be placed). During training, this target object is typically held in the robot’s left gripper, while the robot’s right arm performs the task, as shown to the right. This allows the robot to move the target through a range of known positions. The final visuomotor policy does not receive this position as input, but must instead use the camera images. Due to the modest amount of training data, distractors that are correlated with task-relevant variables can hamper generalization. For this reason, the left arm is covered with cloth to prevent the policy from associating its appearance with the object’s position.



每个通道 c 和每个像素坐标 (i, j) 的形式为 $a_{cij} = \max(0, z_{cij})$ 。第三个卷积层包含 32 个分辨率为 109×109 的响应图。这些响应图通过 $s_{cij} = e^{a_{cij}} / \sum_{i'j'} e^{a_{ci'j'}}$ 形式的空间 softmax 函数传递。softmax 的每个输出通道都是图像中特征位置的概率分布。为了将此分布转换为坐标表示 (f_{cx}, f_{cy}) ，网络计算每个特征的预期图像位置，生成每个通道的 2D 坐标：

$f_{cx} = \sum_{ij} s_{cij} x_{ij}$ 和 $f_{cy} = \sum_{ij} s_{cij} y_{ij}$ ，其中 (x_{ij}, y_{ij}) 是响应图中点 (i, j) 的图像空间位置。由于这是一个线性运算，因此它对应于一个固定的稀疏全连接层，其权重为 $W_{cix} = x_{ij}$ 和 $W_{c jy} = y_{ij}$ 。空间 softmax 和期望算子的组合实现了一种 soft-argmax。空间特征点 (f_{cx}, f_{cy}) 与机器人的配置连接起来，并输入两个完全连接的层，每个层都有 40 个整流单元，然后与扭矩进行线性连接。完整网络包含约 92,000 个参数，其中 86,000 个位于卷积层。

空间 softmax 和预期位置计算用于将卷积层中的像素级表示转换为空间坐标表示，该表示可以由全连接层操纵为 3D 位置或电机扭矩。softmax 还提供横向抑制，抑制低的、错误的激活，只保留更有可能准确的强激活。这使得我们的策略对于干扰因素更加稳健，为新颖的视觉变化提供概括。我们在第 6.3 节中将我们的架构与更标准的替代方案进行比较，并在第 6.4 节中评估对视觉干扰因素的鲁棒性。然而，与更通用的标准卷积网络相比，所提出的架构在某种意义上也更专门用于视觉运动控制。例如，并非所有感知任务都需要可以通过一组空间位置连贯地概括的信息。

5.2 Visuomotor Policy Training

引导策略搜索轨迹优化阶段使用系统的完整状态，尽管最终策略仅使用观察结果。这种类型的仪器训练是许多机器人任务的自然选择，其中机器人在受控条件下进行训练，但随后必须在不受控制的现实环境中智能地行动。在我们的任务中，未观察到的变量是目标物体的姿态（例如必须盖上盖子的瓶子）。在训练过程中，目标物体通常被机器人的左夹具握住，而机器人的右臂则执行任务，如右图所示。这允许机器人将目标移动到一系列已知位置。最终的视觉运动策略不接收该位置作为输入，而是必须使用相机图像。由于谦虚

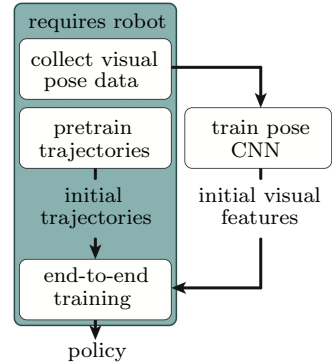


训练数据量、与任务相关变量相关的干扰因素可能会妨碍泛化。因此，左臂上覆盖着布，以防止策略将其外观与物体的位置关联起来。

While we can train the visuomotor policy entirely from scratch, the algorithm would spend a large number of iterations learning basic visual features and arm motions that can more efficiently be learned by themselves, before being incorporated into the policy. To speed up learning, we initialize both the vision layers in the policy and the trajectory distributions for guided policy search by leveraging the fully observed training setup. To initialize the vision layers, the robot moves the target object through a range of random positions, recording camera images and the object’s pose, which is computed automatically from the pose of the gripper. This dataset is used to train a pose regression CNN, which consists of the same vision layers as the policy, followed by a fully connected layer that outputs the 3D points that define the target. Since the training set is still small (we use 1000 images collected from random arm motions), we initialize the filters in the first layer with weights from the model of Szegedy et al. (2014), which is trained on ImageNet (Deng et al., 2009) classification. After training on pose regression, the weights in the convolutional layers are transferred to the policy CNN. This enables the robot to learn the appearance of the objects prior to learning the behavior.

To initialize the linear-Gaussian controllers for each of the initial states, we take 15 iterations of guided policy search without optimizing the visuomotor policy. This allows for much faster training in the early iterations, when the trajectories are not yet successful, and optimizing the full visuomotor policy is unnecessarily time consuming. Since we still want the trajectories to arrive at compatible strategies for each target position, we replace the visuomotor policy during these iterations with a small network that receives the full state, which consisted of two layers with 40 rectified linear hidden units in our experiments. This network serves only to constrain the trajectories and avoid divergent behaviors from emerging for similar initial states, which would make subsequent policy learning difficult.

After initialization, we train the full visuomotor policy with guided policy search. During the supervised policy optimization phase, the fully connected motor control layers are first optimized by themselves, since they are not initialized with pre-training. This can be done very quickly because these layers are small. Then, the entire network is further optimized end-to-end. We found that first training the upper layers before end-to-end optimization prevented the convolutional layers from forgetting useful features learning during pretraining, when the error signal due to the untrained upper layers is very large. The entire pretraining scheme is summarized in the diagram on the right. Note that the trajectories can be pretrained in parallel with the vision layer pretraining, which does not require access to the physical system. Furthermore, the entire initialization procedure does not use any additional information that is not already available from the robot.



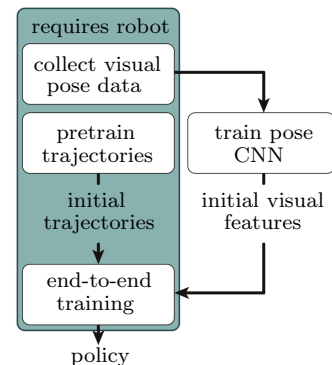
6. Experimental Evaluation

In this section, we present a series of experiments aimed at evaluating our approach and answering the following questions:

虽然我们可以完全从头开始训练视觉运动策略，但该算法将花费大量迭代来学习基本的视觉特征和手臂运动，这些特征和手臂运动可以在纳入策略之前更有效地自行学习。为了加快学习速度，我们利用完全观察到的训练设置来初始化策略中的视觉层和引导策略搜索的轨迹分布。为了初始化视觉层，机器人将目标物体移动到一系列随机位置，记录相机图像和物体的姿势，这是根据夹具的姿势自动计算的。该数据集用于训练姿态回归 CNN，它由与策略相同的视觉层组成，后面是输出定义目标的 3D 点的全连接层。由于训练集仍然很小（我们使用从随机手臂运动收集的 1000 个图像），我们使用 Szegedy 等人的模型中的权重初始化第一层中的滤波器。(2014)，它是在 ImageNet (Deng et al., 2009) 分类上进行训练的。在姿态回归训练之后，卷积层中的权重被转移到策略 CNN。这使得机器人能够在学习行为之前先了解物体的外观。

为了初始化每个初始状态的线性高斯控制器，我们进行 15 次引导策略搜索迭代，而不优化视觉运动策略。当轨迹尚未成功时，这允许在早期迭代中进行更快的训练，并且优化完整的视觉运动策略不必要地耗时。由于我们仍然希望轨迹能够达到每个目标位置的兼容策略，因此我们在这些迭代过程中用一个接收完整状态的小网络替换了视觉运动策略，在我们的实验中，该网络由带有 40 个整流线性隐藏单元的两层组成。该网络仅用于约束轨迹并避免在相似的初始状态下出现不同的行为，这将使后续的政策学习变得困难。

初始化后，我们通过引导策略搜索训练完整的视觉运动策略。在监督策略优化阶段，完全连接的电机控制层首先自行优化，因为它们没有通过预训练进行初始化。这可以很快完成，因为这些层很小。然后，整个网络进一步端到端优化。我们发现，当未经训练的上层导致的误差信号非常大时，在端到端优化之前首先训练上层可以防止卷积层忘记预训练期间学习的有用特征。右图总结了整个预训练方案。请注意，轨迹可以与



视觉层预训练，不需要访问物理系统。此外，整个初始化过程不使用机器人尚未提供的任何附加信息。

6. Experimental Evaluation

在本节中，我们提出了一系列实验，旨在评估我们的方法并回答以下问题：

1. How does the guided policy search algorithm compare to other policy search methods for training complex, high-dimensional policies, such as neural networks?
2. Does our trajectory optimization algorithm work on a real robotic platform with unknown dynamics, for a range of different tasks?
3. How does our spatial softmax architecture compare to other, more standard convolutional neural network architectures?
4. Does training the perception and control systems in a visuomotor policy jointly end-to-end provide better performance than training each component separately?

Evaluating a wide range of policy search algorithms on a real robot would be extremely time consuming, particularly for methods that require a large number of samples. We therefore answer question (1) by using a physical simulator and simpler policies that do not use vision. This also allows us to test the generality of guided policy search on tasks that include manipulation, walking, and swimming. To answer question (2), we present a wide range of experiments on a PR2 robot. These experiments allow us to evaluate the sample efficiency of our trajectory optimization algorithm. To address question (3), we compare a range of different policy architectures on the task of localizing a target object (the cube in the shape sorting cube task). Since localizing the target object is a prerequisite for completing the shape sorting cube task, this serves as a good proxy for evaluating different architectures. Finally, we answer the last and most important question (4) by training visuomotor policies for hanging a coat hanger on a clothes rack, inserting a block into a shape sorting cube, fitting the claw of a toy hammer under a nail with various grasps, and screwing on a bottle cap. These tasks are illustrated in Figure 8.

6.1 Simulated Comparisons to Prior Policy Search Methods

In this section, we compare our method against prior policy search techniques on a range of simulated robotic control tasks. These results previously appeared in our conference paper that introduced the trajectory optimization procedure with local linear models (Levine and Abbeel, 2014). In these tasks, the state $\mathbf{x}_t$ consists of the joint angles and velocities of each robot, and the actions $\mathbf{u}_t$ consist of the torques at each joint. The neural network policies used one hidden layer and soft rectifier nonlinearities of the form $a = \log(1 + \exp(z))$. Since these policies use the state as input, they only have a few hundred parameters, far fewer than our visuomotor policies. However, even this number of parameters can pose a major challenge for prior policy search methods (Deisenroth et al., 2013).

Experimental tasks. We simulated 2D and 3D peg insertion, octopus arm control, and planar swimming and walking. The difficulty in the peg insertion tasks stems from the need to align the peg with the slot and the complex contacts between the peg and the walls, which result in discontinuous dynamics. Octopus arm control involves moving the tip of a flexible arm to a goal position (Engel et al., 2005). The challenge in this task stems from its high dimensionality: the arm has 25 degrees of freedom, corresponding to 50 state dimensions. The swimming task requires controlling a three-link snake, and the walking task requires a seven-link biped to maintain a target velocity. The challenge in these tasks comes from underactuation. Details of the simulation and cost for each task are in Appendix B.1.

1. 引导策略搜索算法与训练复杂、高维策略（例如神经网络）的其他策略搜索方法相比如何？
2. 我们的轨迹优化算法是否可以在动态未知的真实机器人平台上运行，以执行一系列不同的任务？
3. 我们的空间softmax架构与其他更标准的卷积神经网络架构相比如何？
4. 在视觉运动策略中联合训练感知和控制系统是否比单独训练每个组件能提供更好的性能？

在真实机器人上评估各种策略搜索算法将非常耗时，特别是对于需要大量样本的方法。因此，我们通过使用物理模拟器和不使用视觉的更简单的策略来回答问题（1）。这也使我们能够测试引导策略搜索在操作、行走和游泳等任务上的通用性。为了回答问题（2），我们在 PR2 机器人上进行了广泛的实验。这些实验使我们能够评估轨迹优化算法的样本效率。为了解决问题（3），我们比较了定位目标对象（形状排序立方体任务中的立方体）任务的一系列不同策略架构。由于定位目标对象是完成形状排序立方体任务的先决条件，因此这可以作为评估不同架构的良好代理。最后，我们通过训练视觉运动策略来回答最后一个也是最重要的问题（4），例如将衣架挂在衣架上，将块插入形状分类立方体中，将玩具锤的爪子安装在带有各种抓握的钉子下，以及拧上瓶盖。这些任务如图 8 所示。

6.1 Simulated Comparisons to Prior Policy Search Methods

在本节中，我们将我们的方法与先前的策略搜索技术在一系列模拟机器人控制任务上进行比较。这些结果之前出现在我们的会议论文中，该论文介绍了局部线性模型的轨迹优化过程（Levine 和 Abbeel, 2014）。在这些任务中，状态 $\mathbf{x}_t$ 由每个机器人的关节角度和速度组成，动作 $\mathbf{u}_t$ 由每个关节的扭矩组成。神经网络策略使用一个隐藏层和形式为 $a = \log(1 + \exp(z))$ 的软整流器非线性。由于这些政策使用状态作为输入，因此它们只有几百个参数，远少于我们的视觉运动政策。然而，即使如此数量的参数也可能对先前的策略搜索方法构成重大挑战（Deisenroth 等人, 2013）。

Experimental tasks. 我们模拟了 2D 和 3D 钉插入、章鱼臂控制以及平面游泳和行走。钉插入任务的困难源于需要将钉与插槽对齐以及钉与墙壁之间的复杂接触，从而导致不连续的动力学。章鱼臂控制涉及将柔性臂的尖端移动到目标位置（Engel 等, 2005）。这项任务的挑战源于其高维度：手臂有 25 个自由度，对应 50 个状态维度。游泳任务需要控制三连杆蛇，步行任务需要七连杆两足动物来保持目标速度。这些任务的挑战来自于驱动不足。每个任务的模拟和成本的详细信息参见附录 B.1。

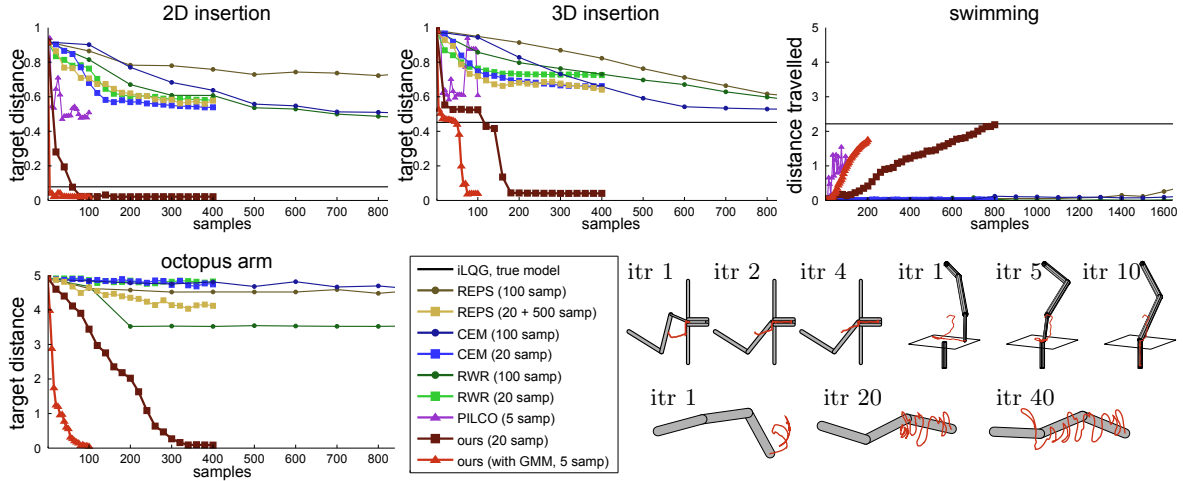


Figure 4: Results for learning linear-Gaussian controllers for 2D and 3D insertion, octopus arm, and swimming. Our approach uses fewer samples and finds better solutions than prior methods, and the GMM further reduces the required sample count. Images in the lower-right show the last time step for each system at several iterations of our method, with red lines indicating end effector trajectories.

Prior methods. We compare to REPS (Peters et al., 2010), reward-weighted regression (RWR) (Peters and Schaal, 2007; Kober and Peters, 2009), the cross-entropy method (CEM) (Rubinstein and Kroese, 2004), and PILCO (Deisenroth and Rasmussen, 2011). We also use iLQG (Li and Todorov, 2004) with a known model as a baseline, shown as a black horizontal line in all plots. REPS is a model-free method that, like our approach, enforces a KL-divergence constraint between the new and old policy. We compare to a variant of REPS that also fits linear dynamics to generate 500 pseudo-samples (Lioutikov et al., 2014), which we label “REPS (20 + 500).” RWR is an EM algorithm that fits the policy to previous samples weighted by the exponential of their reward, and CEM fits the policy to the best samples in each batch. With Gaussian trajectories, CEM and RWR only differ in the weights. These methods represent a class of RL algorithms that fit the policy to weighted samples, including PoWER and PI2 (Kober and Peters, 2009; Theodorou et al., 2010; Stulp and Sigaud, 2012). PILCO is a model-based method that uses a Gaussian process to learn a global dynamics model that is used to optimize the policy. We used the open-source implementation of PILCO provided by the authors. Both REPS and PILCO require solving large nonlinear optimizations at each iteration, while our method does not. Our method used 5 rollouts with the Gaussian mixture model prior, and 20 without. Due to its computational cost, PILCO was provided with 5 rollouts per iteration, while other prior methods used 20 and 100. For all prior methods with free hyperparameters (such as the fraction of elites for CEM), we performed hyperparameter sweeps and chose the most successful settings for the comparison.

Gaussian trajectory distributions. In the first set of comparisons, we evaluate only the trajectory optimization procedure for training linear-Gaussian controllers under unknown dynamics to determine its sample-efficiency and applicability to complex, high-dimensional problems. The results of this comparison for the peg insertion, octopus arm, and swimming

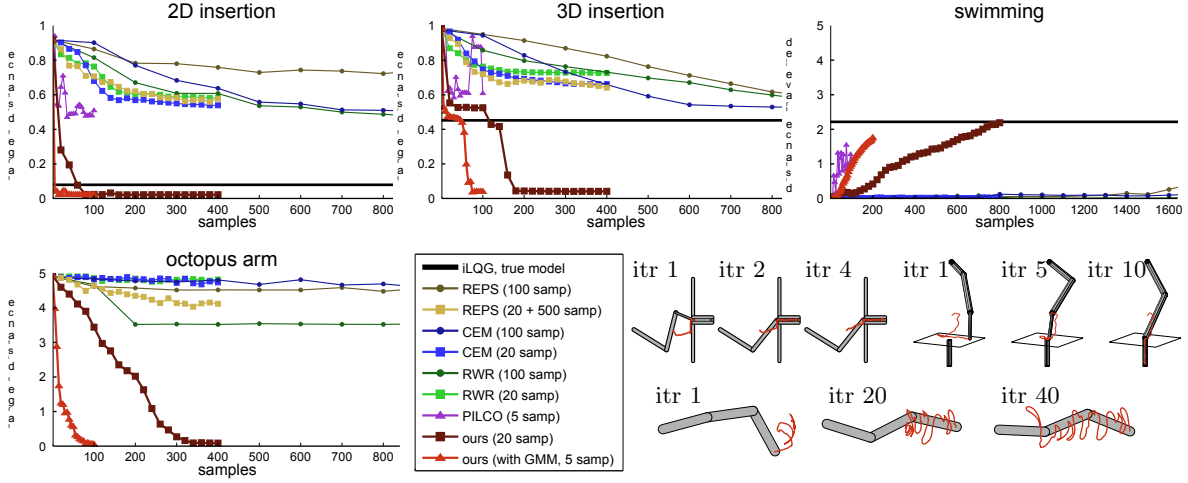


图 4: 学习 2D 和 3D 插入、章鱼臂和游泳的线性高斯控制器的结果。与之前的方法相比，我们的方法使用更少的样本并找到更好的解决方案，并且 GMM 进一步减少了所需的样本数量。右下角的图像显示了每个系统在我们的方法的几次迭代中的最后一个时间步长，红线表示末端执行器轨迹。

Prior methods. 我们与 REPS (Peters 等人, 2010)、奖励加权回归 (RWR) (Peters 和 Schaal, 2007; Kober 和 Peters, 2009)、交叉熵方法 (CEM) (Rubinstein 和 Kroese, 2004) 和 PILCO (Deisenroth 和 Rasmussen, 2011) 进行比较。我们还使用 iLQG (Li 和 Todorov, 2004) 和已知模型作为基线，在所有图中显示为黑色水平线。REPS 是一种无模型方法，与我们的方法一样，它在新旧策略之间强制执行 KL 散度约束。我们与 REPS 的变体进行比较，该变体也适合线性动力学来生成 500 个伪样本 (Lioutikov 等人, 2014)，我们将其标记为“REPS (20 + 500)”。RWR 是一种 EM 算法，它将策略拟合到之前的样本（按奖励指数加权），而 CEM 算法将策略拟合到每批中的最佳样本。对于高斯轨迹，CEM 和 RWR 仅权重不同。这些方法代表了一类适合加权样本策略的强化学习算法，包括 POWER 和 PI2 (Kober 和 Peters, 2009; Theodorou 等人, 2010; Stulp 和 Sigaud, 2012)。PILCO 是一种基于模型的方法，它使用高斯过程来学习用于优化策略的全局动态模型。我们使用了作者提供的 PILCO 的开源实现。REPS 和 PILCO 都需要在每次迭代中求解大型非线性优化，而我们的方法则不需要。我们的方法在使用高斯混合模型之前使用了 5 次推出，在没有使用高斯混合模型的情况下使用了 20 次。由于其计算成本，PILCO 每次迭代提供 5 次推出，而其他先前的方法使用 20 和 100 次。对于所有具有免费超参数的先前方法（例如 CEM 的精英分数），我们执行超参数扫描并选择最成功的设置进行比较。

Gaussian trajectory distributions. 在第一组比较中，我们仅评估在未知动态下训练线性高斯控制器的轨迹优化程序，以确定其样本效率和对复杂高维问题的适用性。钉插入、章鱼臂和游泳的比较结果

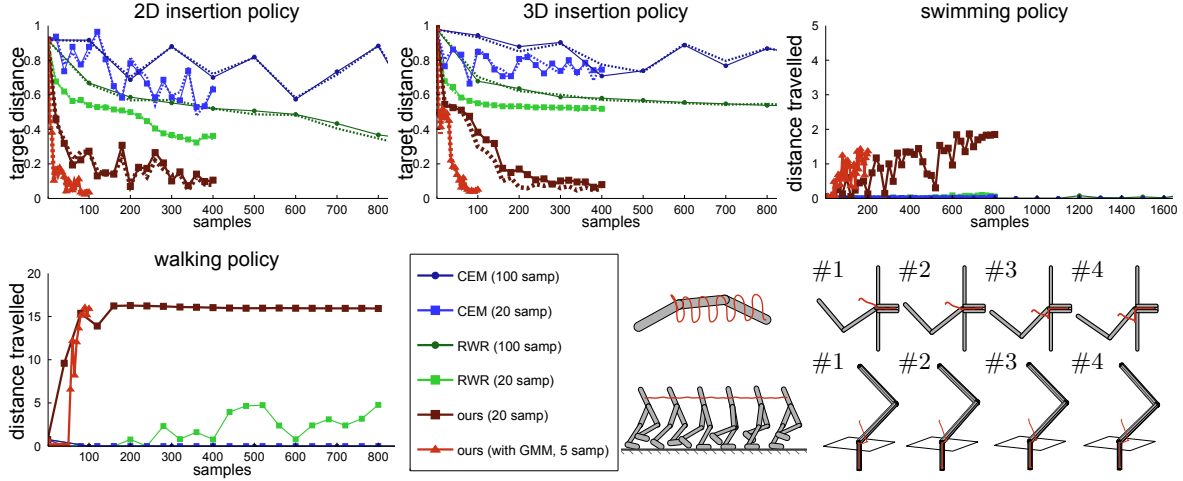


Figure 5: Comparison on neural network policies. For insertion, the policy was trained to search for an unknown slot position on four slot positions (shown above). Generalization to new positions is graphed with dashed lines. Note how the end effector (red) follows the surface to find the slot, and how the swimming gait is smoother due to the stationary policy.

tasks appears in Figure 4. The horizontal axis shows the total number of samples, and the vertical axis shows the minimum distance between the end of the peg and the bottom of the slot, the distance to the target for the octopus arm, or the total distance travelled by the swimmer. Since the peg is 0.5 units long, distances above this amount correspond to controllers that cannot perform an insertion. Our method learned much more effective controllers with fewer samples, especially when using the Gaussian mixture model prior. On 3D insertion, it outperformed the iLQG baseline, which used a known model. Contact discontinuities cause problems for derivative-based methods like iLQG, as well as methods like PILCO that learn a smooth global dynamics model. We use a time-varying local model, which preserves more detail, and fitting the model to samples has a smoothing effect that mitigates discontinuity issues. Prior policy search methods could servo to the hole, but were unable to insert the peg. On the octopus arm, our method succeeded despite the high dimensionality of the state and action spaces.¹ Our method also successfully learned a swimming gait, while prior model-free methods could not initiate forward motion. PILCO also learned an effective gait due to the smooth dynamics of this task, but its GP-based optimization required orders of magnitude more computation time than our method, taking about 50 minutes per iteration. In the case of prior model-free methods, the high dimensionality of the time-varying linear-Gaussian controllers likely caused considerable difficulty (Deisenroth et al., 2013), while our approach exploits the structure of linear-Gaussian controllers for efficient learning.

1. The high dimensionality of the octopus arm made it difficult to run PILCO, though in principle, such methods should perform well on this task given the arm’s smooth dynamics.

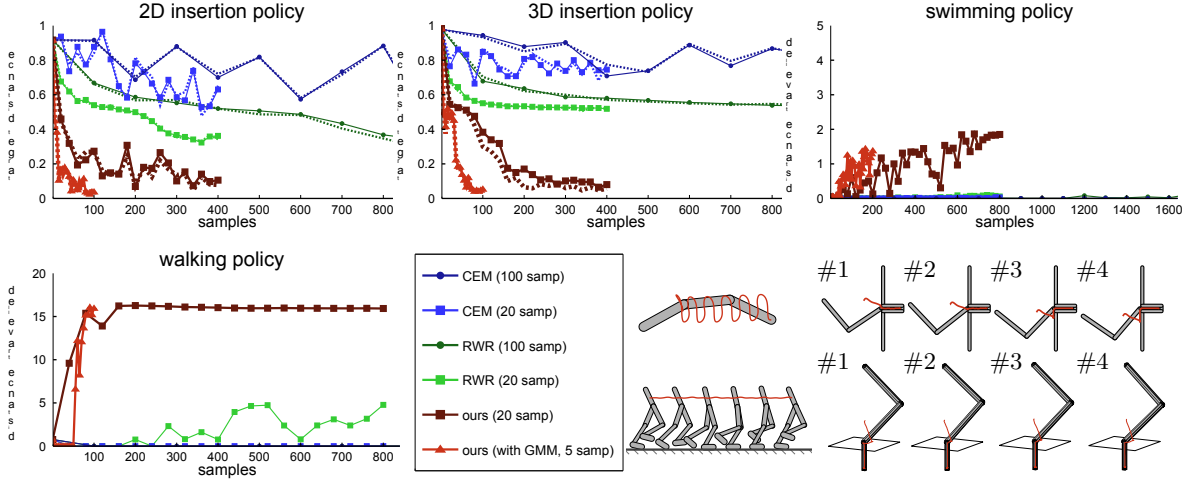


图 5: 神经网络策略比较。对于插入, 策略被训练为在四个槽位置上搜索未知的槽位置 (如上所示)。对新位置的概括用虚线绘制。请注意末端执行器 (红色) 如何跟随表面找到槽, 以及游泳步态如何由于静止策略而变得更加平滑。

任务如图 4 所示。横轴显示样本总数, 纵轴显示挂钩末端与槽底部之间的最小距离、章鱼臂到目标的距离或游泳者游走的总距离。由于钉子的长度为 0.5 个单位, 因此超过此长度的距离对应于无法执行插入的控制器。我们的方法用更少的样本学习了更有效的控制器, 特别是在使用先验高斯混合模型时。在 3D 插入方面, 它的性能优于使用已知模型的 iLQG 基线。接触不连续性会给 iLQG 等基于导数的方法以及学习平滑全局动力学模型的 PILCO 等方法带来问题。我们使用时变局部模型, 它保留了更多细节, 并且将模型拟合到样本具有平滑效果, 可以减轻不连续性问题。先前的政策搜索方法可以伺服到这个洞, 但无法插入钉子。在章鱼手臂上, 尽管状态和动作空间具有高维度, 我们的方法还是成功了。¹我们的方法还成功地学习了游泳步态, 而之前的无模型方法无法启动向前运动。由于该任务的平滑动态, PILCO 还学习了有效的步态, 但其基于 GP 的优化需要比我们的方法多几个数量级的计算时间, 每次迭代大约需要 50 分钟。在现有的无模型方法的情况下, 时变线性高斯控制器的高维性可能会造成相当大的困难 (Deisenroth et al., 2013), 而我们的方法利用线性高斯控制器的结构来进行有效的学习。

1. The high dimensionality of the octopus arm made it difficult to run PILCO, though in principle, such methods should perform well on this task given the arm's smooth dynamics.

Neural network policies. In the second set of comparisons, shown in Figure 5, we compare guided policy search to RWR and CEM² on the challenging task of training high-dimensional neural network policies for the peg insertion and locomotion tasks. The variant of guided policy search used in this comparison differs somewhat from the version described in Section 4, in that it used a simpler dual gradient descent formulation, rather than BADMM. In practice, we found the performance of these methods to be very similar, though the BADMM variant was substantially faster and easier to implement.

On swimming, our method achieved similar performance to the linear-Gaussian case, but since the neural network policy was stationary, the resulting gait was much smoother. Previous methods could only solve this task with 100 samples per iteration, with RWR eventually obtaining a distance of 0.5m after 4000 samples, and CEM reaching 2.1m after 3000. Our method was able to reach such distances with many fewer samples. Following prior work (Levine and Koltun, 2013a), the walker trajectory was initialized from a demonstration, which was stabilized with simple linear feedback. The RWR and CEM policies were initialized with samples from this controller to provide a fair comparison. The graph shows the average distance travelled on rollouts that did not fall, and shows that only our method was able to learn walking policies that succeeded consistently.

On peg insertion, the neural network was trained to insert the peg without precise knowledge of the position of the hole, resulting in a partially observed problem. The holes were placed in a region of radius 0.2 units in 2D and 0.1 units in 3D. The policies were trained on four different hole positions, and then tested on four new hole positions to evaluate generalization. The hole position was not provided to the neural network, and the policies therefore had to search for the hole, with only joint angles and velocities as input. Only our method could acquire a successful strategy to locate both the training and test holes, although RWR was eventually able to insert the peg into one of the four holes in 2D.

These comparisons show that training even medium-sized neural network policies for continuous control tasks with a limited number of samples is very difficult for many prior policy search algorithms. Indeed, it is generally known that model-free policy search methods struggle with policies that have over 100 parameters (Deisenroth et al., 2013). In subsequent sections, we will evaluate our method on real robotic tasks, showing that it can scale from these simulated tasks all the way up to end-to-end learning of visuomotor control.

6.2 Learning Linear-Gaussian Controllers on a PR2 Robot

In this section, we demonstrate the range of manipulation tasks that can be learned using our trajectory optimization algorithm on a real PR2 robot. These experiments previously appeared in our conference paper on guided policy search (Levine et al., 2015). Since performing trajectory optimization is a prerequisite for guided policy search to learn effective visuomotor policies, it is important to evaluate that our trajectory optimization can learn a wide variety of robotic manipulation tasks under unknown dynamics. The tasks in these experiments are shown in Figure 6, while Figure 7 shows the learning curves for each task. For all robotic experiments in this article, the tasks were learned entirely from scratch,

2. PILCO cannot optimize neural network policies, and we could not obtain reasonable results with REPS. Prior applications of REPS generally focus on simpler, lower-dimensional policy classes (Peters et al., 2010; Lioutikov et al., 2014).

Neural network policies. 在第二组比较中, 如图 5 所示, 我们将引导策略搜索与 RWR 和 CEM² 进行比较, 以处理针对钉插入和运动任务训练高维神经网络策略的挑战性任务。本次比较中使用的引导策略搜索的变体与第 4 节中描述的版本有所不同, 因为它使用了更简单的双梯度下降公式, 而不是 BADMM。在实践中, 我们发现这些方法的性能非常相似, 尽管 BADMM 变体速度更快且更容易实现。

在游泳时, 我们的方法实现了与线性高斯情况类似的性能, 但由于神经网络策略是固定的, 因此最终的步态更加平滑。以前的方法每次迭代只能用 100 个样本来解决这个任务, RWR 在 4000 个样本后最终获得 0.5m 的距离, 而 CEM 在 3000 个样本后达到 2.1m。我们的方法能够以更少的样本达到这样的距离。继之前的工作 (Levine 和 Koltun, 2013a) 之后, 步行者轨迹从演示中初始化, 并通过简单的线性反馈来稳定。RWR 和 CEM 策略是使用该控制器中的样本进行初始化的, 以提供公平的比较。该图显示了未下降的首次滚动的平均行驶距离, 并表明只有我们的方法才能学习持续成功的步行策略。

在插入钉子时, 训练神经网络在不精确了解孔位置的情况下插入钉子, 从而导致部分观察到的问题。这些孔放置在 2D 半径为 0.2 单位、3D 半径为 0.1 单位的区域中。这些策略在四个不同的孔位置上进行训练, 然后在四个新的孔位置上进行测试以评估泛化能力。孔位置没有提供给神经网络, 因此策略必须仅以关节角度和速度作为输入来搜索孔。尽管 RWR 最终能够将钉子插入二维的四个孔之一, 但只有我们的方法才能获得成功的策略来定位训练孔和测试孔。

这些比较表明, 对于许多现有的策略搜索算法来说, 使用有限数量的样本来训练即使是中等规模的神经网络策略来执行连续控制任务也是非常困难的。事实上, 众所周知, 无模型策略搜索方法很难处理具有 100 多个参数的策略 (Deisenroth 等, 2013)。在后续部分中, 我们将在真实的机器人任务上评估我们的方法, 表明它可以从这些模拟任务一直扩展到视觉运动控制的端到端学习。

6.2 Learning Linear-Gaussian Controllers on a PR2 Robot

在本节中, 我们将演示可以在真实 PR2 机器人上使用我们的轨迹优化算法来学习的一系列操作任务。这些实验之前出现在我们关于引导策略搜索的会议论文中 (Levine et al., 2015)。由于执行轨迹优化是引导策略搜索学习有效视觉运动策略的先决条件, 因此评估我们的轨迹优化是否可以在未知动态下学习各种机器人操作任务非常重要。这些实验中的任务如图 6 所示, 图 7 显示了每个任务的学习曲线。对于本文中的所有机器人实验, 任务都是完全从头开始学习的,

2. PILCO cannot optimize neural network policies, and we could not obtain reasonable results with REPS. Prior applications of REPS generally focus on simpler, lower-dimensional policy classes (Peters et al., 2010; Lioutikov et al., 2014).

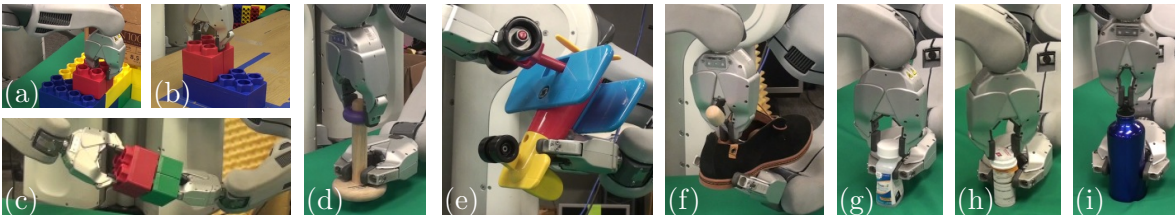


Figure 6: Tasks for linear-Gaussian controller evaluation: (a) stacking lego blocks on a fixed base, (b) onto a free-standing block, (c) held in both gripper; (d) threading wooden rings onto a peg; (e) attaching the wheels to a toy airplane; (f) inserting a shoe tree into a shoe; (g,h) screwing caps onto pill bottles and (i) onto a water bottle.

with the initialization of the controllers $p(\mathbf{u}_t|\mathbf{x}_t)$ described in Appendix B.2. The number of samples required to learn each controller is around 20-25, substantially lower than many prior policy search methods in the literature (Peters and Schaal, 2008; Kober et al., 2010b; Theodorou et al., 2010; Deisenroth et al., 2013). Total learning time was about ten minutes for each task, of which only 3-4 minutes involved system interaction. The rest of the time was spent resetting the robot to the initial state and on computation.

The linear-Gaussian controllers are optimized for a specific condition – e.g., a specific position of the target lego block. To evaluate their robustness to errors in the specified target position, we conducted experiments on the lego block and ring tasks where the target object (the lower block and the peg) was perturbed at each trial during training, and then tested with various perturbations. For each task, controllers were trained with Gaussian perturbations with standard deviations of 0, 1, and 2 cm in the position of the target object, and each controller was tested with perturbations of radius 0, 1, 2, and 3 cm. Note that with a radius of 2 cm, the peg would be placed about one ring-width away from the expected position. The results are shown in Table 2. All controllers were robust to perturbations of 1 cm, and would often succeed at 2 cm. Robustness increased slightly when more noise was injected during training, but even controllers trained without noise exhibited considerable robustness, since the linear-Gaussian controllers themselves add noise during sampling. We also evaluated a kinematic baseline for each perturbation level, which planned a straight path from a point 5 cm above the target to the expected (unperturbed) target location. This baseline was only able to place the lego block in the absence of perturbations. The rounded top of the peg provided an easier condition for the baseline, with occasional successes at higher perturbation levels. Our controllers outperformed the baseline by a wide margin.

All of the robotic experiments discussed in this section may be viewed in the corresponding supplementary video, available online: <http://rll.berkeley.edu/icra2015gps>. A video illustration of the visuomotor policies, discussed in the following sections, is also available: <http://sites.google.com/site/visuomotorpolicy>.

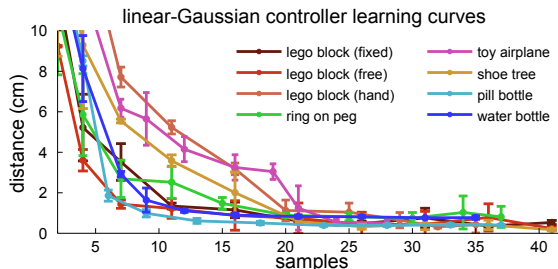


Figure 7: Distance to target point during training of linear-Gaussian controllers. The actual target may differ due to perturbations. Error bars indicate one standard deviation.

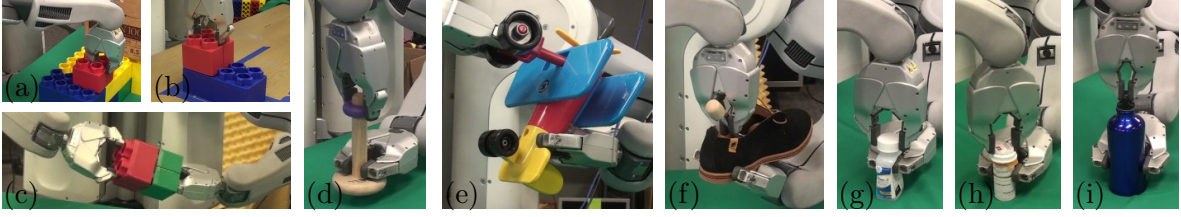


图 6: 线性高斯控制器评估的任务: (a) 将乐高积木堆叠在固定底座上, (b) 堆叠到独立的积木上, (c) 固定在两个夹具中; (d) 将木环穿到钉子上; (e) 将轮子安装到玩具飞机上; (f) 将鞋楦插入鞋子中; (g,h) 将瓶盖拧到药瓶上, 以及 (i) 拧到水瓶上。

控制器 $p(\mathbf{u}_t|\mathbf{x}_t)$ 的初始化如附录 B.2 中所述。学习每个控制器所需的样本数量约为 20-25, 大大低于文献中许多先前的策略搜索方法 (Peters 和 Schaal, 2008; Kober 等人, 2010b; Theodorou 等人, 2010; Deisenroth 等人, 2013)。每个任务的总学习时间约为 10 分钟, 其中只有 3-4 分钟涉及系统交互。剩下的时间用于将机器人重置到初始状态和计算。

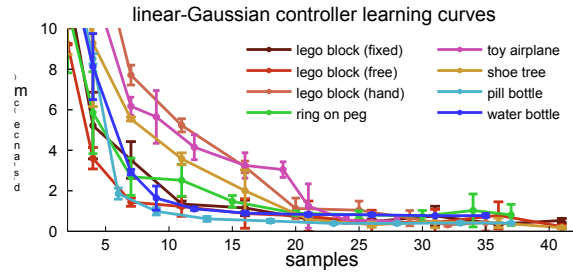


图 7: 线性高斯控制器训练期间到目标点的距离。实际目标可能会因扰动而有所不同。误差线表示一个标准偏差。

线性高斯控制器针对特定条件进行了优化, 例如目标乐高积木的特定位置。为了评估它们对指定目标位置错误的鲁棒性, 我们对乐高积木和环任务进行了实验, 其中目标物体 (下部积木和钉子) 在训练期间的每次试验中都会受到扰动, 然后用各种扰动进行测试。对于每项任务, 控制器均采用目标物体位置标准差为 0、1 和 2 厘米的高斯扰动进行训练, 并且每个控制器均采用半径为 0、1、2 和 3 厘米的扰动进行测试。请注意, 半径为 2 厘米时, 钉子将放置在距预期位置约一圈宽度的位置。结果如表 2 所示。所有控制器对于 1 厘米的扰动都具有鲁棒性, 并且通常在 2 厘米的扰动下也能成功。当在训练期间注入更多噪声时, 鲁棒性略有增加, 但即使是在没有噪声的情况下训练的控制器也表现出相当大的鲁棒性, 因为线性高斯控制器本身在采样期间会增加噪声。我们还评估了每个扰动级别的运动学基线, 该基线规划了从目标上方 5 厘米的点到预期 (未扰动) 目标位置的直线路径。该基线只能在没有扰动的情况下放置乐高积木。钉子的圆形顶部为基线提供了更容易的条件, 在较高扰动水平下偶尔会取得成功。我们的控制器的性能大幅优于基线。

本节讨论的所有机器人实验都可以在相应的补充视频中观看, 该视频可在线观看:

<http://rll.berkeley.edu/icra2015gps>。还提供了以下部分讨论的视觉运动策略的视频说明: <http://sites.google.com/site/visuomotorpolicy>。

		test perturbation							
		lego block				ring on peg			
		0 cm	1 cm	2 cm	3 cm	0 cm	1 cm	2 cm	3 cm
training perturb.	0 cm	5/5	5/5	3/5	2/5	5/5	5/5	0/5	0/5
	1 cm	5/5	5/5	3/5	2/5	5/5	5/5	3/5	0/5
	2 cm	5/5	5/5	5/5	3/5	5/5	5/5	3/5	0/5
	kinematic baseline	5/5	0/5	0/5	0/5	5/5	3/5	0/5	0/5

Table 2: Success rates of linear-Gaussian controllers under target object perturbation.

6.3 Spatial Softmax CNN Architecture Evaluation

In this section, we evaluate the neural network architecture that we propose in Section 5.1 in comparison to more standard convolutional networks. To isolate the architectures from other confounding factors, we measure their accuracy on the pose estimation pretraining task described in Section 5.2. This is a reasonable proxy for evaluating how well the network can overcome two major challenges in visuomotor learning: the ability to handle relatively small datasets without overfitting, and the capability to learn tasks that are inherently spatial. We compare to a network where the expectation operator after the softmax is replaced with a learned fully connected layer, as is standard in the literature, a network where both the softmax and the expectation operators are replaced with a fully connected layer, and a version of this network that also uses 3×3 max pooling with stride 2 at the first two layers. These alternative architectures have many more parameters, since the fully connected layer takes the entire bank of response maps from the third convolutional layer as input. Pooling helps to reduce the number of parameters, but not to the same degree as the spatial softmax and expectation operators in our architecture.

The results in Table 3 indicate that using the softmax and expectation operators improves pose estimation accuracy substantially. Our network is able to outperform the more standard architectures because it is forced by the softmax and expectation operators to learn feature points, which provide a concise representation suitable for spatial inference. Since most of the parameters in this architecture are in the convolutional layers, which benefit from extensive weight sharing, overfitting is also greatly reduced. By removing pooling, our network also maintains higher resolution in the convolutional layers, improving spatial accuracy. Although we did attempt to regularize the larger standard architectures with higher weight decay and dropout, we did not observe a significant improvement on this dataset. We also did not extensively optimize the parameters of this network, such as filter size and number of channels, and investigating these design decisions further would be valuable to investigate in future work.

network architecture	test error (cm)
softmax + feature points (ours)	1.30 $\pm$ 0.73
softmax + fully connected layer	2.59 $\pm$ 1.19
fully connected layer	4.75 $\pm$ 2.29
max-pooling + fully connected	3.71 $\pm$ 1.73

Table 3: Average pose estimation accuracy and standard deviation with various architectures, measured as average Euclidean error for the three target points in 3D, with ground truth determined by forward kinematics from the left arm.

		test perturbation							
		lego block				ring on peg			
		0 cm	1 cm	2 cm	3 cm	0 cm	1 cm	2 cm	3 cm
g n n a r r i s	0 cm	5/5	5/5	3/5	2/5	5/5	5/5	0/5	0/5
	1 cm	5/5	5/5	3/5	2/5	5/5	5/5	3/5	0/5
	2 cm	5/5	5/5	5/5	3/5	5/5	5/5	3/5	0/5
	kinematic baseline	5/5	0/5	0/5	0/5	5/5	3/5	0/5	0/5

表 2: 目标物体扰动下线性高斯控制器的成功率。

6.3 Spatial Softmax CNN Architecture Evaluation

在本节中，我们将与更标准的卷积网络进行比较，评估我们在 5.1 节中提出的神经网络架构。为了将架构与其他混杂因素隔离开来，我们在第 5.2 节中描述的姿势估计预训练任务中测量了它们的准确性。这是评估网络如何克服视觉运动学习中两个主要挑战的合理指标：处理相对较小的数据集而不过度拟合的能力，以及学习固有空间任务的能力。我们比较了一个网络，其中softmax之后的期望算子被替换为学习的全连接层（如文献中的标准），一个网络中softmax和期望算子都被替换为一个全连接层，以及该网络的一个版本，该网络在前两层也使用步幅为2的 3×3 max pooling。这些替代架构具有更多参数，因为全连接层将来自第三卷积层的整个响应图库作为输入。池化有助于减少参数数量，但程度不如我们架构中的空间 softmax 和期望算子。

表 3 中的结果表明，使用 softmax 和期望算子可以显著提高位姿估计的准确性。我们的网络能够超越更标准的架构，因为它是由 softmax 和期望算子强制学习特征点，从而提供适合空间推理的简洁表示。由于该架构中的大多数参数都位于卷积层中，这得益于广泛的权重共享，因此过度拟合也大大减少了。通过删除池化，我们的网络还在卷积层中保持了更高的分辨率，从而提高了空间精度。尽管我们确实尝试通过更高的权重衰减和丢失来规范更大的标准架构，但我们没有观察到该数据集有显著的改进。我们也没有广泛优化该网络的参数，例如滤波器大小和通道数量，进一步研究这些设计决策对于未来的工作进行研究是有价值的。

network architecture	test error (cm)
softmax + feature points (ours)	1.30 $\pm$ 0.73
softmax + fully connected layer	2.59 $\pm$ 1.19
fully connected layer	4.75 $\pm$ 2.29
max-pooling + fully connected	3.71 $\pm$ 1.73

表 3: 各种架构的平均姿态估计精度和标准偏差，以 3D 中三个目标点的平均欧几里得误差来测量，并由左臂的正向运动学确定地面实况。

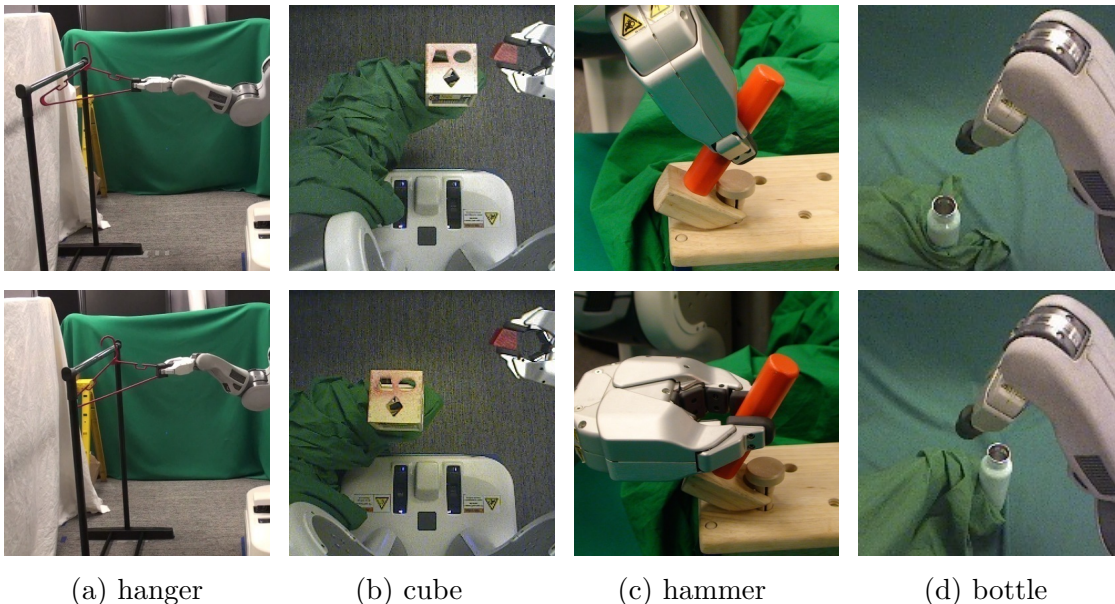


Figure 8: Illustration of the tasks in our visuomotor policy experiments, showing the variation in the position of the target for the hanger, cube, and bottle tasks, as well as two of the three grasps for the hammer, which also included variation in position (not shown).

6.4 Deep Visuomotor Policy Evaluation

In this section, we present an evaluation of our full visuomotor policy training algorithm on a PR2 robot. The aim of this evaluation is to answer the following question: does training the perception and control systems in a visuomotor policy jointly end-to-end provide better performance than training each component separately?

Experimental tasks. We trained policies for hanging a coat hanger on a clothes rack, inserting a block into a shape sorting cube, fitting the claw of a toy hammer under a nail with various grasps, and screwing on a bottle cap. The cost function for these tasks encourages low distance between three points on the end-effector and corresponding target points, low torques, and, for the bottle task, spinning the wrist. The equations for these cost functions and the details of each task are presented in Appendix B.2. The tasks are illustrated in Figure 8. Each task involved variation of 10-20 cm in each direction in the position of the target object (the rack, shape sorting cube, nail, and bottle). In addition, the coat hanger and hammer tasks were trained with two and three grasps, respectively. The current angle of the grasp was not provided to the policy, but had to be inferred from observing the robot’s gripper in the camera images. All tasks used the same policy architecture and model parameters.

Experimental conditions. We evaluated the visuomotor policies in three conditions: (1) the training target positions and grasps, (2) new target positions not seen during training and, for the hammer, new grasps (spatial test), and (3) training positions with visual distractors (visual test). A selection of these experiments is shown in the supplementary video.³ For the visual test, the shape sorting cube was placed on a table rather than held in

3. The video can be viewed at <http://sites.google.com/site/visuomotorpolicy>

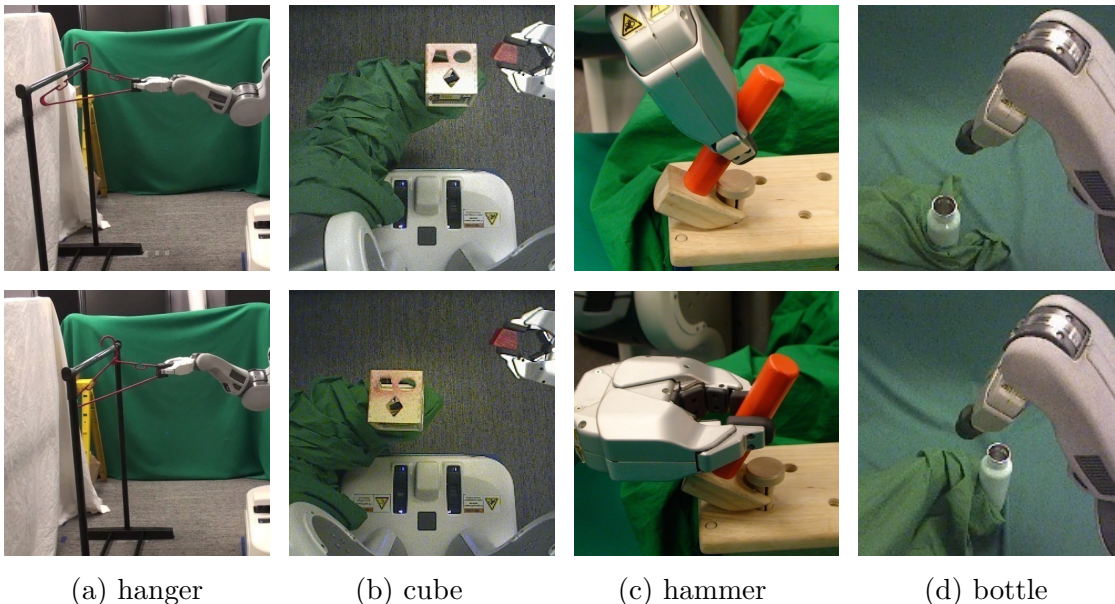


图 8: 我们的视觉运动政策实验中的任务插图, 显示了衣架、立方体和瓶子任务的目标位置的变化, 以及锤子的三个抓握中的两个, 其中也包括位置的变化 (未显示)。

6.4 Deep Visuomotor Policy Evaluation

在本节中, 我们将在 PR2 机器人上评估我们的完整视觉运动策略训练算法。本评估的目的是回答以下问题: 端到端联合训练视觉运动策略中的感知和控制系统是否比单独训练每个组件提供更好的性能?

Experimental tasks. 我们训练了将衣架挂在衣架上、将块插入形状分类立方体、用各种抓手将玩具锤的爪子安装在钉子下以及拧上瓶盖等策略。这些任务的成本函数鼓励末端执行器上的三个点与相应目标点之间的距离较短、扭矩较小, 并且对于瓶子任务来说, 旋转手腕。这些成本函数的方程和每项任务的详细信息在附录 B.2 中给出。这些任务如图 8 所示。每个任务涉及目标物体 (架子、形状分类立方体、钉子和瓶子) 的位置在每个方向上 10-20 厘米的变化。此外, 衣架和锤子任务分别通过两次和三次抓握进行训练。当前的抓握角度并未提供给策略, 但必须通过观察相机图像中机器人的抓手来推断。所有任务都使用相同的策略架构和模型参数。

Experimental conditions. 我们在三种条件下评估了视觉运动策略: (1) 训练目标位置和抓握, (2) 训练期间未看到的新目标位置, 以及对于锤子, 新的抓握 (空间测试), 以及 (3) 带有视觉干扰物的训练位置 (视觉测试)。补充视频中展示了这些实验的一部分。³对于视觉测试, 形状排序立方体被放置在桌子上, 而不是放在里面

3. The video can be viewed at <http://sites.google.com/site/visuomotorpolicy>

the gripper, the coat hanger was placed on a rack with clothes, and the bottle and hammer tasks were done in the presence of clutter. Illustrations of this test are shown in Figure 9.

Comparison. The success rates for each test are shown in Figure 9. We compared to two baselines, both of which train the vision layers in advance for pose prediction, instead of training the entire policy end-to-end. The features baseline discards the last layer of the pose predictor and uses the feature points, resulting in the same architecture as our policy, while the prediction baseline feeds the predicted pose into the control layers. The pose prediction baseline is analogous to a standard modular approach to policy learning, where the vision system is first trained to localize the target, and the policy is trained on top of it. This variant achieves poor performance. As discussed in Section 6.3, the pose estimate is accurate to about 1 cm. However, unlike the tasks in Section 6.2, where robust controllers could succeed even with inaccurate perception, many of these tasks have tolerances of just a few millimeters. In fact, the pose prediction baseline is only successful on the coat hanger, which requires comparatively little accuracy. Millimeter accuracy is difficult to achieve even with calibrated cameras and checkerboards. Indeed, prior work has reported that the PR2 can maintain a camera to end effector accuracy of about 2 cm during open loop motion (Meeussen et al., 2010). This suggests that the failure of this baseline is not atypical, and that our visuomotor policies are learning visual features and control strategies that improve the robot’s accuracy. When provided with pose estimation features, the policy has more freedom in how it uses the visual information, and achieves somewhat higher success rates. However, full end-to-end training performs significantly better, achieving high accuracy even on the challenging bottle task, and successfully adapting to the variety of grasps on the hammer task. This suggests that, although the vision layer pretraining is clearly beneficial for reducing computation time, it is not sufficient by itself for discovering good features for visuomotor policies.

Visual distractors. The policies exhibit moderate tolerance to distractors that are visually separated from the target object. This is enabled in part by the spatial softmax, which has a lateral inhibition effect that suppresses non-maximal activations. Since distractors are unlikely to activate each feature as much as the true object, their activations are therefore suppressed. However, as expected, the learned policies tend to perform poorly under drastic changes to the backdrop, or when the distractors are adjacent to or occluding the manipulated objects, as shown in the supplementary video. A standard solution to this issue is to expose the policy to a greater variety of visual situations during training. This issue could also be mitigated by artificially augmenting the image samples with synthetic transformations, as discussed in prior work in computer vision (Simard et al., 2003), or even incorporating ideas from transfer and semi-supervised learning.

6.5 Features Learned with End-to-End Training

The visual processing layers of our architecture automatically learn feature points using the spatial softmax and expectation operators. These feature points encapsulate all of the visual information received by the motor layers of the policy. In Figure 10, we show the feature points discovered by our visuomotor policy through guided policy search. Each policy learns features on the target object and the robot manipulator, both clearly relevant

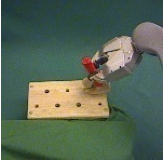
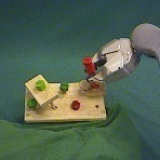
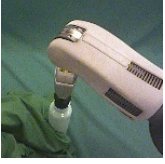
抓手、衣架被放在放有衣服的架子上，瓶子和锤子的任务是在杂乱的情况下完成的。该测试的图示如图 9 所示。

Comparison. 每个测试的成功率如图 9 所示。我们与两个基线进行了比较，这两个基线都提前训练视觉层以进行姿势预测，而不是端到端地训练整个策略。特征基线丢弃姿势预测器的最后一层并使用特征点，从而产生与我们的策略相同的架构，而预测基线将预测的姿势输入控制层。姿态预测基线类似于策略学习的标准模块化方法，其中首先训练视觉系统来定位目标，然后在其之上训练策略。该变体的性能较差。正如第 6.3 节中所讨论的，姿态估计精确到约 1 cm。然而，与第 6.2 节中的任务不同，在第 6.2 节中，即使感知不准确，稳健的控制器也能成功，其中许多任务的容差仅为几毫米。事实上，姿态预测基线仅在衣架上成功，这需要相对较低的精度。即使使用校准的相机和棋盘也很难达到毫米级精度。事实上，之前的工作已报道 PR2 在开环运动期间可以将相机到末端执行器的精度保持在约 2 厘米（Meeussen 等人，2010）。这表明该基线的失败并非不典型，并且我们的视觉运动策略正在学习视觉特征和控制策略，以提高机器人的准确性。当提供姿态估计功能时，该策略在如何使用视觉信息方面具有更大的自由度，并实现更高的成功率。然而，完整的端到端训练表现明显更好，即使在具有挑战性的瓶子任务中也能实现高精度，并成功适应锤子任务的各种抓握方式。这表明，尽管视觉层预训练显然有利于减少计算时间，但它本身不足以发现视觉运动策略的良好特征。

Visual distractors. 这些策略对与目标对象视觉上分离的干扰物表现出适度的容忍度。这在一定程度上是由空间 softmax 实现的，它具有抑制非最大激活的横向抑制效应。由于干扰物不太可能像真实对象那样激活每个特征，因此它们的激活被抑制。然而，正如预期的那样，在背景发生剧烈变化的情况下，或者当干扰物邻近或遮挡被操纵对象时，所学习的策略往往表现不佳，如补充视频所示。此问题的标准解决方案是在训练期间将策略暴露于更多种视觉情况。这个问题也可以通过利用合成变换来人为地增强图像样本来缓解，正如计算机视觉的先前工作中所讨论的那样（Simard 等人，2003），或者甚至结合来自迁移和半监督学习的想法。

6.5 Features Learned with End-to-End Training

我们架构的视觉处理层使用空间 softmax 和期望算子自动学习特征点。这些特征点封装了策略的运动层接收到的所有视觉信息。在图 10 中，我们展示了我们的视觉运动策略通过引导策略搜索发现的特征点。每个策略都会学习目标对象和机器人操纵器的特征，两者都明显相关

	training	visual test				
hanger			coat hanger	training (18)	spatial test (24)	visual test (18)
			end-to-end	100%	100%	100%
			pose features	88.9%	87.5%	83.3%
			pose prediction	55.6%	58.3%	66.7%
cube			shape cube	training (27)	spatial test (36)	visual test (40)
			end-to-end	96.3%	91.7%	87.5%
			pose features	70.4%	83.3%	40%
			pose prediction	0%	0%	n/a
hammer			toy hammer	training (45)	spatial test (60)	visual test (60)
			end-to-end	91.1%	86.7%	78.3%
			pose features	62.2%	75.0%	53.3%
			pose prediction	8.9%	18.3%	n/a
bottle			bottle cap	training (27)	spatial test (12)	visual test (40)
			end-to-end	88.9%	83.3%	62.5%
			pose features	55.6%	58.3%	27.5%

Success rates on training positions, on novel test positions, and in the presence of visual distractors. The number of trials per test is shown in parentheses.

Figure 9: Training and visual test scenes as seen by the policy (left), and experimental results (right). The hammer and bottle images were cropped for visualization only.

to task execution. The policy tends to pick out robust, distinctive features on the objects, such as the left pole of the clothes rack, the left corners of the shape-sorting cube and the bottom-left corner of the toy tool bench. In the bottle task, the end-to-end trained policy outputs points on both sides of the bottle, including one on the cap, while the pose prediction network only finds points on the right edge of the bottle.

In Figure 11, we compare the feature points learned through guided policy search to those learned by a CNN trained for pose prediction. After end-to-end training, the policy acquired a distinctly different set of feature points compared to the pose prediction CNN used for initialization. The end-to-end trained model finds more feature points on task-relevant objects and fewer points on background objects. This suggests that the policy improves its performance by acquiring *goal-driven* visual features that differ from those learned for object localization.

The feature point representation is very simple, since it assumes that the learned features are present at all times, and only one instance of each feature is ever present in the image. While this is a drastic simplification, both the pose predictor and the policy still achieve good results. A more flexible architecture that still learns a concise feature point representation could further improve policy performance. We hope to explore this in future work.

6.6 Computational Performance and Sample Efficiency

We used the Caffe deep learning library (Jia et al., 2014) for CNN training. Each visuomotor policy required a total of 3-4 hours of training time: 20-30 minutes for the pose prediction data collection on the robot, 40-60 minutes for the fully observed trajectory pretraining on

	training	visual test				
r e g n a h			coat hanger	training (18)	spatial test (24)	visual test (18)
			end-to-end	100%	100%	100%
			pose features	88.9%	87.5%	83.3%
			pose prediction	55.6%	58.3%	66.7%
e b u c			shape cube	training (27)	spatial test (36)	visual test (40)
			end-to-end	96.3%	91.7%	87.5%
			pose features	70.4%	83.3%	40%
			pose prediction	0%	0%	n/a
r e m a h			toy hammer	training (45)	spatial test (60)	visual test (60)
			end-to-end	91.1%	86.7%	78.3%
			pose features	62.2%	75.0%	53.3%
			pose prediction	8.9%	18.3%	n/a
c i t t o b			bottle cap	training (27)	spatial test (12)	visual test (40)
			end-to-end	88.9%	83.3%	62.5%
			pose features	55.6%	58.3%	27.5%

Success rates on training positions, on novel test positions, and in the presence of visual distractors. The number of trials per test is shown in parentheses.

图 9: 策略所见的训练和视觉测试场景 (左) 和实验结果 (右)。锤子和瓶子图像仅出于可视化目的而被裁剪。

到任务执行。该策略倾向于挑选物体上稳健、独特的特征，例如衣架的左杆、形状分类立方体的左角和玩具工具台的左下角。在瓶子任务中，端到端训练的策略输出瓶子两侧的点，包括瓶盖上的点，而姿势预测网络仅找到瓶子右边缘的点。

在图 11 中，我们将通过引导策略搜索学习到的特征点与经过姿势预测训练的 CNN 学习到的特征点进行了比较。在端到端训练之后，与用于初始化的姿态预测 CNN 相比，该策略获得了一组明显不同的特征点。端到端训练的模型在任务相关对象上找到更多特征点，在背景对象上找到更少特征点。这表明该策略通过获取与目标定位学习的视觉特征不同的 *goal-driven* 视觉特征来提高其性能。

特征点表示非常简单，因为它假设学习到的特征始终存在，并且每个特征只存在一个实例在图像中。虽然这是一个极大的简化，但姿态预测器和策略仍然取得了良好的结果。仍然学习简洁特征点表示的更灵活的架构可以进一步提高策略性能。我们希望在未来的工作中对此进行探索。

6.6 Computational Performance and Sample Efficiency

我们使用 Caffe 深度学习库 (Jia et al., 2014) 进行 CNN 训练。每个视觉运动策略总共需要 3-4 小时的训练时间：20-30 分钟用于机器人上的姿势预测数据收集，40-60 分钟用于在机器人上进行完全观察的轨迹预训练

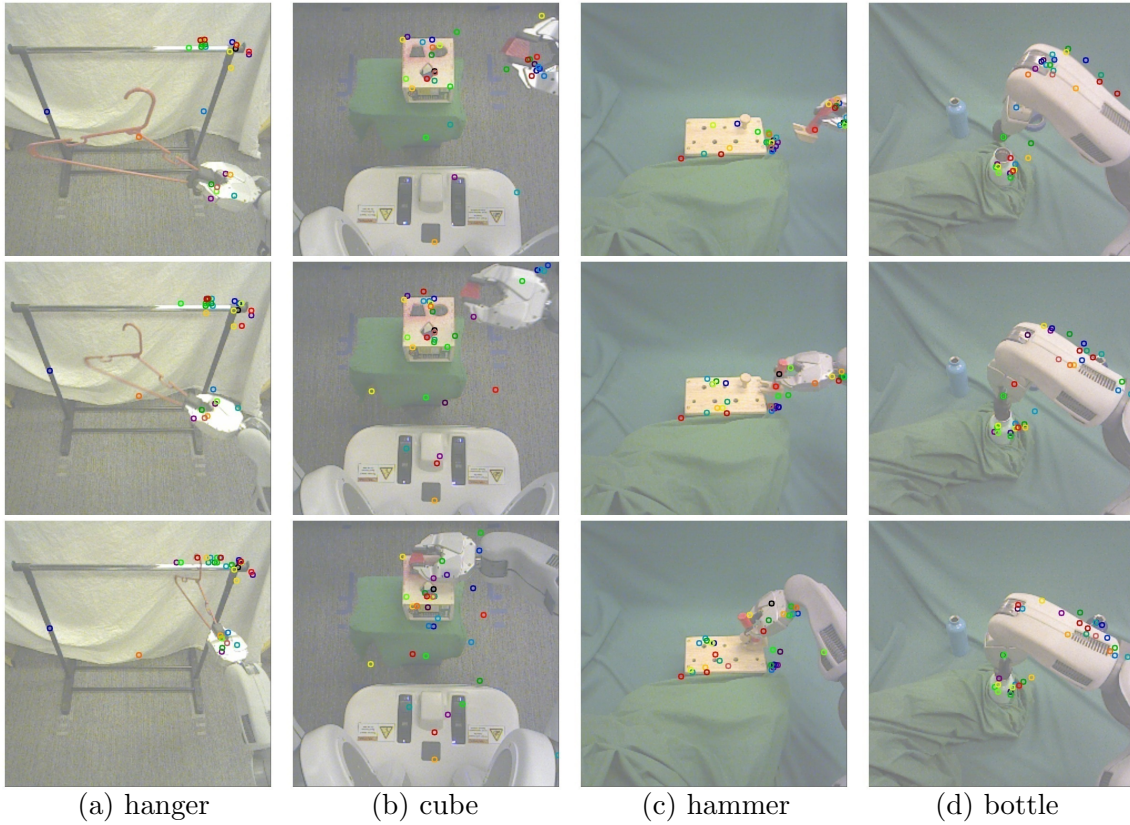


Figure 10: Feature points tracked by the policy during task execution for each of the four tasks. Each feature point is displayed in a different random color, with consistent coloring across images. The policy finds features on the target object and the robot gripper and arm. In the bottle cap task, note that the policy correctly ignores the distractor bottle in the background, even though it was not present during training.

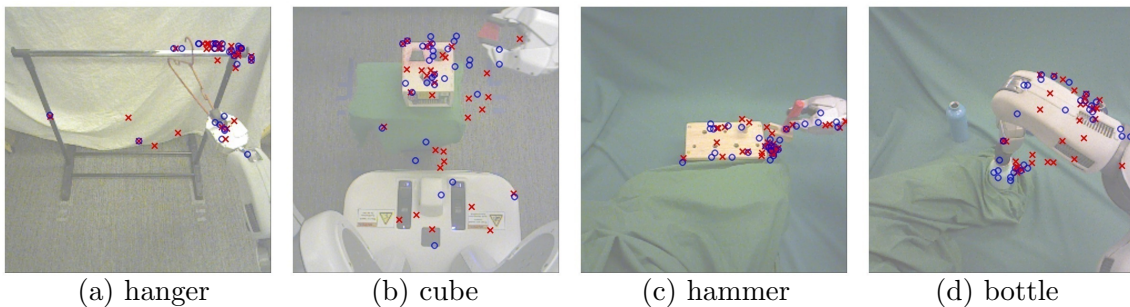


Figure 11: Feature points learned for each task. For each input image, the feature points produced by the policy are shown in blue, while the feature points of the pose prediction network are shown in red. The end-to-end trained policy tends to discover more feature points on the target object and the robot arm than the pose prediction network.

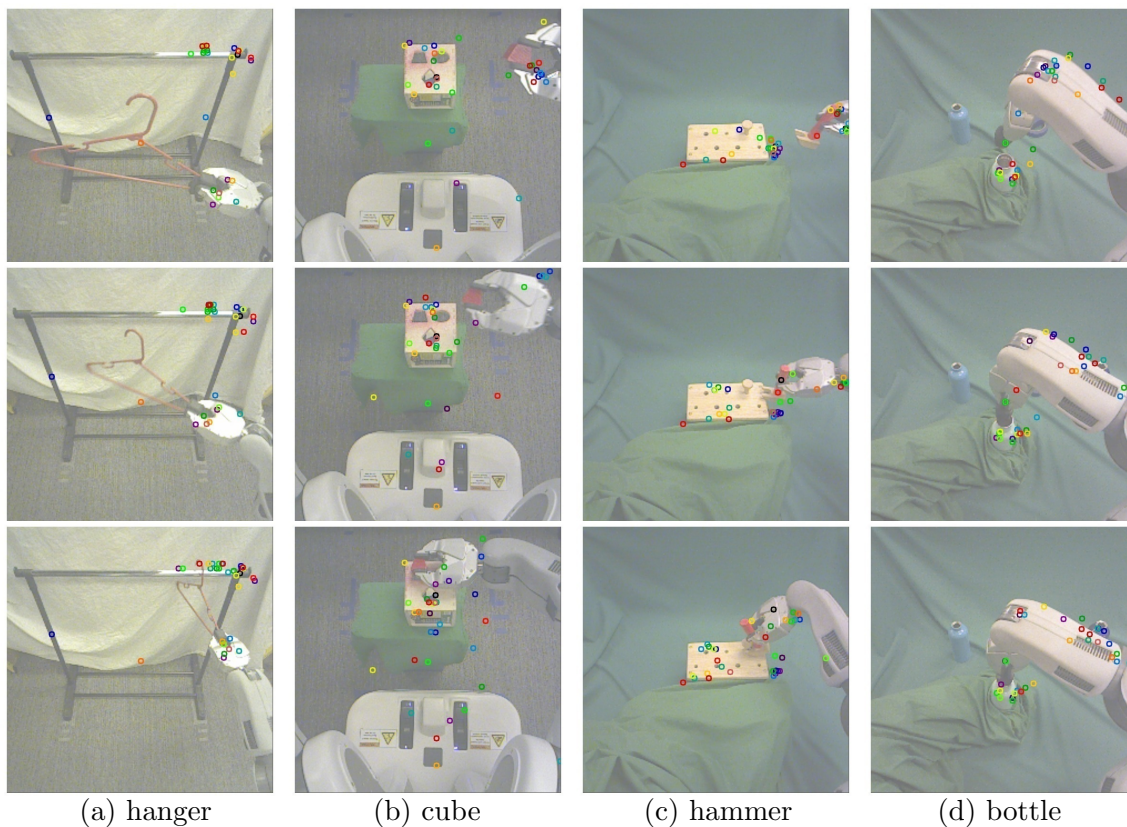


图 10: 四个任务中每一个任务执行期间策略跟踪的特征点。每个特征点以不同的随机颜色显示, 图像之间的颜色保持一致。该策略发现目标物体以及机器人抓手和手臂上的特征。在瓶盖任务中, 请注意, 该策略正确地忽略了背景中的干扰瓶, 即使它在训练期间不存在。

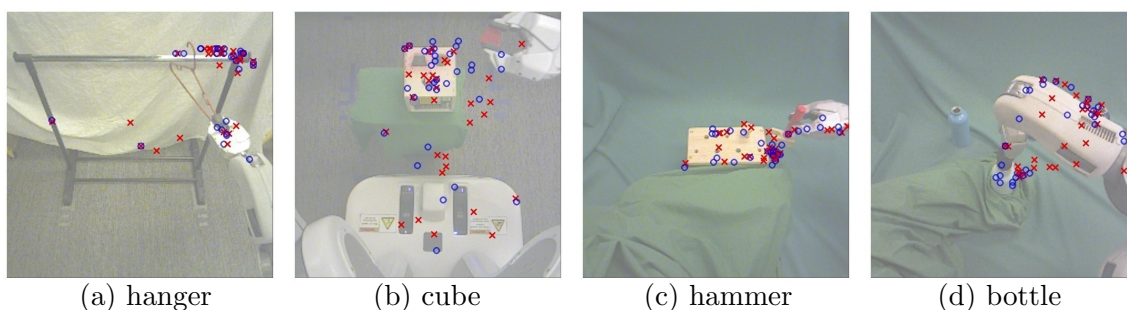


图 11: 为每个任务学习的特征点。对于每个输入图像, 策略产生的特征点显示为蓝色, 而姿态预测网络的特征点显示为红色。与位姿预测网络相比, 端到端训练的策略往往会发现目标对象和机器人手臂上更多的特征点。

the robot and offline pose pretraining (which can be done in parallel), and between 1.5 and 2.5 hours for end-to-end training with guided policy search. The coat hanger task required two iterations of guided policy search, the shape sorting cube and the hammer required three, and the bottle task required four. Only about 15 minutes of the training time consisted of executing trials on the robot. Since training was dominated by computation, we expect significant speedup from a more efficient implementation. The number of samples for training each policy is shown in Table 4. Each trial was five seconds in length, and the numbers do not include the time needed to collect about 1000 images for pretraining the visual processing layers of the policy.

task	number of trials		
	trajectory pretraining	end-to-end training	total
coat hanger	120	36	156
shape cube	90	81	171
toy hammer	150	90	240
bottle cap	180	108	288

Table 4: Total number of trials used for learning each visuomotor policy.

7. Discussion and Future Work

In this paper, we presented a method for learning robotic control policies that use raw input from a monocular camera. These policies are represented by a novel convolutional neural network architecture, and can be trained end-to-end using our guided policy search algorithm, which decomposes the policy search problem in a trajectory optimization phase that uses full state information and a supervised learning phase that only uses the observations. This decomposition allows us to leverage state-of-the-art tools from supervised learning, making it straightforward to optimize extremely high-dimensional policies. Our experimental results show that our method can execute complex manipulation skills, and that end-to-end training produces significant improvements in policy performance compared to using fixed vision layers trained for pose prediction.

Although we demonstrate moderate generalization over variations in the scene, our current method does not generalize to dramatically different settings, especially when visual distractors occlude the manipulated object or break up its silhouette in ways that differ from the training. The success of CNNs on exceedingly challenging vision tasks suggests that this class of models is capable of learning invariance to irrelevant distractor features (LeCun et al., 2015), and in principle this issue can be addressed by training the policy in a variety of environments, though this poses certain logistical challenges. More practical alternatives that could be explored in future work include simultaneously training the policy on multiple robots, each of which is located in a different environment, developing more sophisticated regularization and pretraining techniques to avoid overfitting, and introducing artificial data augmentation to encourage the policy to be invariant to irrelevant clutter. However, even without these improvements, our method has numerous applications in, for example, an industrial setting where the robot must repeatedly and efficiently perform a task that requires visual feedback under moderate variation in background and clutter conditions.

Our method takes advantage of a known, fully observed state space during training. This is both a weakness and a strength. It allows us to train linear-Gaussian controllers

机器人和离线姿势预训练（可以并行完成），以及 1.5 到 2.5 小时的带引导策略搜索的端到端训练。衣架任务需要两次迭代引导策略搜索，形状排序立方体和锤子需要三次，瓶子任务需要四次。只有大约 15 分钟的训练时间用于在机器人上进行试验。由于训练以计算为主，我们期望通过更有效的实施来显著加速。用于训练每个策略的样本数量如表 4 所示。每次试验的长度为 5 秒，并且这些数字不包括收集约 1000 张图像以预训练策略的视觉处理层所需的时间。

task	number of trials		
	trajectory pretraining	end-to-end training	total
coat hanger	120	36	156
shape cube	90	81	171
toy hammer	150	90	240
bottle cap	180	108	288

表 4: 用于学习每个视觉运动策略的试验总数。

7. Discussion and Future Work

在本文中，我们提出了一种使用单目相机的原始输入来学习机器人控制策略的方法。这些策略由新颖的卷积神经网络架构表示，并且可以使用我们的引导策略搜索算法进行端到端训练，该算法在使用完整状态信息的轨迹优化阶段和仅使用观测值的监督学习阶段分解策略搜索问题。这种分解使我们能够利用监督学习中最先进的工具，从而可以直接优化极高维的策略。我们的实验结果表明，我们的方法可以执行复杂的操作技能，并且与使用经过姿势预测训练的固定视觉层相比，端到端训练可以显著提高策略性能。

尽管我们展示了对场景变化的适度泛化，但我们当前的方法并不能泛化到显著不同的设置，特别是当视觉干扰物遮挡被操纵对象或以与训练不同的方式破坏其轮廓时。CNN 在极具挑战性的视觉任务上的成功表明，此类模型能够学习不相关干扰特征的不变性（LeCun 等人，2015），原则上可以通过在各种环境中训练策略来解决这个问题，尽管这会带来一定的逻辑挑战。未来工作中可以探索的更实用的替代方案包括同时在多个机器人上训练策略，每个机器人都位于不同的环境中，开发更复杂的正则化和预训练技术以避免过度拟合，以及引入人工数据增强以鼓励策略对不相关的混乱保持不变。然而，即使没有这些改进，我们的方法也有许多应用，例如，在工业环境中，机器人必须在背景和杂波条件适度变化的情况下重复有效地执行需要视觉反馈的任务。

我们的方法在训练期间利用已知的、完全观察到的状态空间。这既是一个弱点，也是一个优点。它允许我们训练线性高斯控制器

for guided policy search using a very small number of samples, far more efficiently than standard policy search methods. However, the requirement to observe the full state during training limits the tasks to which the method can be applied. In many cases, this limitation is minor, and the only “instrumentation” required at training is to position the objects in the scene at consistent positions. However, tasks that require, for example, manipulating freely moving objects require more extensive instrumentation, such as motion capture. A promising direction for addressing this limitation is to combine our method with unsupervised state-space learning, as proposed in several recent works, including our own (Lange et al., 2012; Watter et al., 2015; Finn et al., 2015).

In future work, we hope to explore more complex policy architectures, such as recurrent policies that can deal with extensive occlusions by keeping a memory of past observations. We also hope to extend our method to a wider range of tasks that can benefit from visual input, as well as a variety of other rich sensory modalities, including haptic input from pressure sensors and auditory input. With a wider range of sensory modalities, end-to-end training of sensorimotor policies will become increasingly important: while it is often straightforward to imagine how vision might help to localize the position of an object in the scene, it is much less apparent how sound can be integrated into robotic control. A learned sensorimotor policy would be able to naturally integrate a wide range of modalities and utilize them to directly aid in control.

Acknowledgements

This research was funded in part by DARPA through a Young Faculty Award, the Army Research Office through the MAST program, NSF awards IIS-1427425 and IIS-1212798, the Berkeley Vision and Learning Center, and a Berkeley EECS Department Fellowship.

Appendix A. Guided Policy Search Algorithm Details

In this appendix, we describe a number of implementation details of our BADMM-based guided policy search algorithm and our linear-Gaussian controller optimization method.

A.1 BADMM Dual Variables and Weight Adjustment

Recall that the inner loop alternating optimization is given by

$$\begin{aligned}\theta &\leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)}[\mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^{\theta}(\theta, p) \\ p &\leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^p(p, \theta) \\ \lambda_{\mu t} &\leftarrow \lambda_{\mu t} + \alpha \nu_t (E_{\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t] - E_{p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)}[\mathbf{u}_t]).\end{aligned}$$

We use a step size of $\alpha = 0.1$ in all of our experiments, which we found to be more stable than $\alpha = 1.0$. The weights ν_t are initialized to 0.01 and incremented based on the following schedule: at every iteration, we compute the average KL-divergence between $p(\mathbf{u}_t|\mathbf{x}_t)$ and $\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)$ at each time step, as well as its standard deviation over time steps.

使用极少数样本进行指导性政策搜索，比标准政策搜索方法高效得多。然而，在训练期间观察完整状态的要求限制了该方法可以应用的任务。在许多情况下，这种限制很小，训练时所需的唯一“仪器”是将场景中的对象放置在一致的位置。然而，例如需要操纵自由移动物体的任务需要更广泛的仪器，例如动作捕捉。解决这一限制的一个有希望的方向是将我们的方法与无监督状态空间学习相结合，正如最近的几项工作中所提出的，包括我们自己的（Lange et al., 2012; Watter et al., 2015; Finn et al., 2015）。

在未来的工作中，我们希望探索更复杂的策略架构，例如可以通过保留过去观察的记忆来处理广泛遮挡的循环策略。我们还希望将我们的方法扩展到更广泛的任务，这些任务可以从视觉输入以及各种其他丰富的感官方式中受益，包括来自压力传感器的触觉输入和听觉输入。随着感觉模式的范围越来越广，感觉运动策略的端到端训练将变得越来越重要：虽然通常很容易想象视觉如何帮助定位场景中物体的位置，但如何将声音集成到机器人控制中却不太明显。学习的感觉运动策略将能够自然地整合多种模式并利用它们直接帮助控制。

Acknowledgements

这项研究的部分资金由 DARPA 通过青年教师奖、陆军研究办公室通过 MAST 计划、NSF 奖项 IIS-1427425 和 IIS-1212798、伯克利视觉与学习中心以及伯克利 EECS 系奖学金提供。

Appendix A. Guided Policy Search Algorithm Details

在本附录中，我们描述了基于 BADMM 的引导策略搜索算法和线性高斯控制器优化方法的许多实现细节。

A.1 BADMM Dual Variables and Weight Adjustment

回想一下，内循环交替优化由下式给出

$$\begin{aligned}\theta &\leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t) \pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)} [\mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^{\theta}(\theta, p) \\ p &\leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)} [\ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^T \lambda_{\mu t}] + \nu_t \phi_t^p(p, \theta) \\ \lambda_{\mu t} &\leftarrow \lambda_{\mu t} + \alpha \nu_t (E_{\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t) p(\mathbf{x}_t)} [\mathbf{u}_t] - E_{p(\mathbf{u}_t|\mathbf{x}_t) p(\mathbf{x}_t)} [\mathbf{u}_t]).\end{aligned}$$

我们在所有实验中使用的步长为 $\alpha = 0.1$ ，我们发现它比 $\alpha = 1.0$ 更稳定。权重 ν_t 初始化为 0.01 并根据以下时间表递增：在每次迭代中，我们计算每个时间步上 $p(\mathbf{u}_t|\mathbf{x}_t)$ 和 $\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)$ 之间的平均 KL 散度，以及其随时间步长的标准偏差。

The weights ν_t corresponding to time steps where the KL-divergence is higher than the average are increased by a factor of 2, and the weights corresponding to time steps where the KL-divergence is two standard deviations or more below the average are decreased by a factor of 2. The rationale behind this schedule is to adjust the KL-divergence penalty to keep the policy and trajectory in agreement by roughly the same amount at all time steps. Increasing ν_t too quickly can lead to the policy and trajectory becoming “locked” together, which makes it difficult for the trajectory to decrease its cost, while leaving it too low requires more iterations for convergence. We found this schedule to work well across all tasks, both during trajectory pretraining and while training the visuomotor policy.

To update the dual variables $\lambda_{\mu t}$, we evaluate the expectations over $p(\mathbf{x}_t)$ by using the latest batch of sampled trajectories. For each state $\{\mathbf{x}_t^i\}$ along these sampled trajectories, we evaluate the expectations over $\mathbf{u}_t$ under $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ and $p(\mathbf{u}_t|\mathbf{x}_t)$, which correspond simply to the means of these conditional Gaussian distributions, in closed form.

A.2 Policy Variance Optimization

As discussed in Section 4, the variance of the Gaussian policy $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ does not depend on the observation, though this dependence would be straightforward to add. Analyzing the objective $\mathcal{L}_\theta(\theta, p)$, we can write out only the terms that depend on Σ^π :

$$\mathcal{L}_\theta(\theta, p) = \frac{1}{2N} \sum_{i=1}^N \sum_{t=1}^T E_{p_i(\mathbf{x}_t, \mathbf{o}_t)} [\text{tr}[\mathbf{C}_{ti}^{-1} \Sigma^\pi] - \log |\Sigma^\pi|].$$

Differentiating and setting the derivative to zero, we obtain the following equation for Σ^π :

$$\Sigma^\pi = \left[\frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T \mathbf{C}_{ti}^{-1} \right]^{-1},$$

where the expectation under $p_i(\mathbf{x}_t)$ is omitted, since $\mathbf{C}_{ti}$ does not depend on $\mathbf{x}_t$.

A.3 Dynamics Fitting

Optimizing the linear-Gaussian controllers $p_i(\mathbf{u}_t|\mathbf{x}_t)$ that induce the trajectory distributions $p_i(\tau)$ requires fitting the system dynamics $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ at each iteration to samples generated on the physical system from the previous controller $\hat{p}_i(\mathbf{u}_t|\mathbf{x}_t)$. In this section, we describe how these dynamics are fitted. As in Section 4, we drop the subscript i , since the dynamics are fitted the same way for all of the trajectory distributions.

The linear-Gaussian dynamics are defined as $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t}\mathbf{x}_t + f_{\mathbf{u}t}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$, and the data that we obtain from the robot can be viewed as tuples $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$. A simple way to fit these linear-Gaussian dynamics is to use linear regression to determine $f_{\mathbf{x}}$, $f_{\mathbf{u}}$, and f_c , and fit $\mathbf{F}_t$ based on the errors. However, the sample complexity of linear regression scales with the dimensionality of $\mathbf{x}_t$. For a high-dimensional robotic system, we might need an impractically large number of samples at each iteration to obtain a good fit. However, we can observe that the dynamics at nearby time steps are strongly correlated, and we can dramatically reduce the sample complexity of the dynamics fitting by bringing in information from other time steps, and even prior iterations. We will bring in this

与 KL 散度高于平均值的时间步对应的权重 ν_t 增加了 2 倍，与 KL 散度低于平均值两个标准差或更多的时间步对应的权重减少了 2 倍。这个时间表背后的基本原理是调整 KL 散度惩罚，以在所有时间步上保持策略和轨迹大致相同的量。太快地增加 ν_t 可能会导致策略和轨迹“锁定”在一起，这使得轨迹难以降低其成本，而使其太低则需要更多的迭代来收敛。我们发现这个时间表在所有任务中都表现良好，无论是在轨迹预训练期间还是在训练视觉运动策略期间。

为了更新双变量 $\lambda_{\mu t}$ ，我们使用最新一批采样轨迹来评估对 $p(\mathbf{x}_t)$ 的期望。对于沿这些采样轨迹的每个状态 $\{\mathbf{x}_t^i\}$ ，我们评估 $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$ 和 $p(\mathbf{u}_t|\mathbf{x}_t)$ 下对 $\mathbf{u}_t$ 的期望，这简单地对应于这些条件高斯分布的均值（以封闭形式）。

A.2 Policy Variance Optimization

正如第 4 节中所讨论的，高斯策略 $\pi_\theta(\mathbf{u}_t|\mathbf{o}_t)$ 的方差不依赖于观察，尽管这种依赖关系很容易添加。分析目标 $\mathcal{L}_\theta(\theta, p)$ ，我们可以只写出依赖于 Σ^π 的项：

$$\mathcal{L}_\theta(\theta, p) = \frac{1}{2N} \sum_{i=1}^N \sum_{t=1}^T E_{p_i(\mathbf{x}_t, \mathbf{o}_t)} [\text{tr}[\mathbf{C}_{ti}^{-1} \Sigma^\pi] - \log |\Sigma^\pi|].$$

对 Σ^π 进行微分并将导数设置为零，我们得到以下方程：

$$\Sigma^\pi = \left[\frac{1}{NT} \sum_{i=1}^N \sum_{t=1}^T \mathbf{C}_{ti}^{-1} \right]^{-1},$$

其中省略了 $p_i(\mathbf{x}_t)$ 下的期望，因为 $\mathbf{C}_{ti}$ 不依赖于 $\mathbf{x}_t$ 。

A.3 Dynamics Fitting

优化产生轨迹分布 $p_i(\tau)$ 的线性高斯控制器 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 需要在每次迭代时将系统动力学 $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 拟合到前一个控制器 $\hat{p}_i(\mathbf{u}_t|\mathbf{x}_t)$ 在物理系统上生成的样本。在本节中，我们将描述这些动态是如何拟合的。与第 4 节一样，我们删除了下标 i ，因为所有轨迹分布的动力学拟合方式都相同。

线性高斯动力学定义为 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t}\mathbf{x}_t + f_{\mathbf{u}t}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$ ，我们从机器人获得的数据可以看作元组 $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ 。拟合这些线性高斯动力学的一个简单方法是使用线性回归来确定 $f_{\mathbf{x}}$ 、 $f_{\mathbf{u}}$ 和 f_c ，并根据误差拟合 $\mathbf{F}_t$ 。然而，线性回归的样本复杂度与 $\mathbf{x}_t$ 的维数成比例。对于高维机器人系统，我们可能在每次迭代时需要大量不切实际的样本才能获得良好的拟合。然而，我们可以观察到附近时间步长的动力学是强相关的，并且我们可以通过引入其他时间步长甚至先前迭代的信息来显著降低动力学拟合的样本复杂性。我们将引入这个

information by fitting a global model to all of the transitions $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ for all t and all tuples from several prior iterations (we use three prior iterations in our implementation), and then use this model as a prior for fitting the dynamics at each time step. Note that this global model does not itself need to be a good forward dynamics model – it just needs to serve as a good prior to reduce the sample complexity of linear regression.

To make it more convenient to incorporate a data-driven prior, we will first reformulate this linear regression fit and view it as fitting a Gaussian model to the dataset $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ at each time step t , and then conditioning this Gaussian to obtain $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$. While this is equivalent to linear regression, it allows us to easily incorporate a normal-inverse-Wishart prior on this Gaussian in order to bring in prior information. Let $\hat{\Sigma}$ be the empirical covariance of our dataset, and let $\hat{\mu}$ be the empirical mean. The normal-inverse-Wishart prior is defined by prior parameters Φ , μ_0 , m , and n_0 . Under this prior, the maximum a posteriori estimates for the covariance Σ and mean μ are given by

$$\Sigma = \frac{\Phi + N\hat{\Sigma} + \frac{Nm}{N+m}(\hat{\mu} - \mu_0)(\hat{\mu} - \mu_0)^T}{N + n_0} \quad \mu = \frac{m\mu_0 + n_0\hat{\mu}}{m + n_0}.$$

Having obtained Σ and μ , we can obtain an estimate of the dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ by conditioning the distribution $\mathcal{N}(\mu, \Sigma)$ on $[\mathbf{x}_t; \mathbf{u}_t]$, which produces linear-Gaussian dynamics $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{xt}\mathbf{x}_t + f_{ut}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$. The parameters of the normal-inverse-Wishart prior are obtained from the global model of the dynamics which, as described previously, is fitted to all available tuples $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$.

The simplest prior can be obtained by fitting a Gaussian distribution to vectors $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$. If the mean and covariance of this data are given by $\bar{\mu}$ and $\bar{\Sigma}$, the prior is given by $\Phi = n_0\bar{\Sigma}$ and $\mu_0 = \bar{\mu}$, while n_0 and m should be set to the number of data points in the datasets. In practice, settings n_0 and m to 1 tends to produce better results, since the prior is fitted to many more samples than are available for linear regression at each time step. While this prior is simple, we can obtain a better prior by employing a nonlinear model.

The particular global model we use in this work is a Gaussian mixture model over vectors $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$. Systems of articulated rigid bodies undergoing contact dynamics, such as robots interacting with their environment, can be coarsely modeled as having piecewise linear dynamics. The Gaussian mixture model provides a good approximation for such piecewise linear systems, with each mixture element corresponding to a different linear mode (Khansari-Zadeh and Billard, 2010). Under this model, the state transition tuple is assumed to come from a distribution that depends on some hidden state h , which corresponds to the mixture element identity. In practice, this hidden state might correspond to the type of contact profile experienced by a robotic arm at step i . The prior for the dynamics fit at time step t is then obtained by inferring the hidden state distribution for the transition dataset $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$, and using the mean and covariance of the corresponding mixture elements (weighted by their probabilities) to obtain $\bar{\mu}$ and $\bar{\Sigma}$. The prior parameters can then be obtained as described above.

In our experiments, we set the number of mixture elements for the Gaussian mixture model prior such that there were at least 40 samples per mixture element, or 20 total mixture elements, whichever was lower. In general, we did not find the performance of the method to be sensitive to this parameter, though overfitting did tend to occur in the early iterations when the number of samples is low, if the number of mixtures was too high.

通过将全局模型拟合到所有 t 的所有转换 $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ 以及来自多个先前迭代的所有元组（我们在实现中使用三个先前迭代）来获取信息，然后使用该模型作为先验来拟合每个时间步的动态。请注意，这个全局模型本身并不需要是一个好的前向动力学模型——它只需要作为一个好的先验模型来降低线性回归的样本复杂性。

为了方便地合并数据驱动的先验，我们将首先重新表述此线性回归拟合，并将其视为在每个时间步 t 将高斯模型拟合到数据集 $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ ，然后调节此高斯模型以获得 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 。虽然这相当于线性回归，但它使我们能够轻松地在此高斯上合并正态逆威沙特先验，以便引入先验信息。令 $\hat{\Sigma}$ 为数据集的经验协方差，令 $\hat{\mu}$ 为经验平均值。正态逆 Wishart 先验由先验参数 Φ 、 μ_0 、 m 和 n_0 定义。在此先验条件下，协方差 Σ 和均值 μ 的最大后验估计由下式给出

$$\Sigma = \frac{\Phi + N\hat{\Sigma} + \frac{Nm}{N+m}(\hat{\mu} - \mu_0)(\hat{\mu} - \mu_0)^T}{N + n_0} \quad \mu = \frac{m\mu_0 + n_0\hat{\mu}}{m + n_0}.$$

获得 Σ 和 μ 后，我们可以通过调节 $[\mathbf{x}_t]$ 上的分布 $\mathcal{N}(\mu, \Sigma)$ 来获得动态 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$ 的估计； $\mathbf{u}_t$ ，产生线性高斯动力学 $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t}\mathbf{x}_t + f_{\mathbf{u}t}\mathbf{u}_t + f_{ct}, \mathbf{F}_t)$ 。正态逆 Wishart 先验的参数是从动力学的全局模型获得的，如前所述，该模型适合所有可用的元组 $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ 。

最简单的先验可以通过将高斯分布拟合到向量 $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$ 。如果该数据的均值和协方差由 $\bar{\mu}$ 和 $\bar{\Sigma}$ 给出，则先验由 $\Phi = n_0\bar{\Sigma}$ 和 $\mu_0 = \bar{\mu}$ 给出，而 n_0 和 m 应设置为数据集中数据点的数量。在实践中，将 n_0 和 m 设置为 1 往往会产生更好的结果，因为先验拟合的样本比每个时间步的线性回归可用的样本多得多。虽然这个先验很简单，但我们可以通过采用非线性模型获得更好的先验。

我们在这项工作中使用的特定全局模型是向量 $[\mathbf{x}; \mathbf{u}; \mathbf{x}']$ 上的高斯混合模型。 $\mathbf{u}; \mathbf{x}'$ 。经历接触动力学的铰接刚体系统（例如与其环境交互的机器人）可以粗略地建模为具有分段线性动力学。高斯混合模型为此类分段线性系统提供了良好的近似，每个混合元素对应于不同的线性模式（Khansari-Zadeh 和 Billard, 2010）。在此模型下，假设状态转换元组来自依赖于某个隐藏状态 h 的分布，该隐藏状态对应于混合元素标识。实际上，这种隐藏状态可能对应于机械臂在步骤 i 中经历的接触轮廓类型。然后，通过推断转换数据集 $\{\mathbf{x}_t^i, \mathbf{u}_t^i, \mathbf{x}_{t+1}^i\}$ 的隐藏状态分布，并使用相应混合元素的均值和协方差（按其概率加权）来获得 $\bar{\mu}$ 和 $\bar{\Sigma}$ ，从而获得时间步 t 处动力学拟合的先验。然后可以如上所述获得先验参数。

在我们的实验中，我们事先设置了高斯混合模型的混合元素数量，使得每个混合元素至少有 40 个样本，或总共 20 个混合元素，以较低者为准。一般来说，我们没有发现该方法的性能对该参数敏感，尽管如果混合数量过高，当样本数量较少时，在早期迭代中确实容易发生拟合。

A.4 Trajectory Optimization

In this section, we show how the LQR backward pass can be used to optimize the constrained objective in Section 4.2. The constrained trajectory optimization problem is given by

$$\min_{p(\tau) \in \mathcal{N}(\tau)} \mathcal{L}_p(p, \theta) \text{ s.t. } D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon.$$

The augmented Lagrangian $\mathcal{L}_p(p, \theta)$ consists of an entropy term and an expectation under $p(\tau)$ of a quantity that is independent of p . We can locally approximate this quantity with a quadratic by using a quadratic expansion of $\ell(\mathbf{x}_t, \mathbf{u}_t)$, and fitting a linear Gaussian to $\pi_\theta(\mathbf{u}_t | \mathbf{x}_t)$ with the same method we used for the dynamics. We can then solve the primal optimization in the dual gradient descent procedure with a standard LQR backward pass. As discussed in Section 4, $\mathcal{L}_p(p, \theta)$ can be written as the expectation of some function $c(\tau)$ that is independent of p , such that $\mathcal{L}_p(p, \theta) = E_{p(\tau)}[c(\tau)] - \nu_t \mathcal{H}(p(\tau))$. Specifically,

$$c(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^\top \lambda_{\mu t} - \nu_t \log \pi_\theta(\mathbf{u}_t | \mathbf{x}_t)$$

Writing the Lagrangian of the constrained optimization, we have

$$\mathcal{L}(p) = E_{p(\tau)}[c(\tau) - \eta \log \hat{p}(\tau)] - (\eta + \nu_t) \mathcal{H}(p(\tau)) - \eta \epsilon,$$

where η is the Lagrange multiplier. Note that $\mathcal{L}(p)$ is the Lagrangian of the constrained trajectory optimization, which is not related to the augmented Lagrangian $\mathcal{L}_p(\tau, \theta)$. Grouping the terms in the expectation and omitting constants, we can rewrite the minimization of the Lagrangian with respect to the primal variables as

$$\min_{p(\tau) \in \mathcal{N}(\tau)} E_{p(\tau)} \left[\frac{1}{\eta + \nu_t} c(\tau) - \frac{\eta}{\eta + \nu_t} \log \hat{p}(\tau) \right] - \mathcal{H}(p(\tau)). \quad (4)$$

Let $\tilde{c}(\tau) = \frac{1}{\eta + \nu_t} c(\tau) - \frac{\eta}{\eta + \nu_t} \log \hat{p}(\tau)$. The above optimization corresponds to minimizing $E_{p(\tau)}[\tilde{c}(\tau)] - \mathcal{H}(p(\tau))$. This type of maximum entropy problem can be solved using the LQR algorithm, and the solution is given by

$$p(\mathbf{u}_t | \mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t; Q_{\mathbf{u}, \mathbf{u}_t}^{-1}),$$

where $\mathbf{K}_t$ and $\mathbf{k}_t$ are the feedback and open loop terms of the optimal linear feedback controller corresponding to the cost $\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)$ and the dynamics $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t)$, and $Q_{\mathbf{u}, \mathbf{u}_t}$ is the quadratic term in the Q-function at time step t . All of these terms can be obtained from a standard LQR backward pass (Li and Todorov, 2004), which we summarize below.

Recall that the estimated linear-Gaussian dynamics have the form $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t} \mathbf{x}_t + f_{\mathbf{u}t} \mathbf{u}_t + f_{ct}, \mathbf{F}_t)$. The quadratic cost approximation has the form

$$\tilde{c}(\mathbf{x}_t, \mathbf{u}_t) \approx \frac{1}{2} [\mathbf{x}_t; \mathbf{u}_t]^\top \tilde{c}_{\mathbf{xu}, \mathbf{xu}t} [\mathbf{x}_t; \mathbf{u}_t] + [\mathbf{x}_t; \mathbf{u}_t]^\top \tilde{c}_{\mathbf{xu}t} + \text{const},$$

where subscripts denote derivatives, e.g. $\tilde{c}_{\mathbf{xu}t}$ is the gradient of $\tilde{c}$ with respect to $[\mathbf{x}_t; \mathbf{u}_t]$, while $\tilde{c}_{\mathbf{xu}, \mathbf{xu}t}$ is the Hessian.⁴ Under this model of the dynamics and cost function, the

4. We assume that all Taylor expansions here are recentered around zero. Otherwise, the point around which the derivatives are computed must be subtracted from $\mathbf{x}_t$ and $\mathbf{u}_t$ in all of these equations.

A.4 Trajectory Optimization

在本节中，我们将展示如何使用 LQR 向后传递来优化 4.2 节中的约束目标。约束轨迹优化问题由下式给出

$$\min_{p(\tau) \in \mathcal{N}(\tau)} \mathcal{L}_p(p, \theta) \text{ s.t. } D_{\text{KL}}(p(\tau) \parallel \hat{p}(\tau)) \leq \epsilon.$$

增广拉格朗日 $\mathcal{L}_p(p, \theta)$ 由熵项和独立于 p 的量的 $p(\tau)$ 下的期望组成。我们可以通过使用 $\ell(\mathbf{x}_t, \mathbf{u}_t)$ 的二次展开来局部近似这个量，并使用我们用于动力学的相同方法将线性高斯拟合到 $\pi_\theta(\mathbf{u}_t | \mathbf{x}_t)$ 。然后，我们可以使用标准 LQR 向后传递来解决双梯度下降过程中的原始优化问题。正如第 4 节中所讨论的， $\mathcal{L}_p(p, \theta)$ 可以写为独立于 p 的某个函数 $c(\tau)$ 的期望，例如 $\mathcal{L}_p(p, \theta) = E_{p(\tau)}[c(\tau)] - \nu_t \mathcal{H}(p(\tau))$ 。具体来说，

$$c(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) - \mathbf{u}_t^\top \lambda_{\mu t} - \nu_t \log \pi_\theta(\mathbf{u}_t | \mathbf{x}_t)$$

写出约束优化的拉格朗日量，我们有

$$\mathcal{L}(p) = E_{p(\tau)}[c(\tau) - \eta \log \hat{p}(\tau)] - (\eta + \nu_t) \mathcal{H}(p(\tau)) - \eta \epsilon,$$

其中 η 是拉格朗日乘子。请注意， $\mathcal{L}(p)$ 是约束轨迹优化的拉格朗日量，与增广拉格朗日量 $\mathcal{L}_p(p, \theta)$ 无关。将期望中的项分组并省略常数，我们可以将拉格朗日关于原始变量的最小化重写为

$$\min_{p(\tau) \in \mathcal{N}(\tau)} E_{p(\tau)} \left[\frac{1}{\eta + \nu_t} c(\tau) - \frac{\eta}{\eta + \nu_t} \log \hat{p}(\tau) \right] - \mathcal{H}(p(\tau)). \quad (4)$$

让 $\tilde{c}(\tau) = \frac{1}{\eta + \nu_t} c(\tau) - \frac{\eta}{\eta + \nu_t} \log \hat{p}(\tau)$ 记录 $\hat{p}(\tau)$ 。上述优化对应于最小化 $E_{p(\tau)}[\tilde{c}(\tau)] - \mathcal{H}(p(\tau))$ 。这类最大熵问题可以使用 LQR 算法来求解，解为：

$$p(\mathbf{u}_t | \mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t; Q_{\mathbf{u}, \mathbf{u}t}^{-1}),$$

其中 $\mathbf{K}_t$ 和 $\mathbf{k}_t$ 是对应于成本 $\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)$ 和动态 $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t)$ 的最优线性反馈控制器的反馈和开环项， $Q_{\mathbf{u}, \mathbf{u}t}$ 是时间步 t 处 Q 函数中的二次项。所有这些项都可以从标准 LQR 向后传递中获得 (Li 和 Todorov, 2004)，我们总结如下。

回想一下，估计的线性高斯动力学具有 $p(\mathbf{x}_{t+1} | \mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(f_{\mathbf{x}t} \mathbf{x}_t + f_{\mathbf{u}t} \mathbf{u}_t + f_{ct}, \mathbf{F}_t)$ 的形式。二次成本近似具有以下形式

$$\tilde{c}(\mathbf{x}_t, \mathbf{u}_t) \approx \frac{1}{2} [\mathbf{x}_t; \mathbf{u}_t]^\top \tilde{c}_{\mathbf{xu}, \mathbf{xut}} [\mathbf{x}_t; \mathbf{u}_t] + [\mathbf{x}_t; \mathbf{u}_t]^\top \tilde{c}_{\mathbf{xut}} + \text{const},$$

其中下标表示导数，例如 $\tilde{c}_{\mathbf{xut}}$ 是 $\tilde{c}$ 相对于 $[\mathbf{x}_t]$ 的梯度； $\mathbf{u}_t]$ ，而 $\tilde{c}_{\mathbf{xu}, \mathbf{xut}}$ 是 Hessian 矩阵。⁴ 在这个动态和成本函数模型下，

4. We assume that all Taylor expansions here are recentered around zero. Otherwise, the point around which the derivatives are computed must be subtracted from $\mathbf{x}_t$ and $\mathbf{u}_t$ in all of these equations.

optimal controller can be computed by recursively computing the quadratic Q -function and value function, starting with the last time step. These functions are given by

$$\begin{aligned} V(\mathbf{x}_t) &= \frac{1}{2} \mathbf{x}_t^T V_{\mathbf{x},\mathbf{x}t} \mathbf{x}_t + \mathbf{x}_t^T V_{\mathbf{x}t} + \text{const} \\ Q(\mathbf{x}_t, \mathbf{u}_t) &= \frac{1}{2} [\mathbf{x}_t; \mathbf{u}_t]^T Q_{\mathbf{xu},\mathbf{xut}} [\mathbf{x}_t; \mathbf{u}_t] + [\mathbf{x}_t; \mathbf{u}_t]^T Q_{\mathbf{xut}} + \text{const} \end{aligned}$$

We can express them with the following recurrence, which is computed starting at the last time step $t = T$ and moving backward through time:

$$\begin{aligned} Q_{\mathbf{xu},\mathbf{xut}} &= \tilde{c}_{\mathbf{xu},\mathbf{xut}} + f_{\mathbf{xut}}^T V_{\mathbf{x},\mathbf{x}t+1} f_{\mathbf{xut}} \\ Q_{\mathbf{xut}} &= \tilde{c}_{\mathbf{xut}} + f_{\mathbf{xut}}^T V_{\mathbf{x}t+1} + f_{\mathbf{xut}}^T V_{\mathbf{x},\mathbf{x}t+1} f_{\mathbf{ct}} \\ V_{\mathbf{x},\mathbf{x}t} &= Q_{\mathbf{x},\mathbf{x}t} - Q_{\mathbf{u},\mathbf{x}t}^T Q_{\mathbf{u},\mathbf{ut}}^{-1} Q_{\mathbf{u},\mathbf{x}t} \\ V_{\mathbf{x}t} &= Q_{\mathbf{x}t} - Q_{\mathbf{u},\mathbf{x}t}^T Q_{\mathbf{u},\mathbf{ut}}^{-1} Q_{\mathbf{u}t}, \end{aligned}$$

and the optimal control law is then given by $g(\mathbf{x}_t) = \mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t$, where $\mathbf{K}_t = -Q_{\mathbf{u},\mathbf{ut}}^{-1} Q_{\mathbf{u},\mathbf{x}t}$ and $\mathbf{k}_t = -Q_{\mathbf{u},\mathbf{ut}}^{-1} Q_{\mathbf{u}t}$. If, instead of simply minimizing the expected cost, we instead wish to optimize the maximum entropy objective in Equation (4), the optimal controller is instead linear-Gaussian, with the solution given by $p(\mathbf{u}_t | \mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t; Q_{\mathbf{u},\mathbf{ut}}^{-1})$, as shown in prior work (Levine and Koltun, 2013a).

Appendix B. Experimental Setup Details

In this appendix, we present a detailed summary of the experimental setup for our simulated and real-world experiments.

B.1 Simulated Experiment Details

All of the simulated experiments used the MuJoCo simulation package (Todorov et al., 2012), with simulated frictional contacts and torque motors at the joints used for actuation. Although no control or state noise was added during simulation, noise was injected naturally by the linear-Gaussian controllers. The linear-Gaussian controllers $p_i(\mathbf{u}_t | \mathbf{x}_t)$ were initialized to stay near the initial state $\mathbf{x}_1$ using linear feedback based on a proportional-derivative control law for all tasks, except for the octopus arm, where $p_i(\mathbf{u}_t | \mathbf{x}_t)$ was initialized to be zero mean with a fixed spherical covariance, and the walker, which was initialized to track a demonstration trajectory with proportional-derivative feedback. The walker was the only task that used a demonstration, as described previously. We describe the details of each system below.

Peg insertion: The 2D peg insertion task has 6 state dimensions (joint angles and angular velocities) and 2 action dimensions. The 3D version of the task has 12 state dimensions, since the arm has 3 degrees of freedom at the shoulder, 1 at the elbow, and 2 at the wrist. Trials were 8 seconds in length and simulated at 100 Hz, resulting in 800 time steps per rollout. The cost function is given by

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} w_{\mathbf{u}} \|\mathbf{u}_t\|^2 + w_{\mathbf{p}} \ell_{12}(\mathbf{p}_{\mathbf{x}t} - \mathbf{p}^*),$$

最优控制器可以通过从最后一个时间步开始递归计算二次 Q 函数和值函数来计算。这些函数由下式给出

$$\begin{aligned} V(\mathbf{x}_t) &= \frac{1}{2} \mathbf{x}_t^T V_{\mathbf{x}, \mathbf{x}t} \mathbf{x}_t + \mathbf{x}_t^T V_{\mathbf{x}t} + \text{const} \\ Q(\mathbf{x}_t, \mathbf{u}_t) &= \frac{1}{2} [\mathbf{x}_t; \mathbf{u}_t]^T Q_{\mathbf{xu}, \mathbf{xut}} [\mathbf{x}_t; \mathbf{u}_t] + [\mathbf{x}_t; \mathbf{u}_t]^T Q_{\mathbf{xut}} + \text{const} \end{aligned}$$

我们可以用以下递归式来表达它们，该递归式是从最后一个时间步 $t = T$ 开始计算并随着时间向后移动：

$$\begin{aligned} Q_{\mathbf{xu}, \mathbf{xut}} &= \tilde{c}_{\mathbf{xu}, \mathbf{xut}} + f_{\mathbf{xut}}^T V_{\mathbf{x}, \mathbf{x}t+1} f_{\mathbf{xut}} \\ Q_{\mathbf{xut}} &= \tilde{c}_{\mathbf{xut}} + f_{\mathbf{xut}}^T V_{\mathbf{x}t+1} + f_{\mathbf{xut}}^T V_{\mathbf{x}, \mathbf{x}t+1} f_{\mathbf{ct}} \\ V_{\mathbf{x}, \mathbf{x}t} &= Q_{\mathbf{x}, \mathbf{x}t} - Q_{\mathbf{u}, \mathbf{x}t}^T Q_{\mathbf{u}, \mathbf{ut}}^{-1} Q_{\mathbf{u}, \mathbf{x}t} \\ V_{\mathbf{x}t} &= Q_{\mathbf{x}t} - Q_{\mathbf{u}, \mathbf{x}t}^T Q_{\mathbf{u}, \mathbf{ut}}^{-1} Q_{\mathbf{ut}}, \end{aligned}$$

最优控制律由 $g(\mathbf{x}_t) = \mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t$ 给出，其中 $\mathbf{K}_t = -Q_{\mathbf{u}, \mathbf{ut}}^{-1} Q_{\mathbf{u}, \mathbf{x}t}$ 和 $\mathbf{k}_t = -Q_{\mathbf{u}, \mathbf{ut}}^{-1} Q_{\mathbf{ut}}$ 。如果我们不是简单地最小化预期成本，而是希望优化方程 (4) 中的最大熵目标，则最优控制器是线性高斯控制器，其解由 $p(\mathbf{u}_t | \mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t \mathbf{x}_t + \mathbf{k}_t; Q_{\mathbf{u}, \mathbf{ut}}^{-1})$ 给出；如之前的工作所示（Levine 和 Koltun, 2013a）。

Appendix B. Experimental Setup Details

在本附录中，我们详细总结了模拟和真实实验的实验设置。

B.1 Simulated Experiment Details

所有模拟实验均使用 MuJoCo 模拟包（Todorov 等人, 2012），并在用于驱动关节处模拟摩擦接触和扭矩电机。尽管在仿真过程中没有添加控制或状态噪声，但噪声是由线性高斯控制器自然注入的。线性高斯控制器 $p_i(\mathbf{u}_t | \mathbf{x}_t)$ 被初始化为保持在初始状态 $\mathbf{x}_1$ 附近，使用基于比例微分控制律的线性反馈，适用于所有任务，但章鱼臂除外，其中 $p_i(\mathbf{u}_t | \mathbf{x}_t)$ 被初始化为具有固定球面协方差的零均值，而步行器被初始化为通过比例微分反馈跟踪演示轨迹。如前所述，步行器是唯一使用演示的任务。我们在下面描述每个系统的详细信息。

Peg insertion: 2D 钉插入任务有 6 个状态维度（关节角度和角速度）和 2 个动作维度。该任务的 3D 版本有 12 个状态维度，因为手臂在肩部有 3 个自由度，在肘部有 1 个自由度，在手腕有 2 个自由度。试验时长 8 秒，模拟频率为 100 Hz，每次推出有 800 个时间步长。成本函数由下式给出

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2} w_{\mathbf{u}} \|\mathbf{u}_t\|^2 + w_{\mathbf{p}} \ell_{12}(\mathbf{p}_{\mathbf{x}t} - \mathbf{p}^*),$$

where $\mathbf{p}_{\mathbf{x}_t}$ is the position of the end effector for state $\mathbf{x}_t$, $\mathbf{p}^*$ is the desired end effector position at the bottom of the slot, and the norm $\ell_{12}(z)$ is given by $\frac{1}{2}\|z\|^2 + \sqrt{\alpha + z^2}$, which corresponds to the sum of an ℓ_2 and soft ℓ_1 norm. We use this norm to encourage the peg to precisely reach the target position at the bottom of the hole, but to also receive a larger penalty when far away. The task also works well in 2D with a simple ℓ_2 penalty, though we found that the 3D version of the task takes longer to insert the peg all the way into the hole without the ℓ_1 -like square root term. The weights were set to $w_{\mathbf{u}} = 10^{-6}$ and $w_{\mathbf{p}} = 1$. Initial states were chosen by moving the shoulder of the arm relative to the hole, with four equally spaced starting states in a 20 cm region for the 2D arm, and four random starting states in a 10 cm radius for the 3D arm.

Octopus arm: The octopus arm consists of six four-sided chambers. Each edge of each chamber is a simulated muscle, and actions correspond to contracting or relaxing the muscle. The state space consists of the positions and velocities of the chamber vertices. The midpoint of one edge of the first chamber is fixed, resulting in a total of 25 degrees of freedom: the 2D positions of the 12 unconstrained points, and the orientation of the first edge. Including velocities, the total dimensionality of the state space is 50. The cost function depends on the activation of the muscles and distance between the tip of the arm and the target point, in the same way as for peg insertion. The weights are set to $w_{\mathbf{u}} = 10^{-3}$ and $w_{\mathbf{p}} = 1$.

Swimmer: The swimmer consists of 3 links and 5 degrees of freedom, including the global position and orientation which, together with the velocities, produces a 10 dimensional state space. The swimmer has 2 action dimensions corresponding to the torques between joints. The simulation applied drag on each link of the swimmer to roughly simulate a fluid, allowing it to propel itself. The rollouts were 20 seconds in length at 20 Hz, resulting in 400 time steps per rollout. The cost function for the swimmer is given by

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2}w_{\mathbf{u}}\|\mathbf{u}_t\|^t + \frac{1}{2}w_v\|v_{x\mathbf{x}_t} - v_x^*\|^2$$

where $v_{x\mathbf{x}_t}$ is the horizontal velocity, $v_x^* = 2.0\text{m/s}$, and the weights were $w_{\mathbf{u}} = 2 \cdot 10^{-5}$ and $w_v = 1$.

Walker: The bipedal walker consists of a torso and two legs, each with three links, for a total of 9 degrees of freedom and 18 dimensions, with velocity, and 6 action dimensions. The simulation ran for 5 seconds at 100 Hz, for a total of 500 time steps. The cost function is given by

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2}w_{\mathbf{u}}\|\mathbf{u}_t\|^t + \frac{1}{2}w_v\|v_{x\mathbf{x}_t} - v_x^*\|^2 + \frac{1}{2}w_h\|p_{y\mathbf{x}_t} - p_y^*\|^2$$

where $v_{x\mathbf{x}_t}$ is again the horizontal velocity, $p_{y\mathbf{x}_t}$ is the vertical position of the root, $v_x^* = 2.1\text{m/s}$, $p_y^* = 1.1\text{m}$, and the weights were set to $w_{\mathbf{u}} = 10^{-4}$, $w_v = 1$, and $w_h = 1$.

其中, $\mathbf{p}_{\mathbf{x}_t}$ 是状态 $\mathbf{x}_t$ 的末端执行器位置, $\mathbf{p}^*$ 是槽底部所需的末端执行器位置, 范数 $\ell_{12}(z)$ 由 $\frac{1}{2}\|z\|^2 + \sqrt{\alpha + z^2}$ 给出, 它对应于 ℓ_2 和软 ℓ_1 范数之和。我们使用这个规范来鼓励钉子精确地到达洞底的目标位置, 但在距离较远时也会受到更大的惩罚。该任务在具有简单的 ℓ_2 惩罚的 2D 环境中也能很好地工作, 尽管我们发现该任务的 3D 版本需要更长的时间才能将钉子完全插入孔中, 而无需类似 ℓ_1 的平方根项。权重设置为 $w_{\mathbf{u}} = 10^{-6}$ 和 $w_{\mathbf{p}} = 1$ 。通过相对于孔移动手臂的肩部来选择初始状态, 对于 2D 手臂, 在 20 cm 区域内有四个等距起始状态, 对于 3D 手臂在 10 cm 半径内有四个随机起始状态。

Octopus arm: 章鱼臂由六个四边室组成。每个腔室的每个边缘都是模拟肌肉, 动作对应于收缩或放松肌肉。状态空间由腔室顶点的位置和速度组成。第一个室的一个边缘的中点是固定的, 总共产生 25 个自由度: 12 个无约束点的 2D 位置以及第一个边缘的方向。包括速度在内, 状态空间的总维数为 50。成本函数取决于肌肉的激活以及臂尖与目标点之间的距离, 与钉插入的方式相同。权重设置为 $w_{\mathbf{u}} = 10^{-3}$ 和 $w_{\mathbf{p}} = 1$ 。

Swimmer: 游泳者由 3 个连杆和 5 个自由度组成, 包括全局位置和方向, 与速度一起产生 10 维状态空间。游泳者有 2 个动作维度, 对应于关节之间的扭矩。该模拟在游泳者的每个链接上施加阻力, 以粗略地模拟流体, 使其能够自行推进。推出时间为 20 秒, 频率为 20 Hz, 每次推出有 400 个时间步长。游泳者的成本函数由下式给出

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2}w_{\mathbf{u}}\|\mathbf{u}_t\|^t + \frac{1}{2}w_v\|v_{x\mathbf{x}_t} - v_x^*\|^2$$

其中 $v_{x\mathbf{x}_t}$ 是水平速度, $v_x^* = 2.0\text{m/s}$, 权重为 $w_{\mathbf{u}} = 2 \cdot 10^{-5}$ 和 $w_v = 1$ 。

Walker: 双足步行器由一个躯干和两条腿组成, 每条腿有三个连杆, 总共有 9 个自由度和 18 个维度, 具有速度和 6 个动作维度。模拟以 100 Hz 的频率运行 5 秒, 总共 500 个时间步长。成本函数由下式给出

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = \frac{1}{2}w_{\mathbf{u}}\|\mathbf{u}_t\|^t + \frac{1}{2}w_v\|v_{x\mathbf{x}_t} - v_x^*\|^2 + \frac{1}{2}w_h\|p_{y\mathbf{x}_t} - p_y^*\|^2$$

其中, $v_{x\mathbf{x}_t}$ 又是水平速度, $p_{y\mathbf{x}_t}$ 是根的垂直位置, $v_x^* = 2.1\text{m/s}$, $p_y^* = 1.1\text{m}$, 权重设置为 $w_{\mathbf{u}} = 10^{-4}$ 、 $w_v = 1$ 和 $w_h = 1$ 。

B.2 Robotic Experiment Details

All of the robotic experiments were conducted on a PR2 robot. The robot was controlled at 20 Hz via direct effort control,⁵ and camera images were recorded using the RGB camera on a PrimeSense Carmine sensor. The images were downsampled to $240 \times 240 \times 3$. The learned policies controlled one 7 DoF arm of the robot, while the other arm was used to move objects in the scene to automatically vary the initial conditions. The camera was kept fixed in each experiment. Each episode was 5 seconds in length. For each task, the cost function required placing the object held in the gripper at a particular location (which might require, for example, to insert a shape into a shape sorting cube). The cost was given by the following equation:

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = w_{\ell_2} d_t^2 + w_{\log} \log(d_t^2 + \alpha) + w_{\mathbf{u}} \|\mathbf{u}_t\|^2,$$

where d_t is the distance between three points in the space of the end-effector and their target positions,⁶ and the weights are set to $w_{\ell_2} = 10^{-3}$, $w_{\log} = 1.0$, and $w_{\mathbf{u}} = 10^{-2}$. The quadratic term encourages moving the end-effector toward the target when it is far, while the logarithm term encourages placing it precisely at the target location, as discussed in prior work (Levine et al., 2015). The bottle cap task used an additional cost term consisting of a quadratic penalty on the difference between the wrist angular velocity and a target velocity.

For all of the tasks, we initialized all of the linear-Gaussian controllers $p_i(\mathbf{u}_t|\mathbf{x}_t)$ to stay near the initial state $\mathbf{x}_1$, with a diagonal noise covariance. The covariance of the noise was chosen to be proportional to a diagonal approximation of the inverse effective mass at each joint, as provided by the manufacturer of the PR2 robot, and the feedback controller was constructed using LQR, with an approximate linear model obtained from the same diagonal inverse mass matrix. The role of this initial controller was primarily to avoid dangerous actions during the first iteration. We discuss the particular setup for each experiment below:

Coat hanger: The coat hanger task required the robot to hang a coat hanger on a clothes rack. The coat hanger was grasped at one of two angles, about 35° apart, and the rack was positioned at three different distances from the robot during training, with differences of about 10 cm between each position. The rack was moved manually between these positions during training. A trial was considered successful if, when the coat hanger was released, it remained hanging on the rack rather than dropping to the ground.

Shape sorting cube: The shape sorting cube task required the robot to insert a red trapezoid into a trapezoidal hole on a shape sorting cube. During training, the cube was positioned at nine different positions, situated at the corners, edges, and middle of a rectangular region 16 cm $\times$ 10 cm in size. During training, the shape sorting cube was moved through the training positions by using the left arm. A trial was considered successful if the bottom face of the trapezoid was completely inside the shape sorting cube, such that if the robot were to release the trapezoid, it would fall inside the cube.

5. The PR2 robot does not provide for closed loop torque control, but instead supports an effort control interface that directly sets feedforward motor voltages. In practice, these voltages are roughly proportional to feedforward torques, but are also affected by friction and damping.

6. Three points fully define the pose of the end-effector. For the bottle cap task, which is radially symmetric, we use only two points.

B.2 Robotic Experiment Details

所有的机器人实验都是在 PR2 机器人上进行的。机器人通过直接力控制以 20 Hz 的频率进行控制,⁵ 并使用 PrimeSense Carmine 传感器上的 RGB 相机记录相机图像。图像被下采样到 $240 \times 240 \times 3$ 。学习的策略控制机器人的一个 7 DoF 手臂, 而另一只手臂用于移动场景中的对象以自动改变初始条件。每次实验中相机都保持固定。每集时长 5 秒。对于每项任务, 成本函数需要将夹具中的物体放置在特定位置 (例如, 这可能需要将形状插入形状排序立方体中)。成本由以下等式给出:

$$\ell(\mathbf{x}_t, \mathbf{u}_t) = w_{\ell_2} d_t^2 + w_{\log} \log(d_t^2 + \alpha) + w_{\mathbf{u}} \|\mathbf{u}_t\|^2,$$

其中 d_t 是末端执行器空间中的三个点与其目标位置⁶ 之间的距离, 权重设置为 $w_{\ell_2} = 10^{-3}$ 、 $w_{\log} = 1.0$ 和 $w_{\mathbf{u}} = 10^{-2}$ 。二次项鼓励末端执行器在距离较远时将其移向目标, 而对数项则鼓励将其精确地放置在目标位置, 正如之前的工作中所讨论的那样 (Levine et al., 2015)。瓶盖任务使用了一个额外的成本项, 其中包括对手腕角速度和目标速度之间的差异进行二次惩罚。

对于所有任务, 我们初始化所有线性高斯控制器 $p_i(\mathbf{u}_t|\mathbf{x}_t)$ 以保持在初始状态 $\mathbf{x}_1$ 附近, 并具有对角噪声协方差。噪声的协方差被选择与每个关节处的逆有效质量的对角线近似成正比, 如 PR2 机器人制造商所提供的, 并且反馈控制器使用 LQR 构建, 并从相同的对角线逆质量矩阵获得近似线性模型。这个初始控制器的作用主要是避免第一次迭代期间的危险行为。我们在下面讨论每个实验的特定设置:

Coat hanger: 衣架任务要求机器人将衣架挂在衣架上。在训练过程中, 以两个角度之一抓住衣架, 间隔约 35° , 并将架子放置在距机器人三个不同的距离处, 每个位置之间的距离相差约 10 厘米。在训练期间, 机架在这些位置之间手动移动。如果当衣架被释放时, 它仍然悬挂在衣架上而不是掉到地上, 则认为试验是成功的。

Shape sorting cube: 形状分类立方体任务要求机器人将红色梯形插入形状分类立方体上的梯形孔中。在训练过程中, 立方体被放置在九个不同的位置, 位于大小为 $16 \text{ cm} \times 10 \text{ cm}$ 的矩形区域的角、边缘和中间。在训练期间, 使用左臂将形状分类立方体移动到训练位置。如果梯形的底面完全位于形状排序立方体内部, 则试验被认为是成功的, 这样如果机器人释放梯形, 它就会落入立方体内部。

5. The PR2 robot does not provide for closed loop torque control, but instead supports an effort control interface that directly sets feedforward motor voltages. In practice, these voltages are roughly proportional to feedforward torques, but are also affected by friction and damping.

6. Three points fully define the pose of the end-effector. For the bottle cap task, which is radially symmetric, we use only two points.

Toy hammer: The hammer task required the robot to insert the claw of a toy hammer underneath a toy plastic nail, placing the claw around the base of the nail. The hammer was grasped at one of three angles, each 22.5° apart, for a total variation of 45° degrees, and the nail was positioned at five positions, at the corners and center of a rectangular region $10\text{ cm} \times 7\text{ cm}$ in size. During training, the toy tool bench containing the nail was moved using the left arm. A trial was considered successful if the tip of the claw of the hammer was at least under the centerline of the nail.

Bottle cap: The bottle cap task required the robot to screw a cap onto a bottle at various positions. The bottle was located at nine different positions, situated at the corners, edges, and middle of a rectangular region $16\text{ cm} \times 10\text{ cm}$ in size, and the left arm was used to move the bottle through the training positions. A trial was considered successful if, after completion, the cap could not be removed from bottle simply by pulling vertically.

References

- J. A. Bagnell and J. Schneider. Covariant policy search. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2003.
- B. Bakker, V. Zhumatiy, G. Gruener, and J. Schmidhuber. A robot that reinforcement-learns to identify and memorize important previous observations. In *International Conference on Intelligent Robots and Systems (IROS)*, 2003.
- G. Bekey and K. Goldberg. *Neural Networks in Robotics*. Springer US, 1992.
- H. Benbrahim and J. A. Franklin. Biped dynamic walking using reinforcement learning. *Robotics and Autonomous Systems*, 22:283–302, 1997.
- W. Böhmer, S. Grünwälder, Y. Shen, M. Musial, and K. Obermayer. Construction of approximation spaces for reinforcement learning. *Journal of Machine Learning Research*, 14(1):2067–2118, January 2013.
- S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1122, 2011.
- D. Ciresan, U. Meier, J. Masci, L. Gambardella, and J. Schmidhuber. Flexible, high performance convolutional neural networks for image classification. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2011.
- D. Ciresan, U. Meier, and J. Schmidhuber. Multi-column deep neural networks for image classification. In *Computer Vision and Pattern Recognition (CVPR)*, 2012.
- M. Deisenroth and C. Rasmussen. PILCO: a model-based and data-efficient approach to policy search. In *International Conference on Machine Learning (ICML)*, 2011.
- M. Deisenroth, C. Rasmussen, and D. Fox. Learning to control a low-cost manipulator using data-efficient reinforcement learning. In *Robotics: Science and Systems (RSS)*, 2011.

Toy hammer: 锤子任务要求机器人将玩具锤的爪子插入玩具塑料钉子下面，将爪子放在钉子的底部周围。从三个角度之一抓住锤子，每个角度相距 22.5° ，总变化为 45° 度，并将钉子放置在五个位置，即大小为 10 厘米 $\times$ 7 厘米的矩形区域的角和中心。在训练过程中，用左臂移动装有钉子的玩具工具台。如果锤子的爪尖至少位于钉子的中心线下方，则认为试验成功。

Bottle cap: 瓶盖任务要求机器人在不同位置将瓶盖拧到瓶子上。瓶子位于 9 个不同的位置，位于 16 cm $\times$ 10 cm 大小的矩形区域的角、边缘和中间，左臂用于将瓶子移动通过训练位置。如果试验完成后，仅通过垂直拉动无法将瓶盖从瓶子上取下，则认为试验成功。

References

J.A.巴格内尔和J.施奈德。协变策略搜索。 *International Joint Conference on Artificial Intelligence (IJCAI)*, 2003 年。B. Bakker、V. Zhumatiy、G. Gruener 和 J. Schmidhuber。一种强化学习识别和记忆先前重要观察结果的机器人。 *International Conference on Intelligent Robots and Systems (IROS)*, 2003 年。G. Bekey 和 K. Goldberg。 *Neural Networks in Robotics*。 Springer US, 1992。H. Benbrahim 和 J. A. Franklin。使用强化学习的两足动物动态行走。 *Robotics and Autonomous Systems*, 22:283–302, 1997。W. B öhmer、S. Grünewälder、Y. Shen、M. Musial 和 K. Obermayer。构建强化学习的近似空间。 *Journal of Machine Learning Research*, 14(1): 2067–2118, 2013 年 1 月。S. Boyd、N. Parikh、E. Chu、B. Peleato 和 J. Eckstein。通过乘法器的交替方向方法进行分布式优化和统计学习。 *Foundations and Trends in Machine Learning*, 3(1):1122, 2011。D. Ciresan、U. Meier、J. Masci、L. Gambardella 和 J. Schmidhuber。用于图像分类的灵活、高性能的卷积神经网络。 *International Joint Conference on Artificial Intelligence (IJCAI)*, 2011 年。D. Ciresan、U. Meier 和 J. Schmidhuber。用于图像分类的多列深度神经网络。 *Computer Vision and Pattern Recognition (CVPR)*, 2012 年。M. Deisenroth 和 C. Rasmussen。PILCO：一种基于模型和数据高效的策略搜索方法。 *International Conference on Machine Learning (ICML)*, 2011 年。M. Deisenroth、C. Rasmussen 和 D. Fox。使用数据高效的强化学习来学习控制低成本操纵器。在 *Robotics: Science and Systems (RSS)*, 2011 年。

- M. Deisenroth, G. Neumann, and J. Peters. A survey on policy search for robotics. *Foundations and Trends in Robotics*, 2(1-2):1–142, 2013.
- J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Computer Vision and Pattern Recognition (CVPR)*, 2009.
- G. Endo, J. Morimoto, T. Matsubara, J. Nakanishi, and G. Cheng. Learning CPG-based biped locomotion with a policy gradient method: Application to a humanoid robot. *International Journal of Robotic Research*, 27(2):213–228, 2008.
- I. Endres and D. Hoiem. Category independent object proposals. In *European Conference on Computer Vision (ECCV)*. 2010.
- Y. Engel, P. Szabó, and D. Volkinshtein. Learning to control an octopus arm with Gaussian process temporal difference methods. In *Advances in Neural Information Processing Systems (NIPS)*, 2005.
- B. Espiau, F. Chaumette, and P. Rives. A new approach to visual servoing in robotics. *IEEE Transactions on Robotics and Automation*, 8(3), 1992.
- C. Finn, X. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel. Learning visual feature spaces for robotic manipulation with deep spatial autoencoders. *arXiv preprint arXiv:1509.06113*, 2015.
- K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36:193–202, 1980.
- T. Geng, B. Porr, and F. Wörgötter. Fast biped walking with a reflexive controller and realtime policy searching. In *Advances in Neural Information Processing Systems (NIPS)*, 2006.
- R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014a.
- R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014b.
- V. Gullapalli. A stochastic reinforcement learning algorithm for learning real-valued functions. *Neural Networks*, 3(6):671–692, 1990.
- V. Gullapalli. Skillful control under uncertainty via direct reinforcement learning. *Reinforcement Learning and Robotics*, 15(4):237–246, 1995.
- X. Guo, S. Singh, H. Lee, R. L. Lewis, and X. Wang. Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.

- M. Deisenroth、G. Neumann 和 J. Peters。关于机器人政策搜索的调查。 *Foundations and Trends in Robotics*, 2(1-2): 1–142, 2013。
- J. Deng、W. Dong、R. Socher、L. Li、K. Li 和 L. Fei-Fei。ImageNet: 大规模分层图像数据库。在 *Computer Vision and Pattern Recognition (CVPR)*, 2009 年。
- G. Endo、J. Morimoto、T. Matsubara、J. Nakanishi 和 G. Cheng。使用策略梯度方法学习基于 CPG 的双足运动: 在人形机器人中的应用。 *International Journal of Robotic Research*, 27(2): 213–228, 2008。
- I.恩德烈斯和D.霍伊姆。类别独立对象提案。在 *European Conference on Computer Vision (ECCV)* 中。2010年。
- Y. Engel、P. Szabó 和 D. Volkinshtein。学习用高斯过程时间差分方法控制章鱼臂。在 *Advances in Neural Information Processing Systems (NIPS)*, 2005 年。
- B.埃斯皮奥、F.肖梅特和 P.里夫斯。机器人视觉伺服的新方法。 *IEEE Transactions on Robotics and Automation*, 8(3), 1992。
- C. Finn、X. Tan、Y. Duan、T. Darrell、S. Levine 和 P. Abbeel。使用深度空间自动编码器学习用于机器人操作的视觉特征空间。 *arXiv preprint arXiv:1509.06113*, 2015 年。
- K.福岛。Neocognitron: 一种自组织神经网络模型, 用于不受位置变化影响的模式识别机制。 *Biological Cybernetics*, 36: 193-202, 1980。
- T. Geng、B. Porr 和 F. Wörgötter。使用反射控制器和实时策略搜索进行快速双足行走。在 *Advances in Neural Information Processing Systems (NIPS)*, 2006 年。
- R. Girshick、J. Donahue、T. Darrell 和 J. Malik。丰富的特征层次结构, 用于准确的对象检测和语义分割。在 *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014a。
- R. Girshick、J. Donahue、T. Darrell 和 J. Malik。丰富的特征层次结构, 用于准确的对象检测和语义分割。在 *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014b。
- V.古拉帕利。用于学习实值函数的随机强化学习算法。 *Neural Networks*, 3(6): 671–692, 1990。V. Gullapalli。通过直接强化学习在不确定性下进行熟练的控制。 *Reinforcement Learning and Robotics*, 15(4): 237–246, 1995。X.Guo、S.Singh、H.Lee、R.L. Lewis 和 X.Wang。使用离线蒙特卡罗树搜索规划进行实时 Atari 游戏的深度学习。在 *Advances in Neural Information Processing Systems (NIPS)*, 2014 年。

- R. Hadsell, P. Sermanet, J. B. A. Erkan, and M. Scoffier. Learning long-range vision for autonomous off-road driving. *Journal of Field Robotics*, pages 120–144, 2009.
- K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.
- S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies. In *A Field Guide to Dynamic Recurrent Neural Networks*. IEEE Press, 2001.
- K. J. Hunt, D. Sbarbaro, R. Żbikowski, and P. J. Gawthrop. Neural networks for control systems: A survey. *Automatica*, 28(6):1083–1112, November 1992.
- D. Jacobson and D. Mayne. *Differential Dynamic Programming*. Elsevier, 1970.
- M. Jägersand, O. Fuentes, and R. C. Nelson. Experimental evaluation of uncalibrated visual servoing for precision manipulation. In *International Conference on Robotics and Automation (ICRA)*, 1997.
- Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv preprint arXiv:1408.5093*, 2014.
- S. Jodogne and J. H. Piater. Closed-loop learning of visual control policies. *Journal of Artificial Intelligence Research*, 28:349–391, 2007.
- R. Jonschkowski and O. Brock. State representation learning in robotics: Using prior knowledge about physical interaction. In *Proceedings of Robotics: Science and Systems*, 2014.
- M. Kalakrishnan, L. Righetti, P. Pastor, and S. Schaal. Learning force control policies for compliant manipulation. In *International Conference on Intelligent Robots and Systems (IROS)*, 2011.
- S. M. Khansari-Zadeh and A. Billard. BM: An iterative algorithm to learn stable non-linear dynamical systems with Gaussian mixture models. In *International Conference on Robotics and Automation (ICRA)*, 2010.
- J. Kober and J. Peters. Learning motor primitives for robotics. In *International Conference on Robotics and Automation (ICRA)*, 2009.
- J. Kober, K. Muelling, O. Kroemer, C.H. Lampert, B. Schoelkopf, and J. Peters. Movement templates for learning of hitting and batting. In *International Conference on Robotics and Automation (ICRA)*, 2010a.
- J. Kober, E. Oztop, and J. Peters. Reinforcement learning to adjust robot movements to new situations. In *Robotics: Science and Systems (RSS)*, 2010b.
- J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *International Journal of Robotic Research*, 32(11):1238–1274, 2013.

- R. Hadsell、P. Sermanet、J. B. A. Erkan 和 M. Scoffier。学习自动越野驾驶的远距离视觉。 *Journal of Field Robotics*, 第 120-144 页, 2009 年。
- K. He、X. 张、S. Ren 和 J. Sun。用于图像识别的深度残差学习。 *arXiv preprint arXiv:1512.03385*, 2015 年。
- S. Hochreiter、Y. Bengio、P. Frasconi 和 J. Schmidhuber。循环网络中的梯度流：学习长期依赖性的困难。在 *A Field Guide to Dynamic Recurrent Neural Networks* 中。IEEE 出版社, 2001 年。
- K. J. Hunt、D. Sbarbaro、R. Żbikowski 和 P. J. Gawthrop。控制系统的神经网络：一项调查。 *Automatica*, 28(6): 1083–1112, 1992 年 11 月。D. 雅各布森和 D. 梅恩。 *Differential Dynamic Programming*。爱思唯尔, 1970。
- M. Jägersand、O. Fuentes 和 R. C. Nelson。用于精密操纵的未校准视觉伺服的实验评估。在 *International Conference on Robotics and Automation (ICRA)*, 1997 年。
- Y. Jia、E. Shelhamer、J. Donahue、S. Karayev、J. Long、R. Girshick、S. Guadarrama 和 T. Darrell。Caffe：用于快速特征嵌入的卷积架构。 *arXiv preprint arXiv:1408.5093*, 2014 年。
- S. Jodogne 和 J. H. Piater。视觉控制策略的闭环学习。 *Journal of Artificial Intelligence Research*, 28:349–391, 2007。
- R. Jonschkowski 和 O. Brock。机器人技术中的状态表示学习：使用有关物理交互的先验知识。在 *Proceedings of Robotics: Science and Systems*, 2014 年。
- M. Kalakrishnan、L. Righetti、P. Pastor 和 S. Schaal。学习合规操纵的力控制策略。在 *International Conference on Intelligent Robots and Systems (IROS)*, 2011 年。
- S. M. Khansari-Zadeh 和 A. Billard。BM：一种使用高斯混合模型学习稳定非线性动力系统的迭代算法。在 *International Conference on Robotics and Automation (ICRA)*, 2010 年。
- J. 科伯和 J. 彼得斯。学习机器人的运动原语。在 *International Conference on Robotics and Automation (ICRA)*, 2009 年。
- J. Kober、K. Muelling、O. Kroemer、C. H. 兰伯特、B. 舍尔科普夫和 J. 彼得斯。用于学习击球和击球的动作模板。在 *International Conference on Robotics and Automation (ICRA)*, 2010a。
- J. Kober、E. Oztop 和 J. Peters。强化学习以调整机器人动作以适应新情况。在 *Robotics: Science and Systems (RSS)*, 2010b。J. 科伯、J. A. 巴格内尔和 J. 彼得斯。机器人技术中的强化学习：一项调查。 *International Journal of Robotic Research*, 32(11): 1238–1274, 2013。

- N. Kohl and P. Stone. Policy gradient reinforcement learning for fast quadrupedal locomotion. In *International Conference on Robotics and Automation (IROS)*, 2004.
- J. Koutník, G. Cuccu, J. Schmidhuber, and F. Gomez. Evolving large-scale neural networks for vision-based reinforcement learning. In *Conference on Genetic and Evolutionary Computation, GECCO '13*, 2013.
- A. Krizhevsky, I. Sutskever, and G. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*. 2012.
- T. Lampe and M. Riedmiller. Acquiring visual servoing reaching and grasping skills using neural reinforcement learning. In *International Joint Conference on Neural Networks (IJCNN)*, 2013.
- A. Lanfranco, A. Castellanos, J. Desai, and W. Meyers. Robotic surgery: a current perspective. *Annals of surgery*, 239(1):14, 2004.
- S. Lange, M. Riedmiller, and A. Voigtlaender. Autonomous reinforcement learning on raw visual input data in a real world application. In *International Joint Conference on Neural Networks*, 2012.
- Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Handwritten digit recognition with a back-propagation network. In *Advances in Neural Information Processing Systems (NIPS)*, 1989.
- Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521:436–444, May 2015.
- H. Lee, R. Grosse, R. Ranganath, and A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *International Conference on Machine Learning (ICML)*, 2009.
- Ian Lenz, Ross Knepper, and Ashutosh Saxena. DeepMPC: Learning deep latent features for model predictive control. In *RSS*, 2015a.
- Ian Lenz, Honglak Lee, and Ashutosh Saxena. Deep learning for detecting robotic grasps. *IJRR*, 2015b.
- S. Levine and P. Abbeel. Learning neural network policies with guided policy search under unknown dynamics. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- S. Levine and V. Koltun. Guided policy search. In *International Conference on Machine Learning (ICML)*, 2013a.
- S. Levine and V. Koltun. Variational policy search via trajectory optimization. In *Advances in Neural Information Processing Systems (NIPS)*, 2013b.
- S. Levine and V. Koltun. Learning complex neural network policies with trajectory optimization. In *International Conference on Machine Learning (ICML)*, 2014.
- S. Levine, N. Wagener, and P. Abbeel. Learning contact-rich manipulation skills with guided policy search. In *International Conference on Robotics and Automation (ICRA)*, 2015.

- N.科尔和P.斯通。快速四足运动的策略梯度强化学习。在 *International Conference on Robotics and Automation (IROS)*, 2004 年。
- J. Koutník、G. Cuccu、J. Schmidhuber 和 F. Gomez。发展大规模神经网络以实现基于视觉的强化学习。在 *Conference on Genetic and Evolutionary Computation*, GECCO '13, 2013 年。
- A. Krizhevsky、I. Sutskever 和 G. Hinton。使用深度卷积神经网络进行 ImageNet 分类。在 *Advances in Neural Information Processing Systems (NIPS)* 中。2012 年。
- T. Lampe 和 M. Riedmiller。使用神经强化学习获得视觉伺服到达和抓取技能。在 *International Joint Conference on Neural Networks (IJCNN)*, 2013 年。
- A. Lanfranco、A. Castellanos、J. Desai 和 W. Meyers。机器人手术：当前的观点。 *Annals of surgery*, 239(1):14, 2004。
- S. Lange、M. Riedmiller 和 A. Voigtlaender。在现实世界应用中对原始视觉输入数据进行自主强化学习。在 *International Joint Conference on Neural Networks*, 2012 年。
- Y. LeCun、B. Boser、J. S. Denker、D. Henderson、R. E. Howard、W. Hubbard 和 L. D. Jackel。使用反向传播网络进行手写数字识别。在 *Advances in Neural Information Processing Systems (NIPS)*, 1989 年。
- Y. LeCun、Y. Bengio 和 G. Hinton。深度学习。 *Nature*, 521: 436–444, 2015 年 5 月。
- H. Lee、R. Grosse、R. Ranganath 和 A. Y. Ng。用于分层表示的可扩展无监督学习的卷积深度信念网络。在 *International Conference on Machine Learning (ICML)*, 2009 年。
- 伊恩·伦茨、罗斯·克内珀和阿舒托什·萨克塞纳。DeepMPC：学习模型预测控制的深层潜在特征。在 *RSS*, 2015a。
- 伊恩·伦茨、洪拉克·李和阿舒托什·萨克塞纳。用于检测机器人抓取的深度学习。 *IJRR*, 2015b。
- S. 莱文和P.阿贝尔。在未知动态下通过引导策略搜索来学习神经网络策略。 *Advances in Neural Information Processing Systems (NIPS)*, 2014 年。S. Levine 和 V. Koltun。引导政策搜索。在 *International Conference on Machine Learning (ICML)*, 2013a。
- S. Levine 和 V. Koltun。通过轨迹优化进行变分策略搜索。在 *Advances in Neural Information Processing Systems (NIPS)*, 2013b。
- S. Levine 和 V. Koltun。通过轨迹优化学习复杂的神经网络策略。在 *International Conference on Machine Learning (ICML)*, 2014 年。
- S. Levine、N. Wagener 和 P. Abbeel。通过引导策略搜索来学习接触丰富的操作技能。在 *International Conference on Robotics and Automation (ICRA)*, 2015 年。

- F. L. Lewis, A. Yesildirak, and S. Jagannathan. *Neural Network Control of Robot Manipulators and Nonlinear Systems*. Taylor & Francis, Inc., 1998.
- W. Li and E. Todorov. Iterative linear quadratic regulator design for nonlinear biological movement systems. In *ICINCO (1)*, pages 222–229, 2004.
- T. Lillicrap, J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- R. Lioutikov, A. Paraschos, G. Neumann, and J. Peters. Sample-based information-theoretic stochastic optimal control. In *International Conference on Robotics and Automation*, 2014.
- H. Mayer, F. Gomez, D. Wierstra, I. Nagy, A. Knoll, and J. Schmidhuber. A system for robotic heart surgery that learns to tie knots using recurrent neural networks. In *International Conference on Intelligent Robots and Systems (IROS)*, 2006.
- W. Meeussen, M. Wise, S. Glaser, S. Chitta, C. McGann, P. Mihelich, E. Marder-Eppstein, M. Muja, Victor Eruhimov, T. Foote, J. Hsu, R.B. Rusu, B. Marthi, G. Bradski, K. Konolige, B. Gerkey, and E. Berger. Autonomous door opening and plugging in with a personal robot. In *International Conference on Robotics and Automation (ICRA)*, 2010.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with deep reinforcement learning. *NIPS '13 Workshop on Deep Learning*, 2013.
- K. Mohta, V. Kumar, and K. Daniilidis. Vision based control of a quadrotor for perching on planes and lines. In *International Conference on Robotics and Automation (ICRA)*, 2014.
- I. Mordatch and E. Todorov. Combining the benefits of function approximation and trajectory optimization. In *Robotics: Science and Systems (RSS)*, 2014.
- A. Y. Ng, H. J. Kim, M. I. Jordan, and S. Sastry. Inverted autonomous helicopter flight via reinforcement learning. In *International Symposium on Experimental Robotics*, 2004.
- R. Pascanu and Y. Bengio. On the difficulty of training recurrent neural networks. Technical Report arXiv:1211.5063, Universite de Montreal, 2012.
- B. Pepik, M. Stark, P. Gehler, and B. Schiele. Teaching 3D geometry to deformable part models. In *Computer Vision and Pattern Recognition (CVPR)*, 2012.
- J. Peters and S. Schaal. Applying the episodic natural actor-critic architecture to motor primitive learning. In *European Symposium on Artificial Neural Networks (ESANN)*, 2007.
- J. Peters and S. Schaal. Reinforcement learning of motor skills with policy gradients. *Neural Networks*, 21(4):682–697, 2008.

- F. L. Lewis、A. Yesildirak 和 S. Jagannathan。 *Neural Network Control of Robot Manipulators and Nonlinear Systems*。 泰勒和弗朗西斯公司, 1998。
- W. Li 和 E. Todorov。 非线性生物运动系统的迭代线性二次调节器设计。 *ICINCO (1)*, 第 222-229 页, 2004 年。
- T. Lillicrap、J. Hunt、A. Pritzel、N. Heess、T. Erez、Y. Tassa、D. Silver 和 D. Wierstra。 通过深度强化学习进行持续控制。 *arXiv preprint arXiv:1509.02971*, 2015。
- R. Lioutikov、A. Paraschos、G. Neumann 和 J. Peters。 基于样本的信息论随机最优控制。在 *International Conference on Robotics and Automation*, 2014 年。
- H. Mayer、F. Gomez、D. Wierstra、I. Nagy、A. Knoll 和 J. Schmidhuber。 一种用于机器人心脏手术的系统, 可以使用循环神经网络学习打结。在 *International Conference on Intelligent Robots and Systems (IROS)*, 2006 年。
- W. Meeussen、M. Wise、S. Glaser、S. Chitta、C. McGann、P. Mihelich、E. Marder-Eppstein、M. Muja、Victor Eruhimov、T. Foote、J. Hsu、R.B. Rusu、B. Marthi、G. Bradski、K. Konolige、B. Gerkey 和 E. Berger。 使用个人机器人自动开门和插入电源。在 *International Conference on Robotics and Automation (ICRA)*, 2010 年。
- V. Mnih、K. Kavukcuoglu、D. Silver、A. Graves、I. Antonoglou、D. Wierstra 和 M. Riedmiller。 通过深度强化学习玩 Atari。 *NIPS '13 Workshop on Deep Learning*, 2013 年。
- K. Mohta、V. Kumar 和 K. Daniilidis。 基于视觉的四旋翼飞行器栖息在平面和直线上的控制。在 *International Conference on Robotics and Automation (ICRA)*, 2014 年。
- I. 莫达奇和E. 托多罗夫。 结合函数逼近和轨迹优化的优点。在 *Robotics: Science and Systems (RSS)*, 2014 年。
- A. Y. Ng、H. J. Kim、M. I. Jordan 和 S. Sastry。 通过强化学习实现倒转自主直升机飞行。在 *International Symposium on Experimental Robotics*, 2004 年。
- R. 帕斯卡努和Y. 本吉奥。 关于训练循环神经网络的困难。技术报告 arXiv:1211.5063, 蒙特利尔大学, 2012 年。
- B. 佩皮克、M. 斯塔克、P. 盖勒和 B. 席勒。 向可变形零件模型教授 3D 几何。 *Computer Vision and Pattern Recognition (CVPR)*, 2012 年。 J. Peters 和 S. Schaal。 将情景自然演员批评家架构应用于运动原始学习。在 *European Symposium on Artificial Neural Networks (ESANN)*, 2007 年。
- J. 彼得斯和S. 沙尔。 通过策略梯度强化运动技能学习。 *Neural Networks*, 21(4): 682–697, 2008。

- J. Peters, K. Mülling, and Y. Altün. Relative entropy policy search. In *AAAI Conference on Artificial Intelligence*, 2010.
- Lerrel Pinto and Abhinav Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. *CoRR*, abs/1509.06825, 2015.
- D. Pomerleau. ALVINN: an autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems (NIPS)*, 1989.
- S. Ross, G. Gordon, and A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *Journal of Machine Learning Research*, 15:627–635, 2011.
- S. Ross, N. Melik-Barkhudarov, K. Shaurya Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert. Learning monocular reactive UAV control in cluttered natural environments. In *International Conference on Robotics and Automation (ICRA)*, 2013.
- R. Rubinstein and D. Kroese. *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.
- S. Savarese and L. Fei-Fei. 3D generic object categorization, localization and pose estimation. In *International Conference on Computer Vision (ICCV)*, 2007.
- J. Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61: 85–117, 2015.
- P. Y. Simard, D. Steinkraus, and J. C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Seventh International Conference on Document Analysis and Recognition*, 2003.
- F. Stulp and O. Sigaud. Path integral policy improvement with covariance matrix adaptation. In *International Conference on Machine Learning (ICML)*, 2012.
- Jaeyong Sung, Seok Hyun Jin, and Ashutosh Saxena. Robobarista: Object part based transfer of manipulation trajectories from crowd-sourcing in 3d pointclouds. *CoRR*, abs/1504.03071, 2015.
- C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *arXiv preprint arXiv:1409.4842*, 2014.
- R. Tedrake, T. Zhang, and H. Seung. Stochastic policy gradient reinforcement learning on a simple 3d biped. In *International Conference on Intelligent Robots and Systems (IROS)*, 2004.
- E. Theodorou, J. Buchli, and S. Schaal. Reinforcement learning of motor skills in high dimensions. In *International Conference on Robotics and Automation (ICRA)*, 2010.
- E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

- J. Peters、K. Mülling 和 Y. Altün。相对熵策略搜索。 *AAAI Conference on Artificial Intelligence*, 2010 年。Lerrel Pinto 和 Abhinav Gupta。超大型自我监督：从 50 k 次尝试和 700 个机器人小时中学习抓取。 *CoRR*, abs/1509.06825, 2015。D. Pomerleau。ALVINN：神经网络中的自主陆地车辆。 *Advances in Neural Information Processing Systems (NIPS)*, 1989 年。S. Ross、G. Gordon 和 A. Bagnell。将模仿学习和结构化预测减少为无悔的在线学习。 *Journal of Machine Learning Research*, 15: 627-635, 2011。
- S. Ross、N. Melik-Barkhudarov、K. Shaurya Shankar、A. Wendel、D. Dey、J. A. Bagnell 和 M. Hebert。在杂乱的自然环境中学习单目反应式无人机控制。在 *International Conference on Robotics and Automation (ICRA)*, 2013 年。
- R. 鲁宾斯坦和 D. 罗斯。 *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*。施普林格, 2004。
- S. Savarese 和 L. Fei-Fei。3D 通用对象分类、定位和姿态估计。在 *International Conference on Computer Vision (ICCV)*, 2007 年。
- J. 施米德胡贝尔。神经网络中的深度学习：概述。 *Neural Networks*, 61: 85-117, 2015。
- P. Y. Simard、D. Steinkraus 和 J. C. Platt。卷积神经网络应用于视觉文档分析的最佳实践。在 *Seventh International Conference on Document Analysis and Recognition*, 2003 年。
- F. Stulp 和 O. Sigaud。协方差矩阵自适应的路径积分策略改进。在 *International Conference on Machine Learning (ICML)*, 2012 年。
- Jaeyong Sung、Seok Hyun Jin 和 Ashutosh Saxena。Robobarista：基于对象部分的 3D 点云众包操作轨迹传输。 *CoRR*, abs/1504.03071, 2015。
- C. Szegedy、W. Liu、Y. Jia、P. Sermanet、S. Reed、D. Anguelov、D. Erhan、V. Vanhoucke 和 A. Rabinovich。更深入地了解卷积。 *arXiv preprint arXiv:1409.4842*, 2014 年。
- R. Tedrake、T. Zhang 和 H. Seung。简单 3D Biped 上的随机策略梯度强化学习。在 *International Conference on Intelligent Robots and Systems (IROS)*, 2004 年。
- E. Theodorou、J. Buchli 和 S. Schaal。高维度运动技能的强化学习。 *International Conference on Robotics and Automation (ICRA)*, 2010 年。E. Todorov、T. Erez 和 Y. Tassa。MuJoCo：用于基于模型的控制的物理引擎。在 *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012 年。

- J. J. Tompson, A. Jain, Y. LeCun, and C. Bregler. Joint training of a convolutional network and a graphical model for human pose estimation. In *Advances in Neural Information Processing Systems (NIPS)*, 2014.
- J. Uijlings, K. van de Sande, T. Gevers, and A. Smeulders. Selective search for object recognition. *International Journal of Computer Vision*, 2013.
- H. van Hoof, J. Peters, and G. Neumann. Learning of non-parametric control policies with high-dimensional state features. In *International Conference on Artificial Intelligence and Statistics*, 2015.
- H. Wang and A. Banerjee. Bregman alternating direction method of multipliers. In *Advances in Neural Information Processing Systems (NIPS)*. 2014.
- M. Watter, J. Springenberg, J. Boedecker, and M. Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advanced in Neural Information Processing Systems (NIPS)*, 2015.
- R. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, May 1992.
- W. J. Wilson, C. W. Williams Hulls, and G. S. Bell. Relative end-effector control using cartesian position based visual servoing. *IEEE Transactions on Robotics and Automation*, 12(5), 1996.
- K.A. Wyrobek, E.H. Berger, HF M. Van der Loos, and K. Salisbury. Towards a personal robotics development platform: Rationale and design of an intrinsically safe personal robot. In *International Conference on Robotics and Automation (ICRA)*, 2008.
- B. H. Yoshimi and P. K. Allen. Active, uncalibrated visual servoing. In *International Conference on Robotics and Automation (ICRA)*, 1994.

J. J. 汤普森、A. Jain、Y. LeCun 和 C. Bregler。用于人体姿势估计的卷积网络和图形模型的联合训练。 *Advances in Neural Information Processing Systems (NIPS)*, 2014 年。J. Uijlings、K. van de Sande、T. Gevers 和 A. Smeulders。选择性搜索对象识别。 *International Journal of Computer Vision*, 2013。H. van Hoof、J. Peters 和 G. Neumann。学习具有高维状态特征的非参数控制策略。 *International Conference on Artificial Intelligence and Statistics*, 2015 年。H. Wang 和 A. Banerjee。布雷格曼乘法器交替方向法。在 *Advances in Neural Information Processing Systems (NIPS)* 中。2014 年。M. Watter、J. Springenberg、J. Boedecker 和 M. Riedmiller。嵌入控制：用于从原始图像进行控制的局部线性潜在动力学模型。在 *Advanced in Neural Information Processing Systems (NIPS)*, 2015 年。R. Williams。用于联结强化学习的简单统计梯度跟踪算法。 *Machine Learning*, 8(3-4): 229–256, 1992 年 5 月。W. J. Wilson、C. W. Williams Hulls 和 G. S. Bell。使用基于笛卡尔位置的视觉伺服的相对末端执行器控制。 *IEEE Transactions on Robotics and Automation*, 12(5), 1996。

K.A. 维罗贝克、E.H. 伯格、H.F. M. 范德洛斯和 K. 索尔兹伯里。走向个人机器人开发平台：本质安全个人机器人的基本原理和设计。在 *International Conference on Robotics and Automation (ICRA)*, 2008 年。

B. H. Yoshimi 和 P. K. Allen。主动、未校准的视觉伺服。在 *International Conference on Robotics and Automation (ICRA)*, 1994 年。